HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and communications technology

Software Design Document

Version 1.3

An Internet Media Store

Subject: ITSS Software Development

Group 6

Vũ Hải Đăng-20225962

Nguyễn Minh Khôi-20226050

Lê Đại Lâm-20225982

Phạm Thành Nam-20225989

Bùi Xuân Sơn-20226065

Hanoi, 05, 2025

<All notations inside the angle bracket are not part of this document, for its purpose is for extra instruction. When using this document, please erase all these notations and/or replace them with corresponding content as instructed>

<This document, written by Prof. NGUYEN Thi Thu Trang, is used as a case study for student with related courses. Any modifications and/or utilization without the consent of the author is strictly forbidden>

Table of Contents

List of Figures

No table of figures entries found.

List of Tables

• Introduction

The following subsections of the Software Design Document (SDD) provide an overview of the architectural and design specifications for AIMS: An Internet Media Store. This document translates the functional requirements defined in the Software Requirements Specification (SRS) into technical design components and implementation guidelines.

• *Objective*

The objective of this Software Design Document is to provide detailed architectural and design specifications for the AIMS: An Internet Media Store system. This document serves as a technical blueprint that outlines the system's structural organization, component interactions, data models, and implementation strategies.

The intended audience includes:

- **Software Architects**: To validate the overall system architecture and design patterns
- **Software Engineers/Developers**: To implement the system components according to specified designs
- **Database Administrators**: To design and implement the data storage solutions
- **Technical Project Managers**: To oversee technical development progress and resource allocation
- **Quality Assurance Engineers**: To develop test strategies based on the architectural design
- **System Integrators**: To understand component interfaces and integration points
- **Technical Writers**: To create implementation documentation and API specifications

This document bridges the gap between requirements (defined in the SRS) and implementation, ensuring all technical stakeholders have a clear understanding of how the system will be constructed.

• *Scope*

The AIMS: An Internet Media Store is a desktop-based e-commerce application designed for the management and sale of physical media products. This design document covers the complete technical architecture for:

1. Product Management Subsystem

- Product catalog management with support for Books, CDs, LP records, and DVDs
- Inventory management with real-time stock tracking
- Price management with business rule enforcement
- Product attribute management based on media type

2. User Management Subsystem

- Role-based access control for Administrators, Product Managers, and Customers
- User authentication using token-based security
- User account lifecycle management

3. E-commerce Transaction Subsystem

- Shopping cart management with session persistence
- Order processing workflow including standard and rush delivery options
- Payment integration with VNPay payment gateway
- Order status tracking and management

4. Integration Subsystem

- VNPay API integration for payment processing
- Email notification system for order confirmations and user management
- External shipping calculation services

Technical Scope:

- Multi-user desktop application supporting up to 1,000 concurrent users
- Response time requirements: ≤2 seconds normal operation, ≤5 seconds peak hours
- Uptime requirement: 300 hours continuous operation
- Recovery time: ≤1 hour maximum downtime after failure

Out of Scope:

- Digital media product sales

- Alternative payment methods beyond credit cards
- Mobile application versions
- Real-time chat support
- Customer account creation (customers can purchase without accounts)

The system architecture will be designed for scalability, maintainability, and performance, with clear separation of concerns between business logic, data access, and presentation layers.

- ***Glossary***

No	Term	Explanation	Example	Note
1	MVC (Model-View-Controller)	Architectural pattern separating application logic into three interconnected components	User interface (View), Business logic (Controller), Data handling (Model)	Used for organizing code structure
2	API (Application Programming Interface)	Set of protocols and tools for building software applications	VNPay Payment API	Enables system integration
3	Session Management	Process of maintaining user state and cart data during software interaction	Shopping cart persistence during user session	Critical for e-commerce functionality
4	Database Transaction	Sequence of database operations performed as a single logical unit	Order placement with inventory update	Ensures data consistency
5	Repository Pattern	Design pattern that separates data access logic from business logic	ProductRepository, OrderRepository	Abstracts database operations
6	Factory Pattern	Design pattern for creating objects without specifying the exact class	MediaProductFactory for Books, CDs, DVDs	Creates objects based on media type
7	Observer Pattern	Design pattern where objects notify other objects about state changes	Inventory update notifications	Used for real-time updates

8	DTO (Data Transfer Object)	Object that carries data between processes	PaymentDTO, OrderDTO	Reduces method calls
9	Singleton Pattern	Design pattern ensuring a class has only one instance	DatabaseConnection, EmailService	Manages shared resources
10	Service Layer	Architectural layer containing business logic	OrderService, PaymentService	Separates business logic from data access
11	ORM (Object-Relational Mapping)	Technique for converting incompatible type systems	Entity classes mapped to database tables	Simplifies database operations
12	Dependency Injection	Design pattern for achieving Inversion of Control	Injecting PaymentService into OrderController	Improves testability and flexibility
13	RESTful Architecture	Architectural style for designing networked applications	HTTP methods for CRUD operations	Used for external API integration
14	Load Balancer	System that distributes network traffic across multiple servers	Round-robin request distribution	Supports 1,000 concurrent users
15	Cache	Temporary storage for frequently accessed data	Product catalog cache	Improves response time

- ***References***

Centers for Medicare & Medicaid Services. (n.d.). *System Design Document Template.* Retrieved from Centers for Medicare & Medicaid Services: https://www.cms.gov/Research-Statistics-Data-and-Systems/CMS-Information-Technology/XLC/Downloads/SystemDesignDocument.docx

<Listing the referenced material used in this document, including the one related to the project>

Overall Description

General Overview

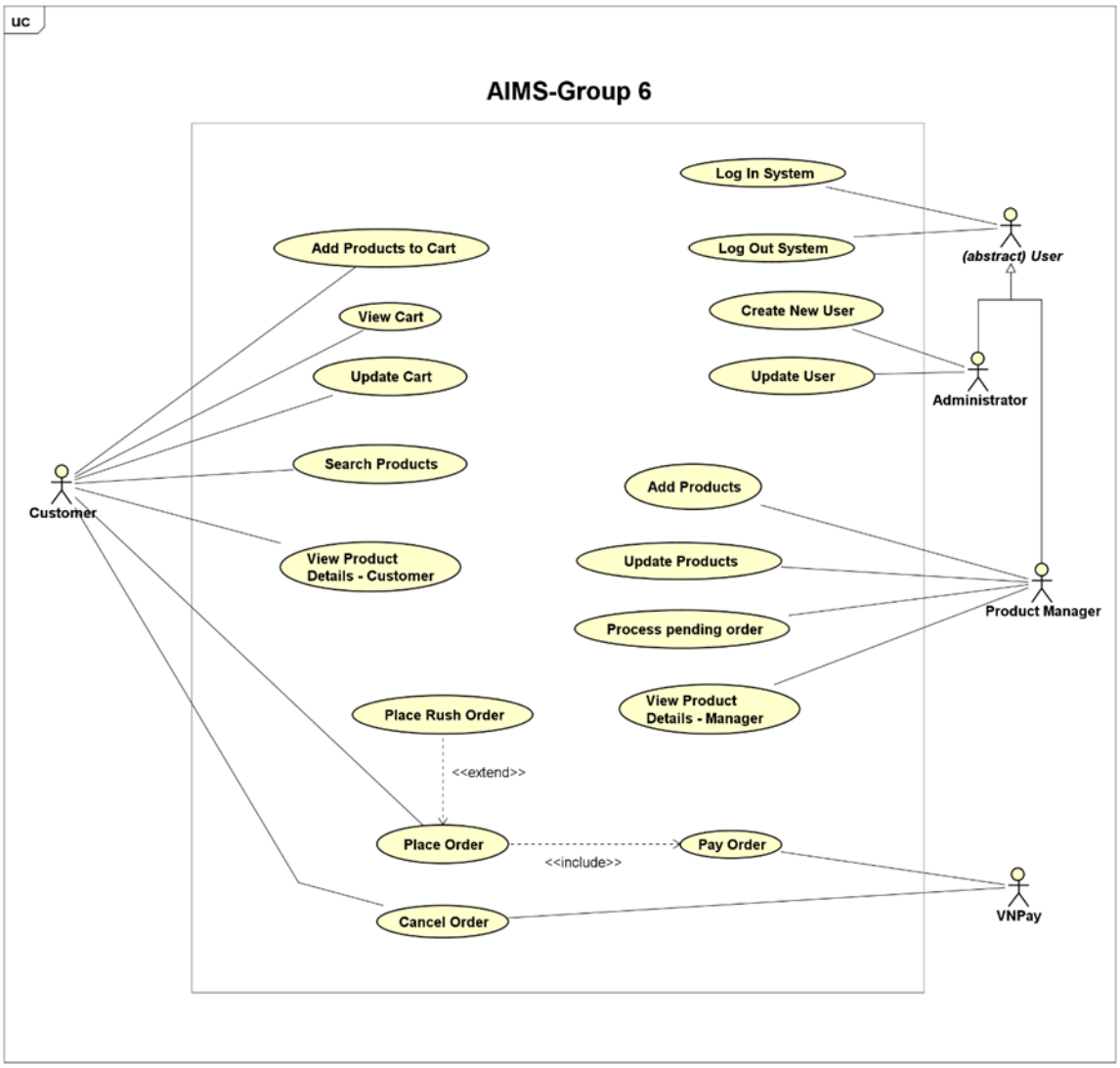
The system in question is an online media store designed to enable customers to purchase digital media products conveniently. This software functions as a comprehensive platform, catering not only to customers but also to product managers.

The system consists of three primary actors:

- **Customer**: Customers can look at, search for, and sort products in the store. To buy something, they must add or update items in their cart and provide delivery details. Once all information is correctly entered, they can proceed with payment via VNPay. Upon a successful transaction, AIMS will send an email confirmation along with the invoice. In certain cases, customers also have the option to place rush orders..
- **Product Manager**: Using the management interface provided by AIMS, product managers can efficiently oversee their store's inventory. They have full control over adding, removing, and updating product details. Additionally, they can manage pending orders by either approving or rejecting them.
- **Administrator**: Administrators hold the authority to manage users within the system. They can create new user accounts, update user information, and block or unblock users as needed.

A Use Case diagram shows how actors interact with different use cases in the system. It represents the system's functional requirements by illustrating the interactions between external and internal agents.

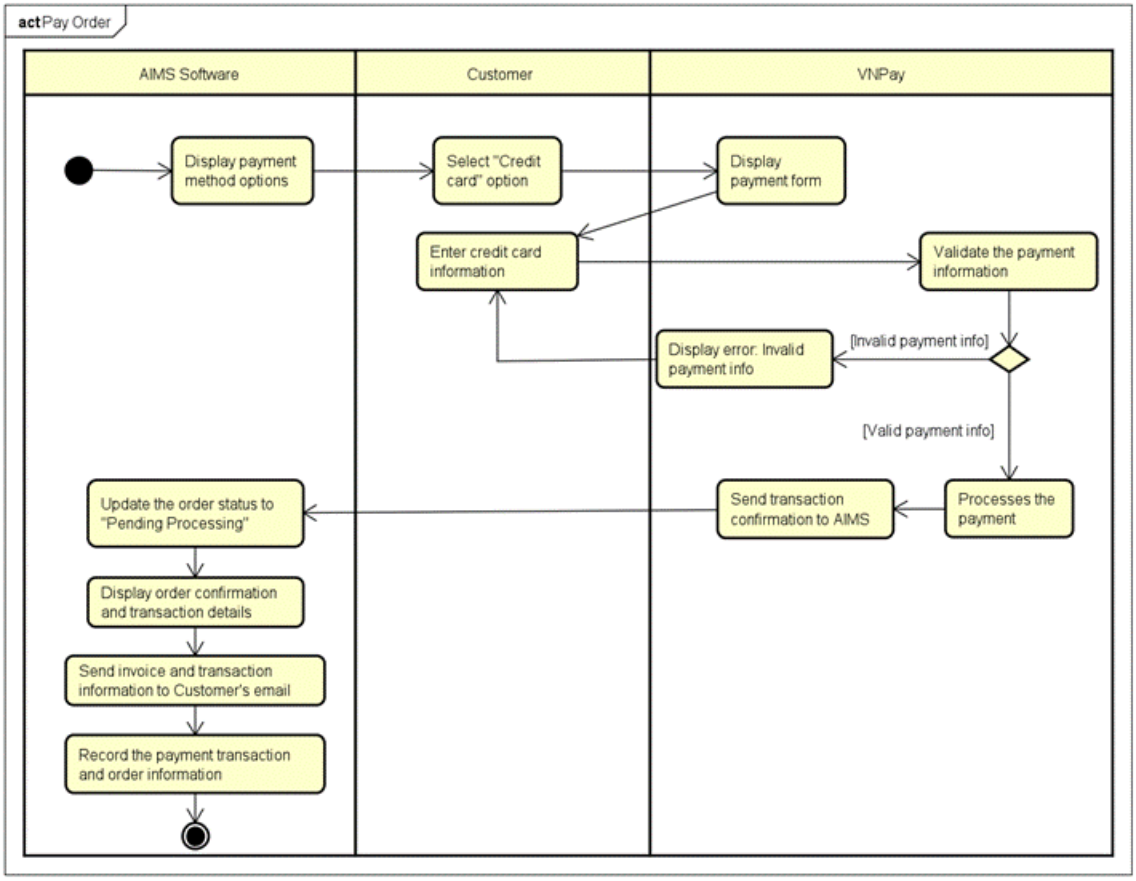
The diagram below presents the general use case diagram of AIMS software, highlighting the actors and their associated use cases.



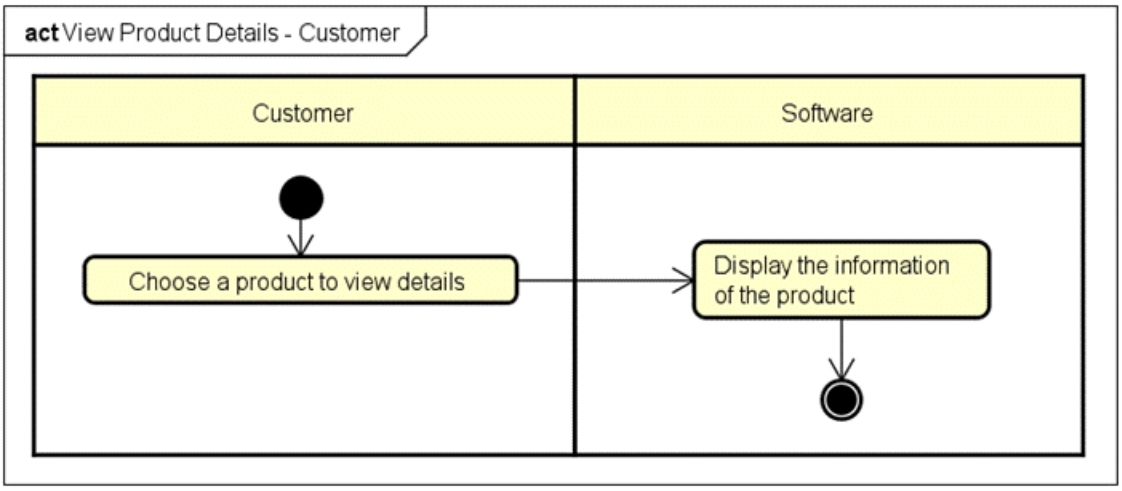
In the AIMS software, five core business operations are defined: "Place Order," "Place Rush Order," "Pay Order," "Cancel Order," and "Process Pending Order." Each of these operations is thoroughly illustrated using an activity diagram in its respective section. Below, we present the activity diagrams for all use cases in the AIMS software.

Activity diagrams are utilized to model business processes as they provide a structured visual representation of workflows, decision points, and resource allocation. These diagrams enable stakeholders to understand, analyze, and refine complex processes by depicting the sequence of activities, concurrency, error handling, and integration with other models. Additionally, they function as effective documentation tools, enhancing communication and collaboration among stakeholders.

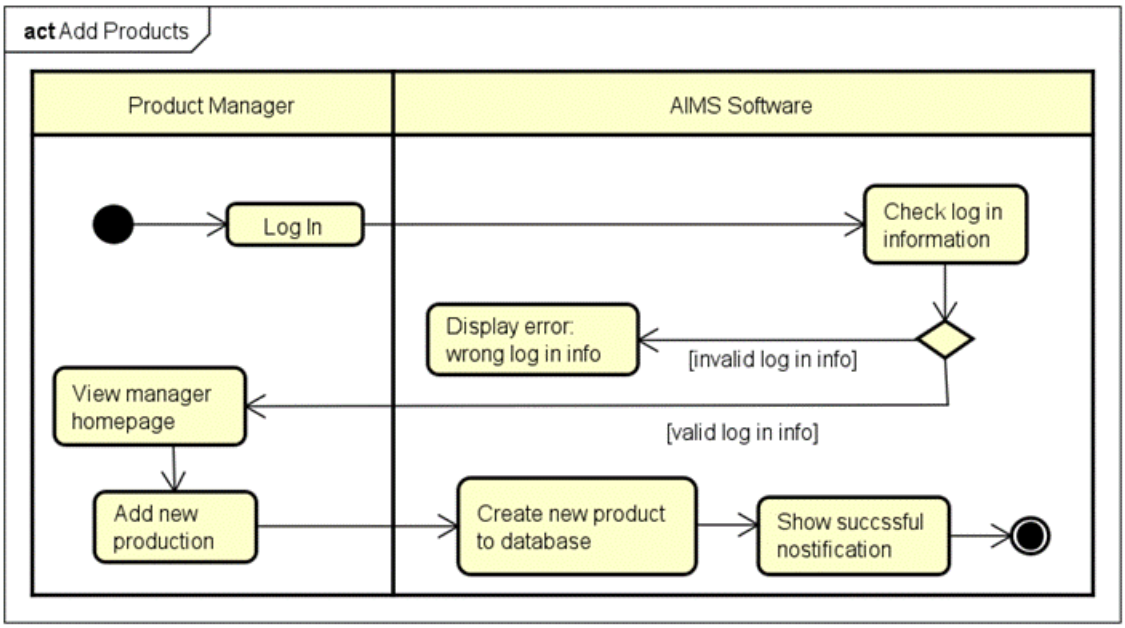
2.1.1 Business operation- Pay Order



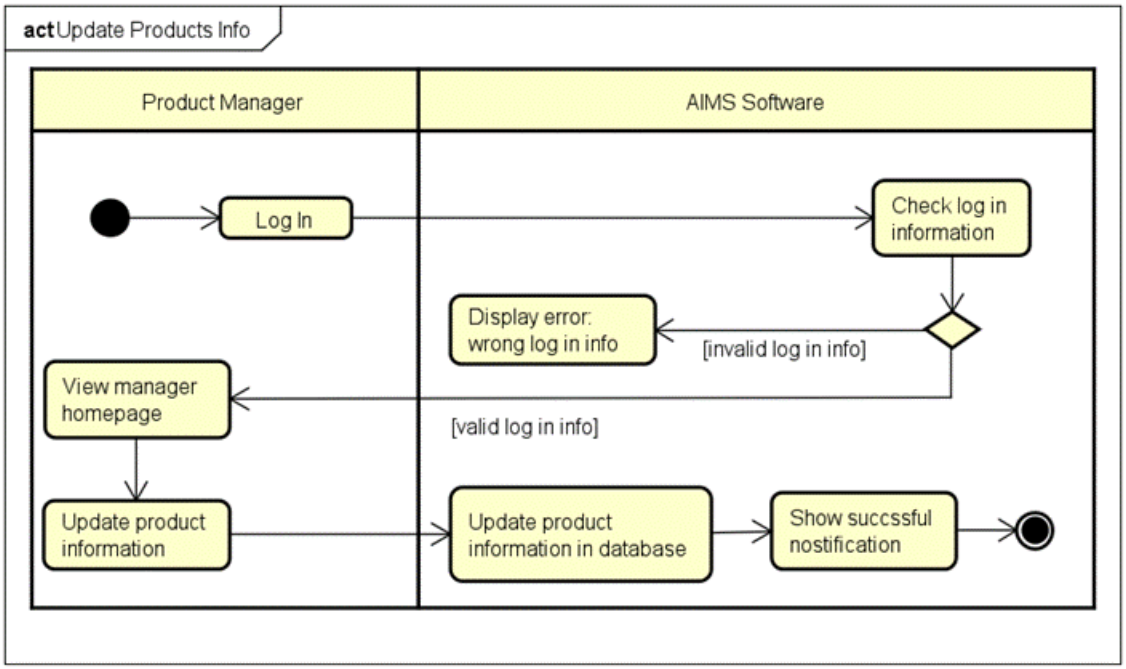
2.1.2 Business operation- View Product Detail



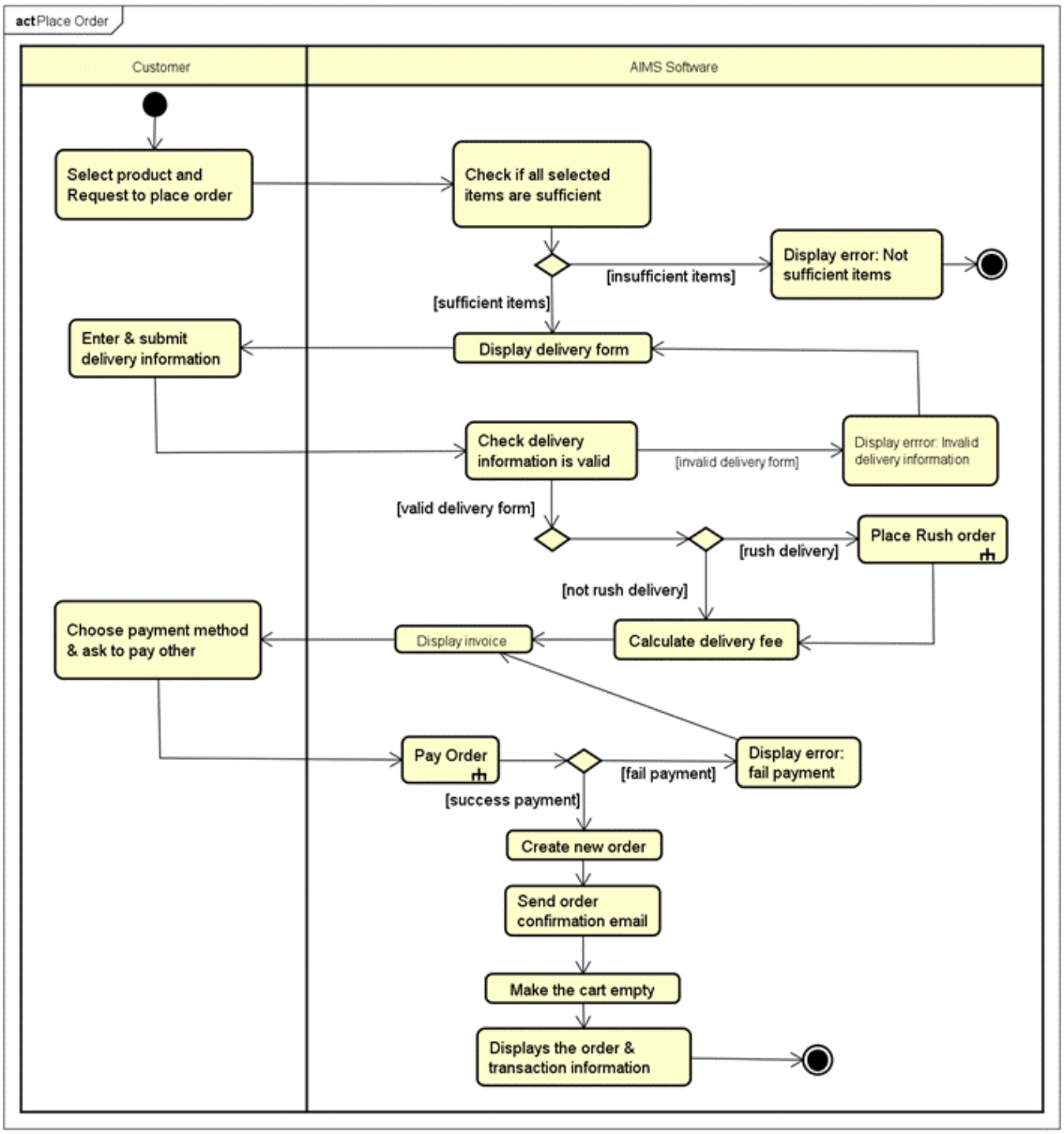
2.1.3 Business operation- Add Product



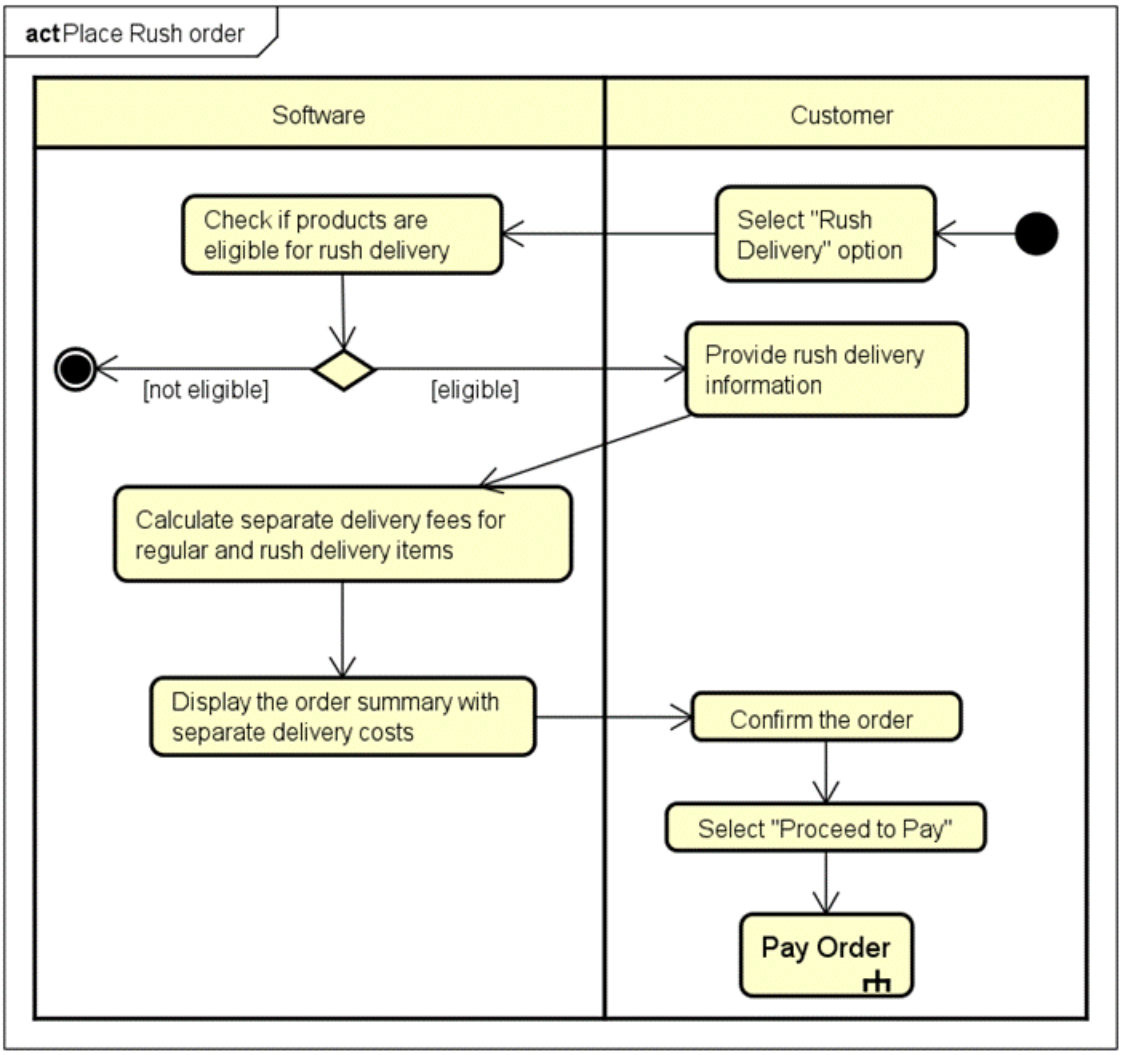
2.1.4 Business operation- Update Product



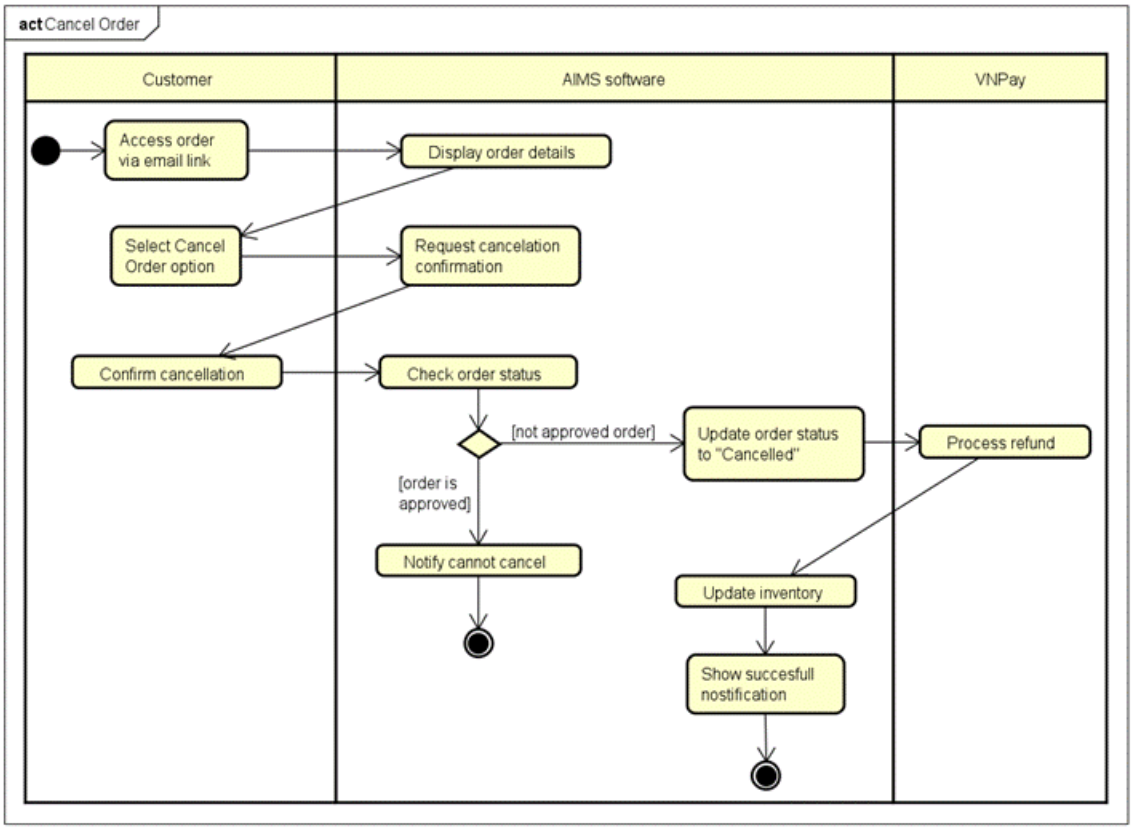
2.1.5 Business operation- Place Order



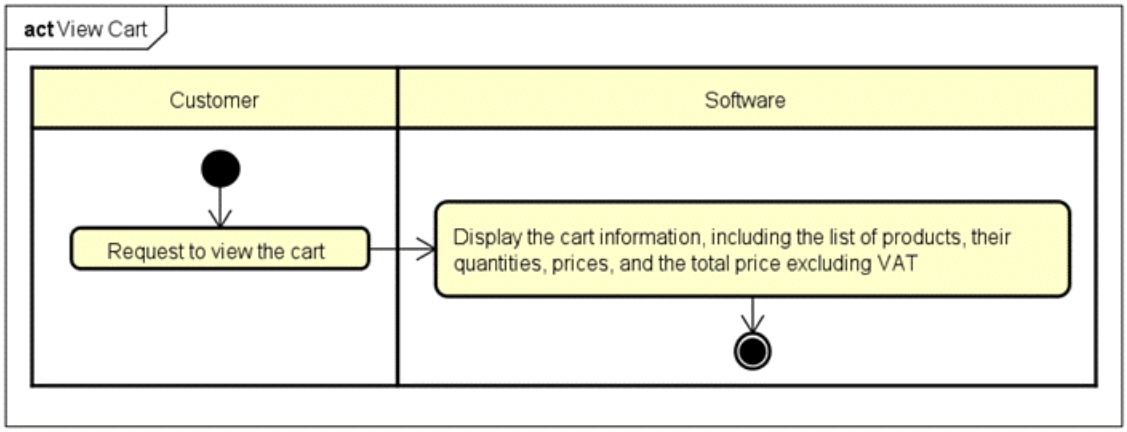
2.1.6 Business operation- Place Rush Order



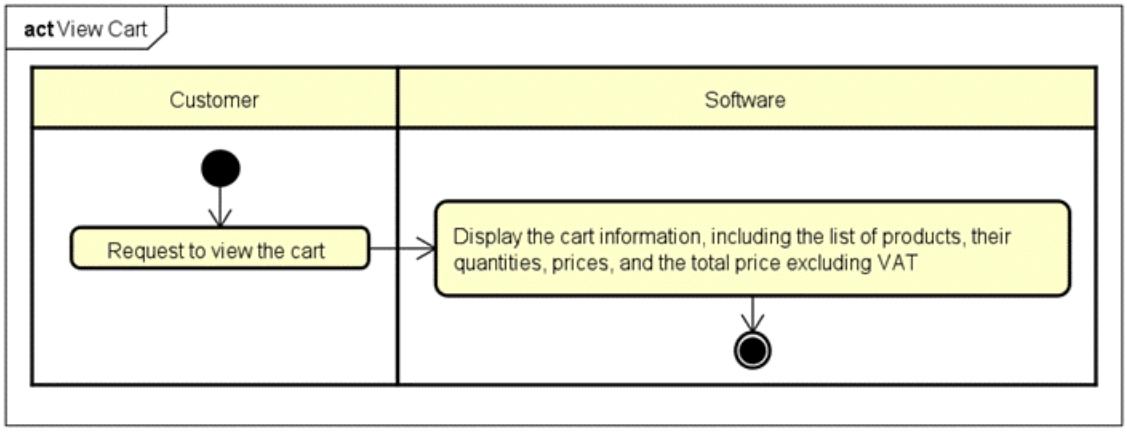
2.1.7 Business operation- Cancel Order



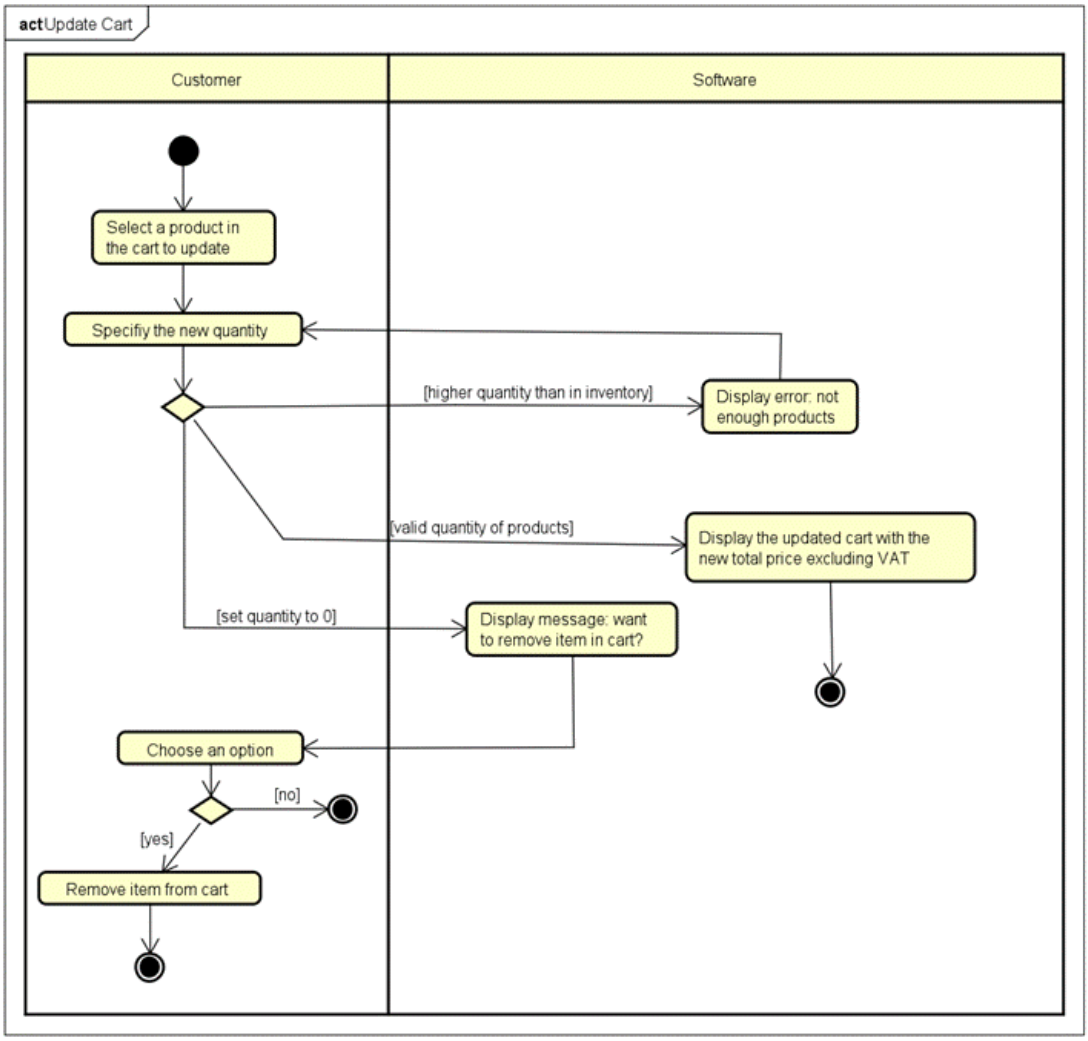
2.1.8 Business operation- Add product to Cart



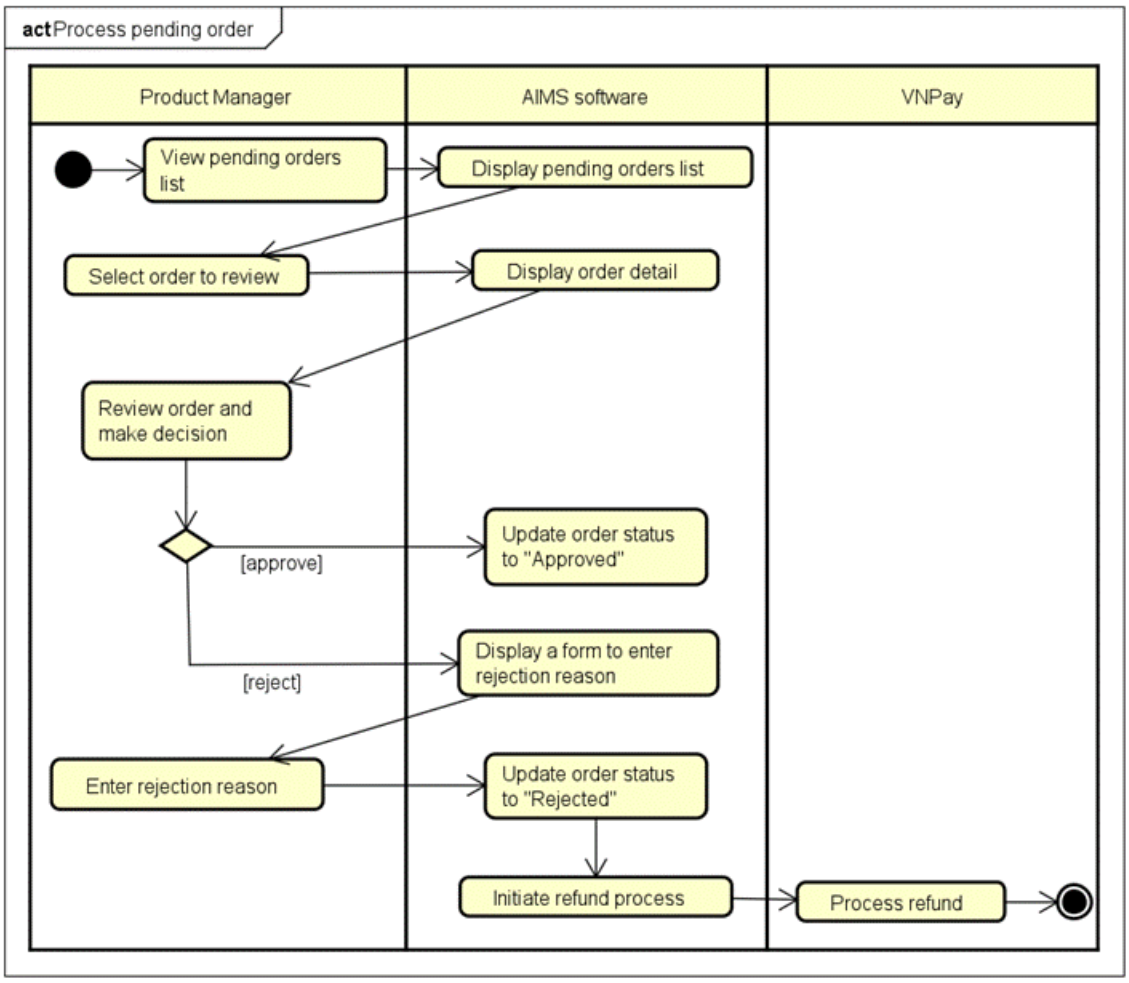
2.1.9 Business operation- View Cart



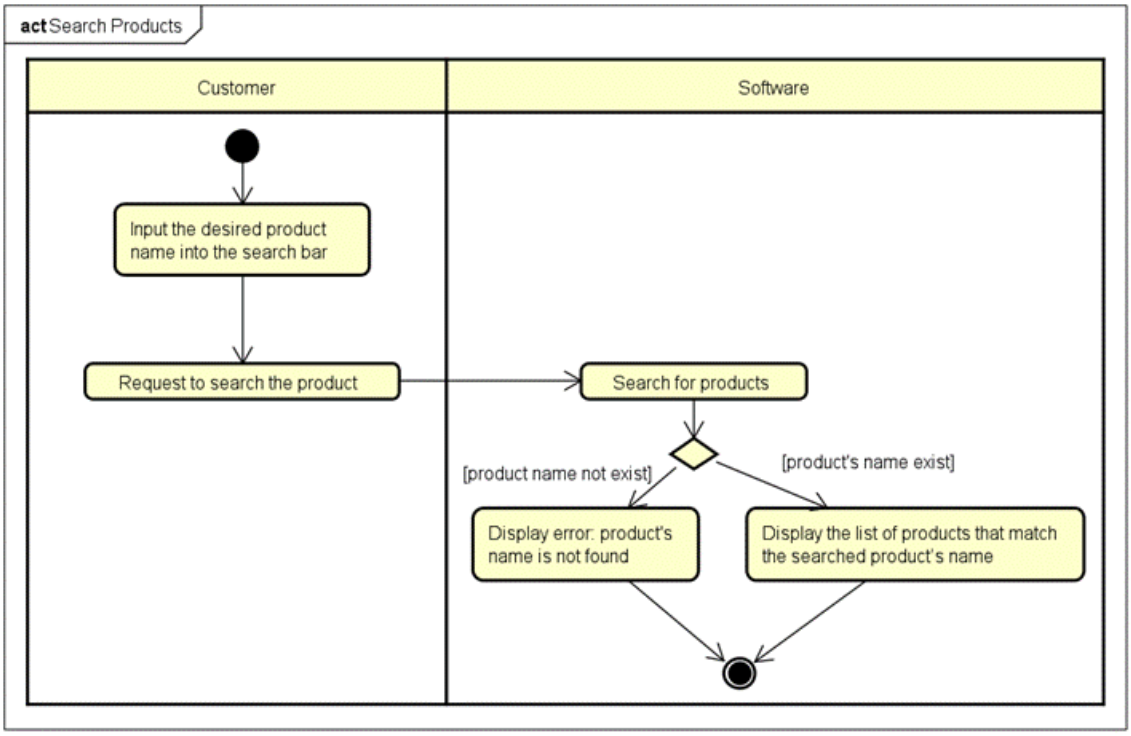
2.1.10 Business operation- Update Cart



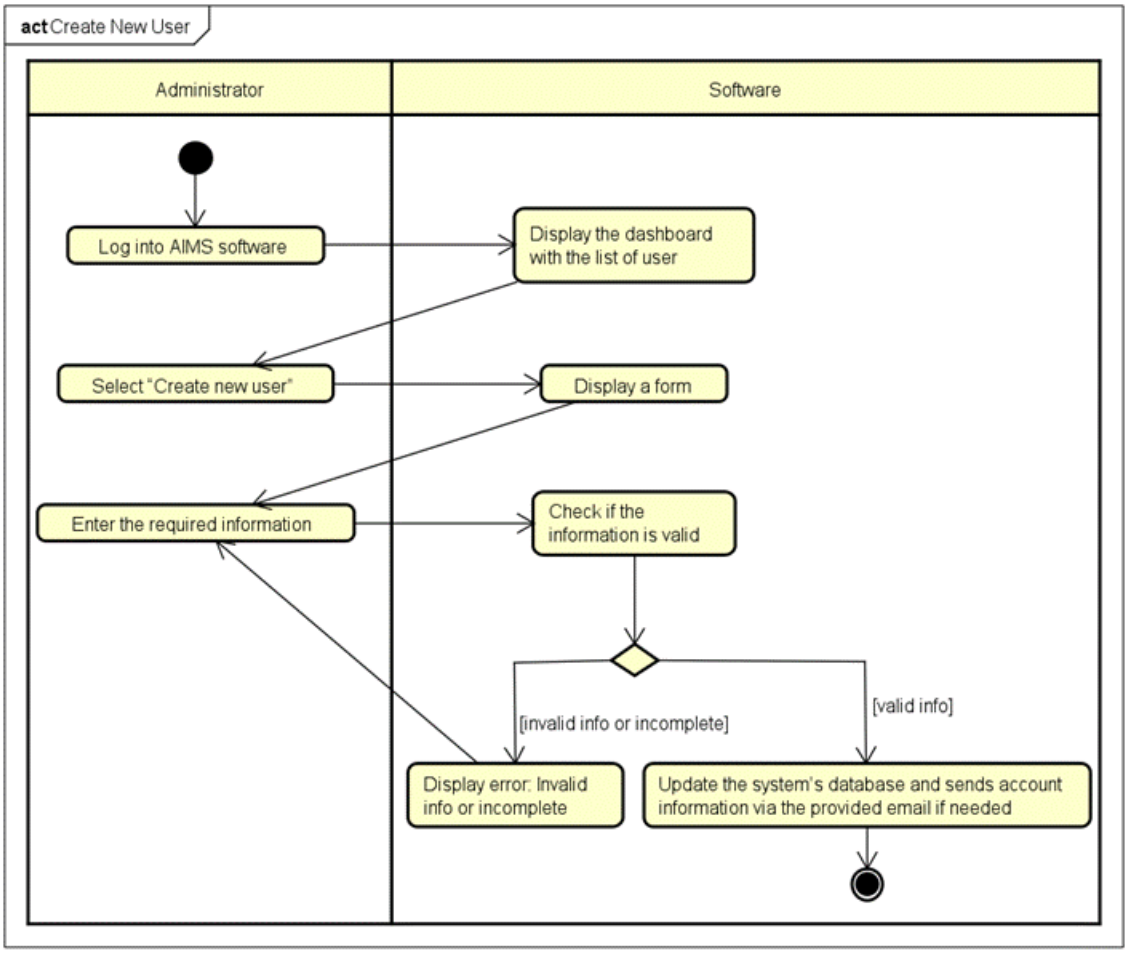
2.1.11 Business operation- Process Pending Order



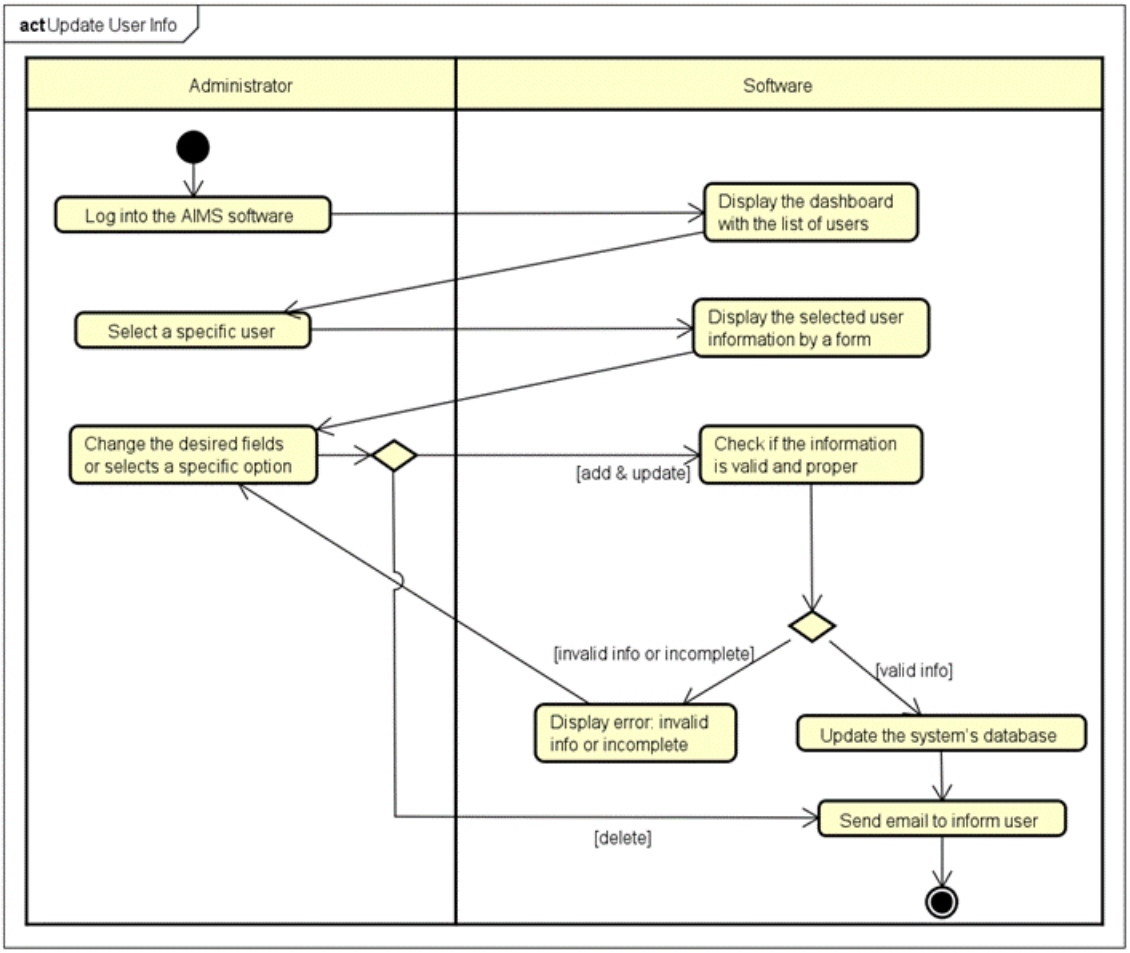
2.1.12 Business operation- Search Product



2.1.13 Business operation- Create User



2.1.14 Business operation- Update User



- ***Assumptions/Constraints/Risks***
- **Assumptions**

  - **Software and Hardware Dependencies:** The AIMS platform is intended to function efficiently within a standard Java application environment. A conventional relational database management system (such as SQLite) will be employed for data storage and management.
  - **Operating System Compatibility:** The system will be primarily developed and deployed in a Windows environment; however, the final application must ensure cross-platform compatibility, particularly with both Windows and macOS operating systems.
  - **User Profile Assumptions:** It is assumed that target users possess a basic level of digital literacy, including familiarity with navigating online marketplaces and interacting with digital media platforms.Users are expected to have reliable internet connectivity and access to devices capable of streaming or downloading media content.

- **Functional Evolution :**Anticipated future enhancements may involve the addition of new media formats or the integration of alternative payment systems. Accordingly, the system must be designed with a modular and adaptable architecture. The platform architecture should facilitate scalability, allowing for the integration of new features with minimal structural modifications.

• Constraints

The design and implementation of the system are subject to a range of global constraints that significantly influence decisions regarding hardware architecture, software development, communication protocols, and overall system functionality. These constraints, derived from technical, operational, legal, and environmental factors, are outlined below along with their associated impacts:

- **Hardware and Software Environment Constraints:** The system must be compatible with standard Java-based applications and operate efficiently within commonly used desktop environments (e.g., Windows and macOS). This constraint necessitates cross-platform compatibility in both the user interface and backend components, thereby influencing the choice of development frameworks and runtime environments.
- **End-User Environment Constraints:** The target user base is expected to possess only a basic level of digital proficiency and operate within diverse computing environments. This necessitates a user-friendly interface and robust error handling to accommodate varying levels of technical expertise and device capabilities.
- **Availability and Volatility of Resources:** Fluctuations in human resources and funding availability may affect development timelines and operational continuity. Moreover, reliance on third-party APIs for critical functions such as payment processing and media delivery requires the system to incorporate fallback mechanisms and modular integration to ensure resilience.
- **Standards Compliance and Licensing:** The system must adhere to relevant open-source software licenses and digital media distribution agreements. These legal constraints influence the selection of third-party libraries, content handling procedures, and deployment strategies to avoid intellectual property violations.
- **Interoperability and Interface Requirements:** Integration with third-party services, such as payment gateways (e.g., VNPay) and external media content providers, demands conformance to specified

communication protocols and data formats. This imposes architectural constraints on API design, error management, and data exchange mechanisms.

- **Data Repository and Distribution Requirements:**
  A centralized SQLite database is employed to manage user accounts, transaction histories, and media inventory. Given the localized nature of SQLite, the system must implement optimized query execution, efficient indexing, and data access patterns to ensure performance, particularly during concurrent operations. While content distribution is not cloud-based, media must be managed efficiently for local playback or controlled download.

- **Security and Regulatory Compliance:**
  The system must safeguard user data and financial transactions in compliance with standard security protocols (e.g., HTTPS, encryption, secure authentication). This impacts system architecture by requiring the integration of secure communication channels and robust access control mechanisms.
- **Capacity Limitations**
  As a desktop-based solution, the application must be optimized for limited system memory and processing power. Efficient memory management, resource allocation, and JavaFX performance tuning are essential to support concurrent user actions, background processes, and media operations without degrading user experience.

- **Performance Requirements:**
  Users expect quick application startup, smooth navigation, and uninterrupted media playback. To meet these expectations, the system must employ caching strategies, preloading techniques, and efficient media handling logic to minimize latency and ensure responsiveness under varying system loads.
- **Network Communication Constraints:**
  The application depends on a stable internet connection for payment processing and media-related operations. However, it must also tolerate network variability through retry logic, connection timeout handling, and asynchronous processing to ensure graceful degradation and uninterrupted user interaction.

- **Verification and Validation Requirements:**
  The system must undergo rigorous validation through automated unit tests, integration tests, and user acceptance testing. This is critical to ensure that all components—from the JavaFX front end to the SQLite database and VNPay API integration—function correctly and securely under expected operational conditions.

- **Quality Assurance Measures:** To ensure long-term reliability and maintainability, the system should adopt continuous integration and delivery (CI/CD) practices, regular code reviews, and scheduled maintenance cycles. Performance monitoring and user feedback mechanisms will also be incorporated to guide iterative improvements and adaptation over time.

- ## Risks

  - **Third-Party Dependency Risks:**
    - **Identified Risk:** The system relies on third-party services for critical functions such as payment processing and media content delivery, which introduces potential points of failure or service disruption.
    - **Mitigation Strategy:** To reduce dependency-related vulnerabilities, the system rchitecture will support integration with multiple service providers. In addition, contingency protocols will be developed to ensure business continuity in the event of third-party service failures.
  - **Human Resource Risks:**
    - **Identified Risk:** A shortage of qualified developers or the loss of key technical personnel may adversely affect project timelines and product quality.
    - **Mitigation Strategy:** This risk will be addressed through competitive recruitment processes, investment in ongoing staff development, and initiatives focused on employee retention and knowledge transfer.
  - **Technical and Scalability Risks:**
    - **Identified Risk:** As the user base expands, the system may encounter performance degradation or infrastructure limitations.
    - **Mitigation Strategy:** The platform will be designed using scalable cloud-based infrastructure, incorporating techniques such as horizontal scaling and load balancing to accommodate increased demand while maintaining performance standards.

• System Architecture and Architecture Design

In this section, we propose the general overview of System Architecture Design. This starts from defining the Architectural Patterns, then we design the interaction diagram using sequence diagrams for the proposed architectural patterns. From the interaction diagrams, we can have the analysis class diagrams, which is the fundamental steps for the detailed-level class diagrams for implementation.

• *Architectural Patterns*

The Entity-Control-Boundary (ECB) architecture design pattern is a software design approach that helps in organizing and structuring code to enhance maintainability, scalability, and clarity. This pattern divides the system into three primary types of components: entities, controls, and boundaries.

Entities represent the core business logic and data of the application. These components encapsulate the main data structures and the operations directly related to the core functionality of the system. Entities are typically designed to be reusable and independent of the specific use cases. They model real-world objects and are responsible for managing and maintaining their state and behavior. For example, in a banking application, entities might include classes such as *Account*, *Customer*, and *Transaction*.

Controls are responsible for coordinating the flow of information between entities and boundaries. They manage the application's use cases, orchestrating the interactions between entities and ensuring the correct sequence of operations. Controls contain the logic that drives the application's processes, ensuring that user inputs are properly processed and that the necessary business rules are applied. For instance, a *PlaceOrder* control might handle the logic for executing a financial transaction, validating inputs, and updating account balances accordingly.

Boundaries define the interfaces through which the system interacts with external actors, such as users, other systems, or external services. They serve as the point of communication between the application and its environment, encapsulating the specifics of input and output mechanisms. Boundaries translate user actions into requests that the control components can handle and present the results back to the user. Examples of boundaries include user interface components like forms and buttons, web service interfaces, or API endpoints, which serve as subsystems in interface design.

We select this architecture pattern because many advantages it delivers:

- **Separation of Concerns**: By clearly dividing the system into entities, controls, and boundaries, the ECB pattern ensures a clean separation of concerns, making the codebase more modular and easier to understand.

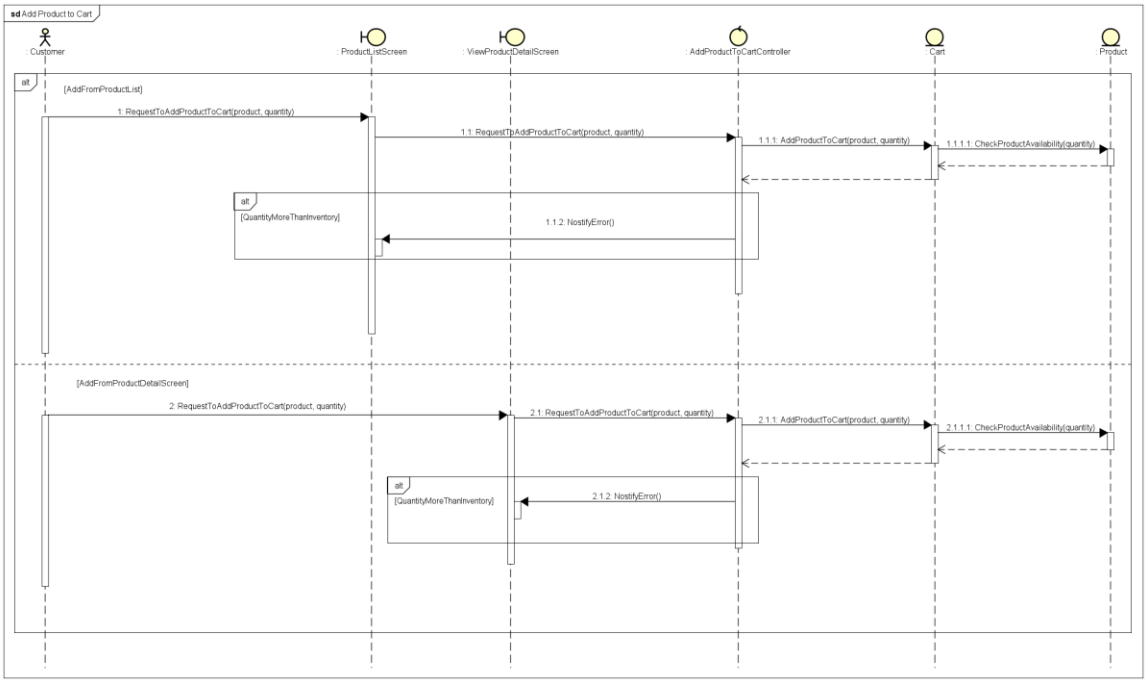
- **Maintainability**: Changes in one part of the system (e.g., user interface changes) do not directly impact the core business logic encapsulated in entities, thereby reducing the risk of introducing bugs when making modifications.

- **Scalability**: The modular nature of the ECB pattern facilitates scalability. New features can be added by creating new controls and boundaries without altering existing entities.

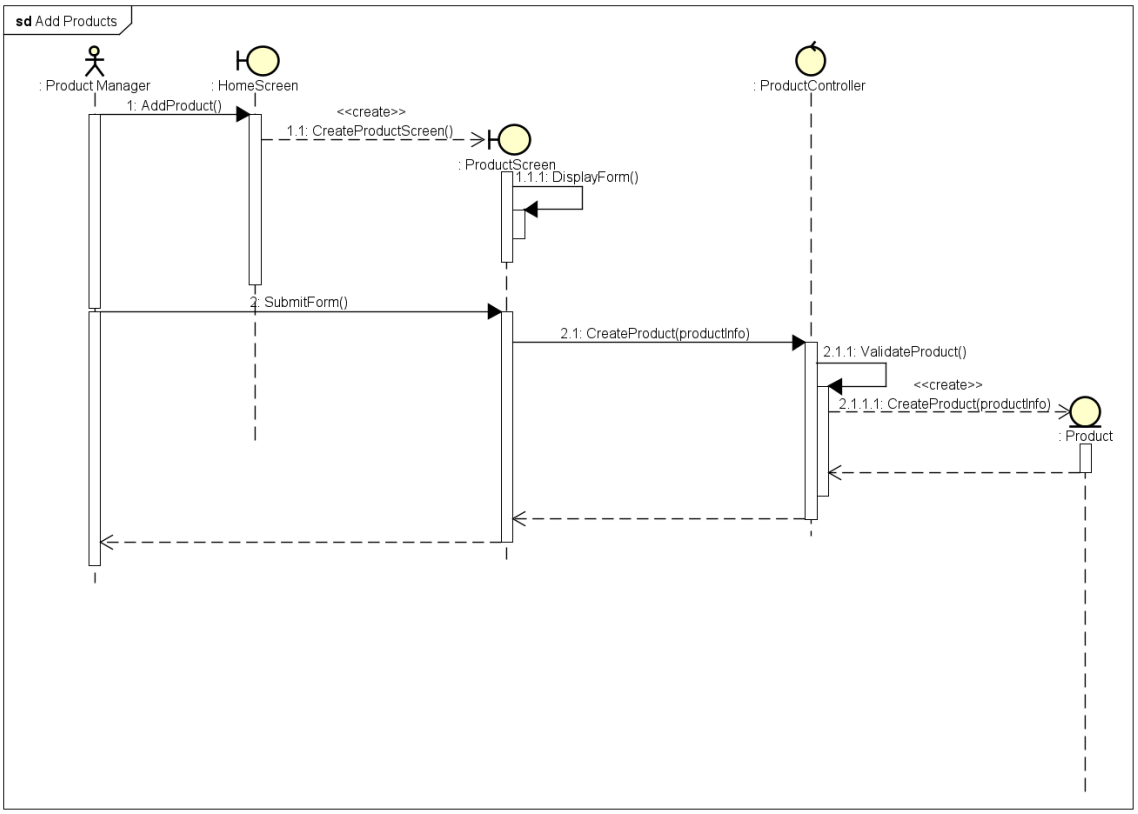
- **Testability**: The clear separation between entities, controls, and boundaries enhances testability. Business logic can be tested independently of the user interface, and mock boundaries can be used to simulate interactions during testing.

- ***Interaction Diagrams***

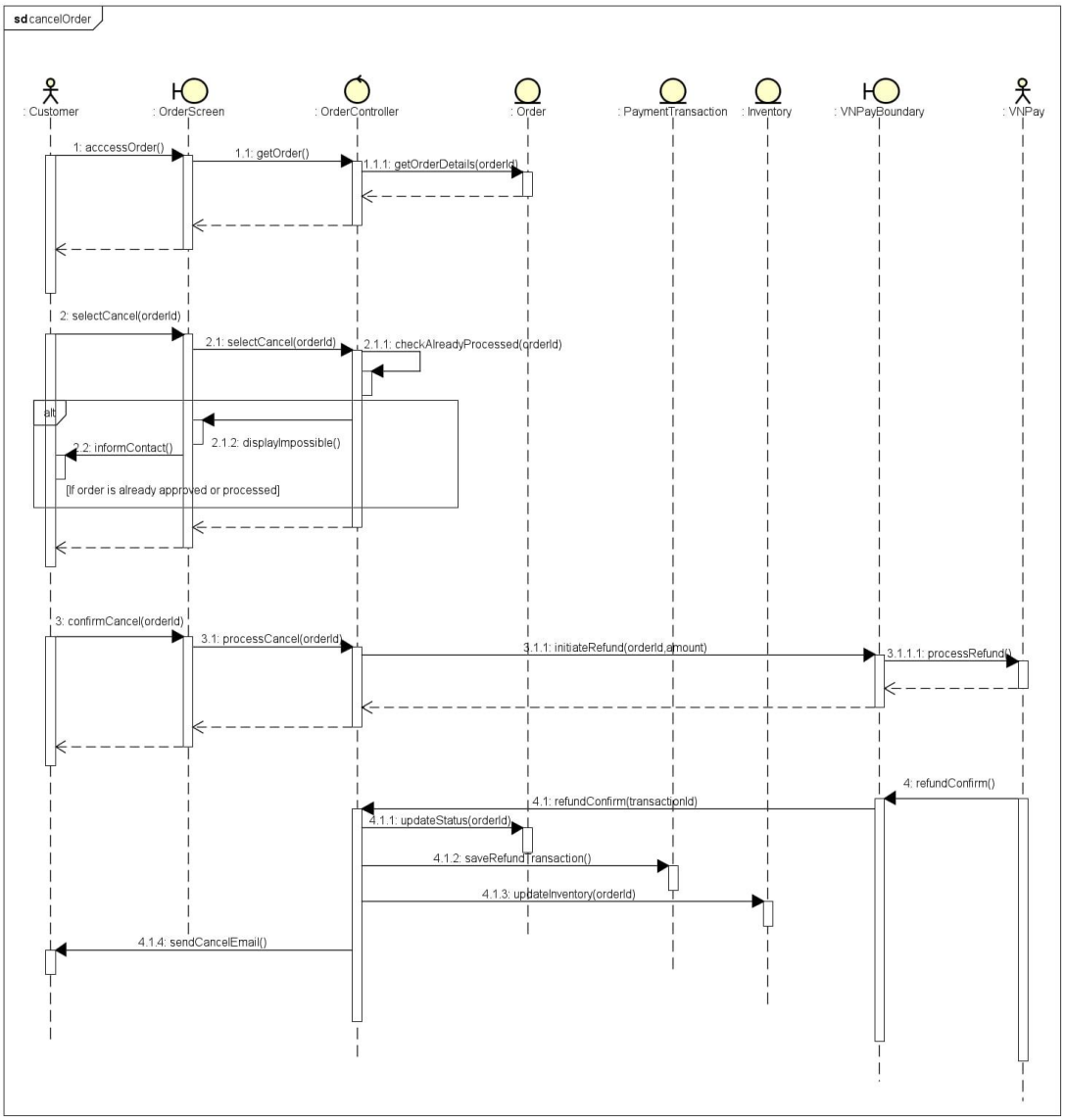
1. Add Product to Cart



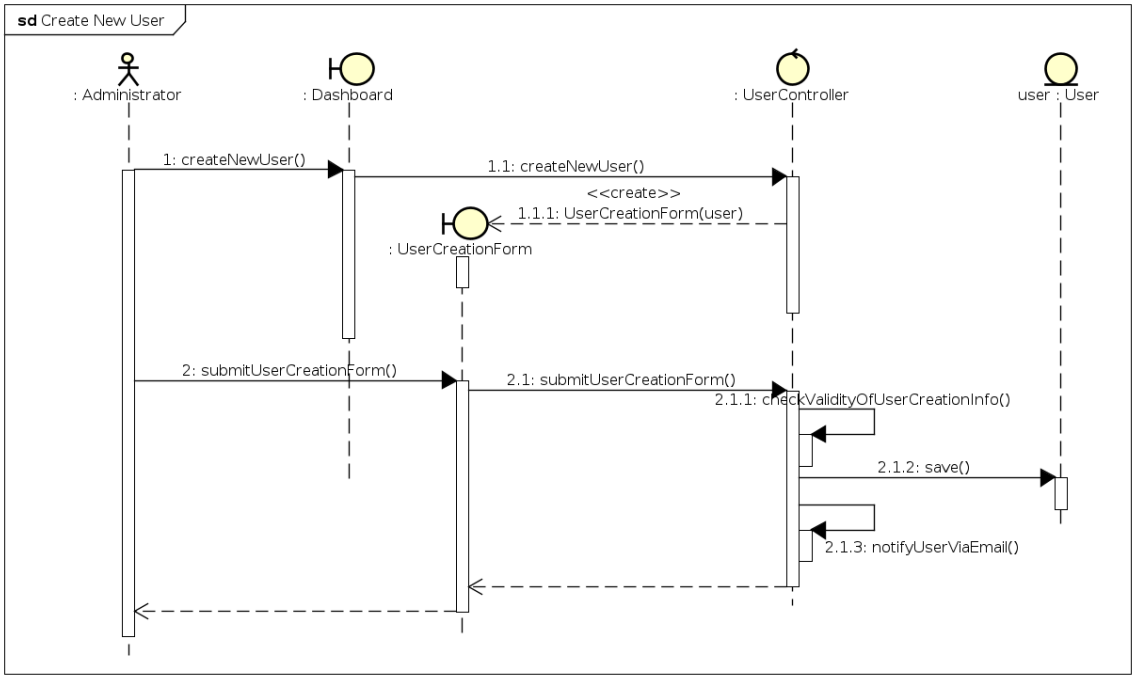
2. Add Products



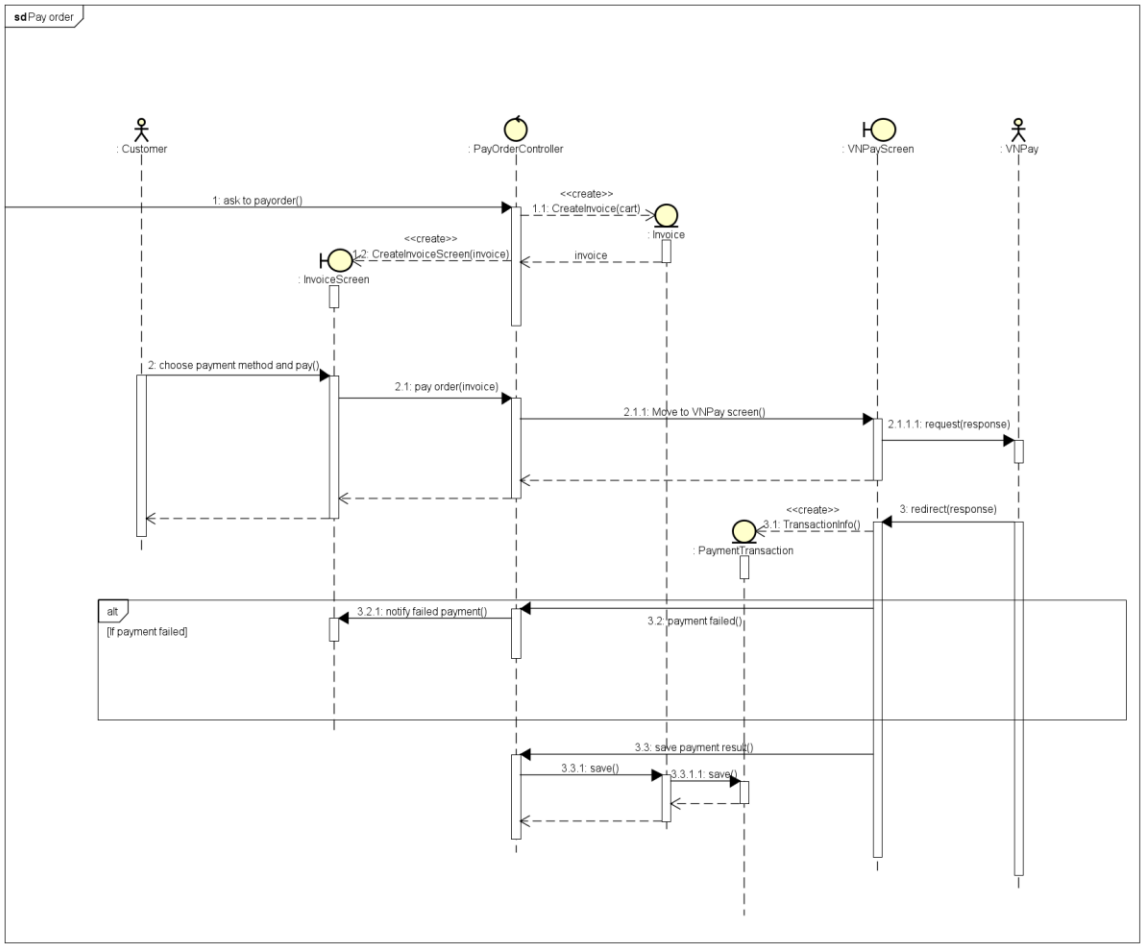
3. Cancel Order



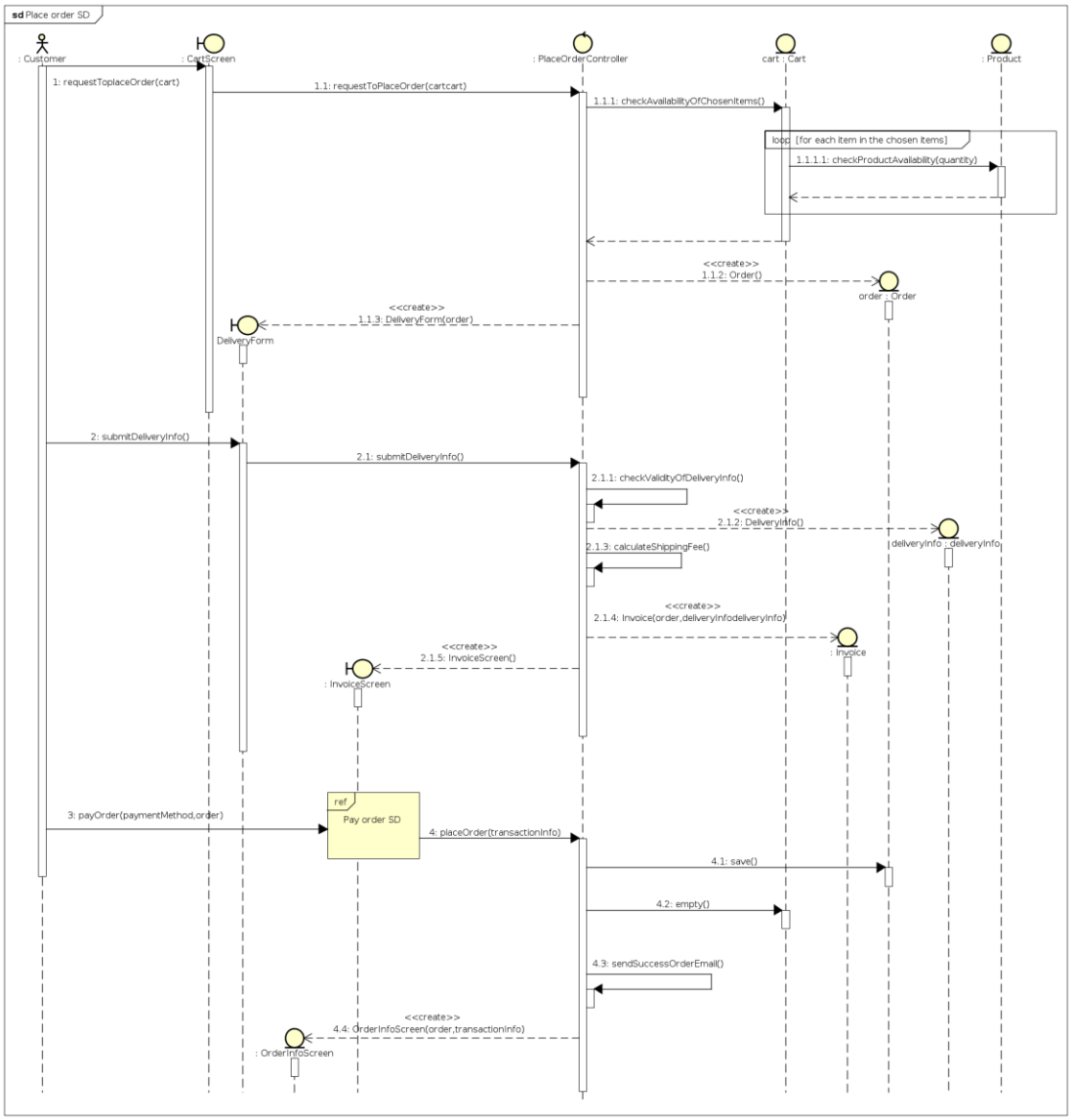
4. Create New User


sd Create New User
: Administrator
: Dashboard
: UserController
user : User
1: createNewUser()
1.1: createNewUser()
<<create>>
1.1.1: UserCreationForm(user)
: UserCreationForm
2: submitUserCreationForm()
2.1: submitUserCreationForm()
2.1.1: checkValidityOfUserCreationInfo()
2.1.2: save()
2.1.3: notifyUserViaEmail()

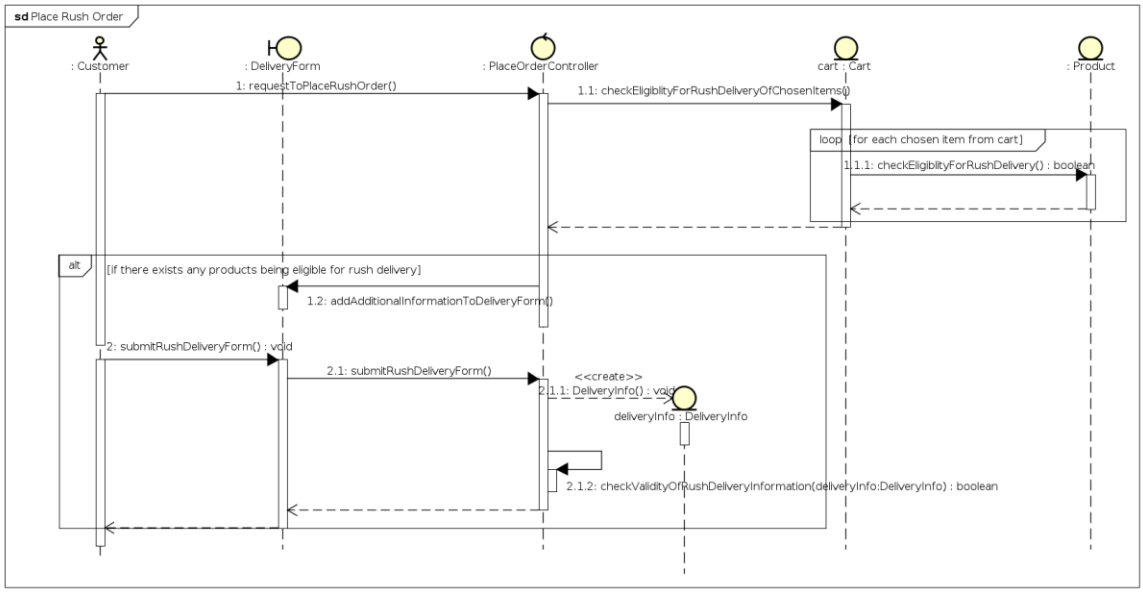
5. Pay Order



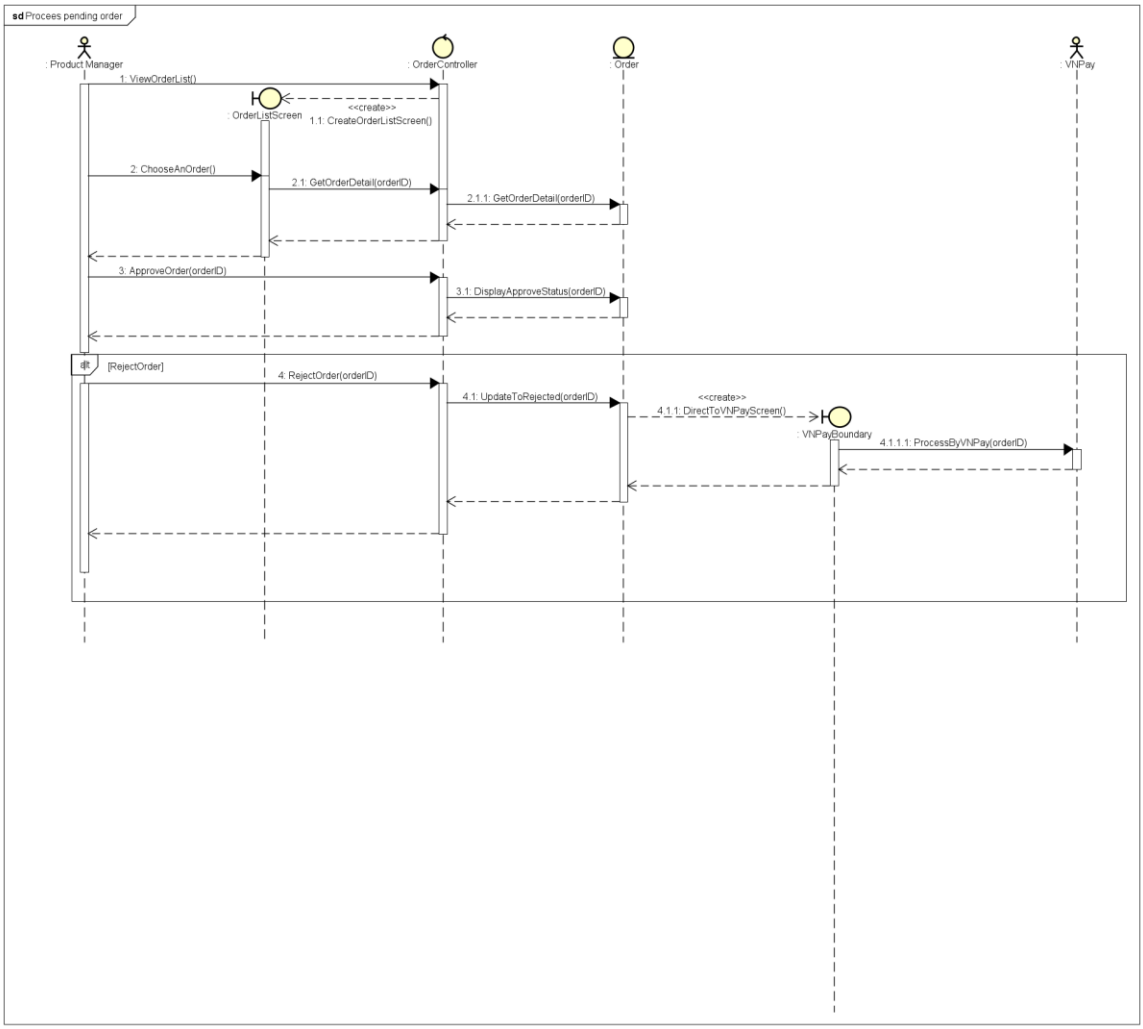
6. Place Order



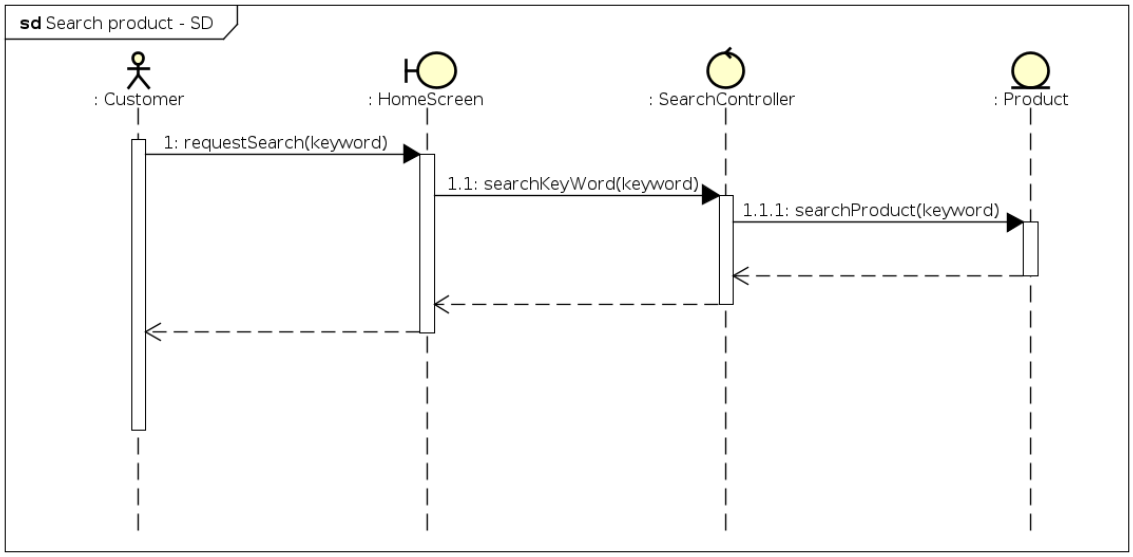
7. Place Rush Order



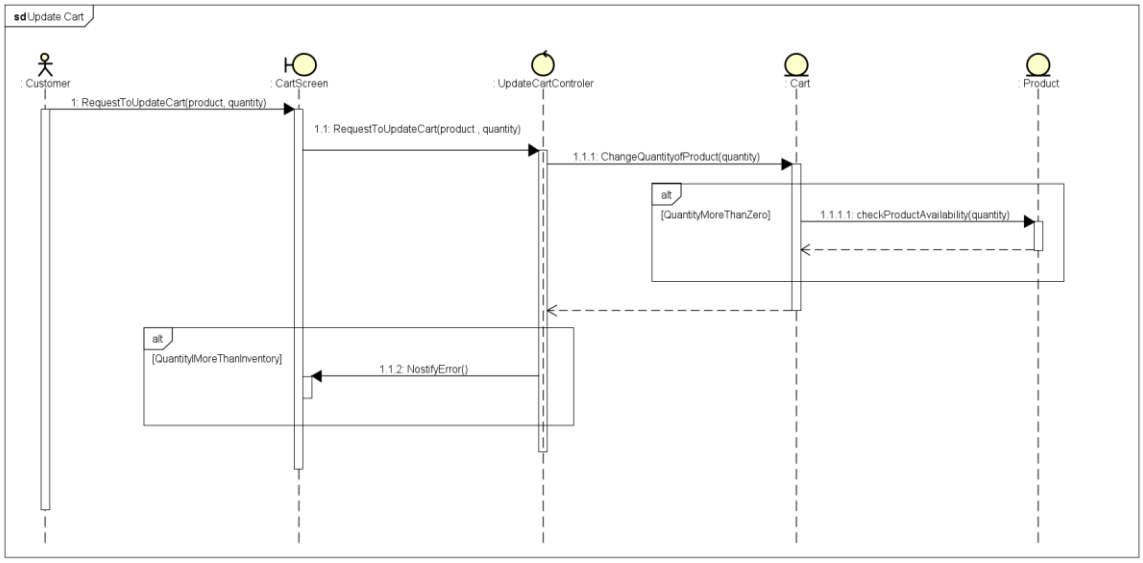
8. Process pending order



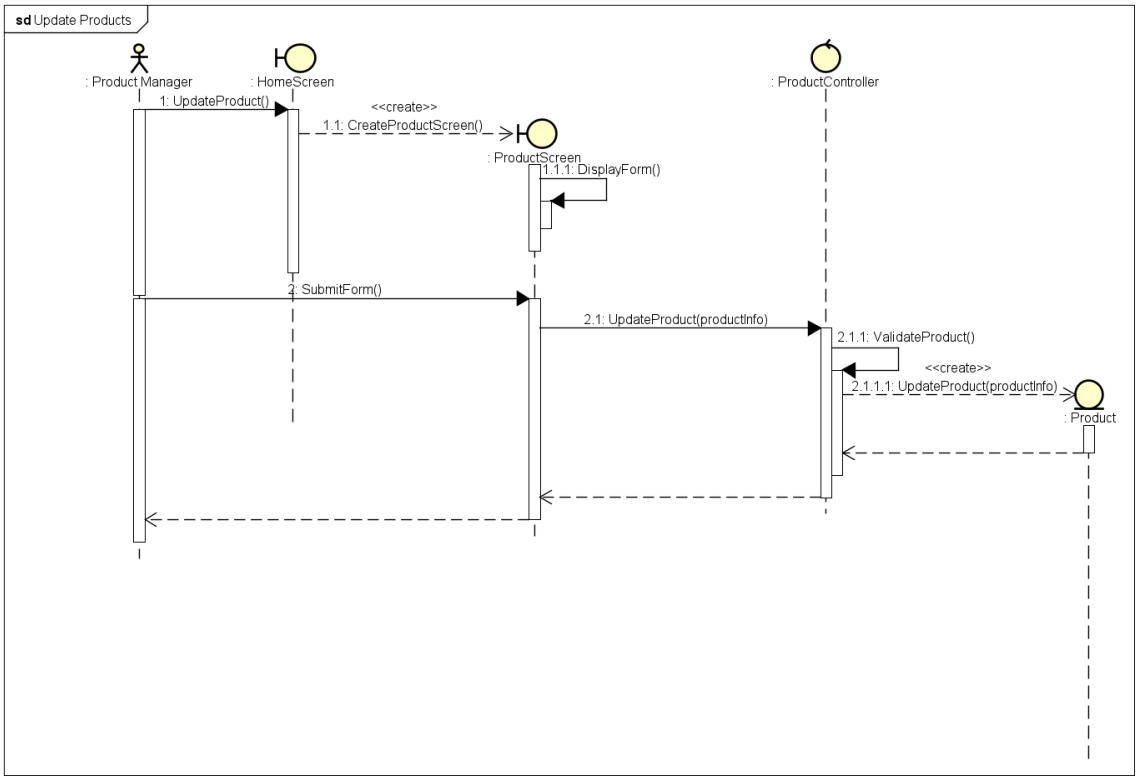
9. Search product



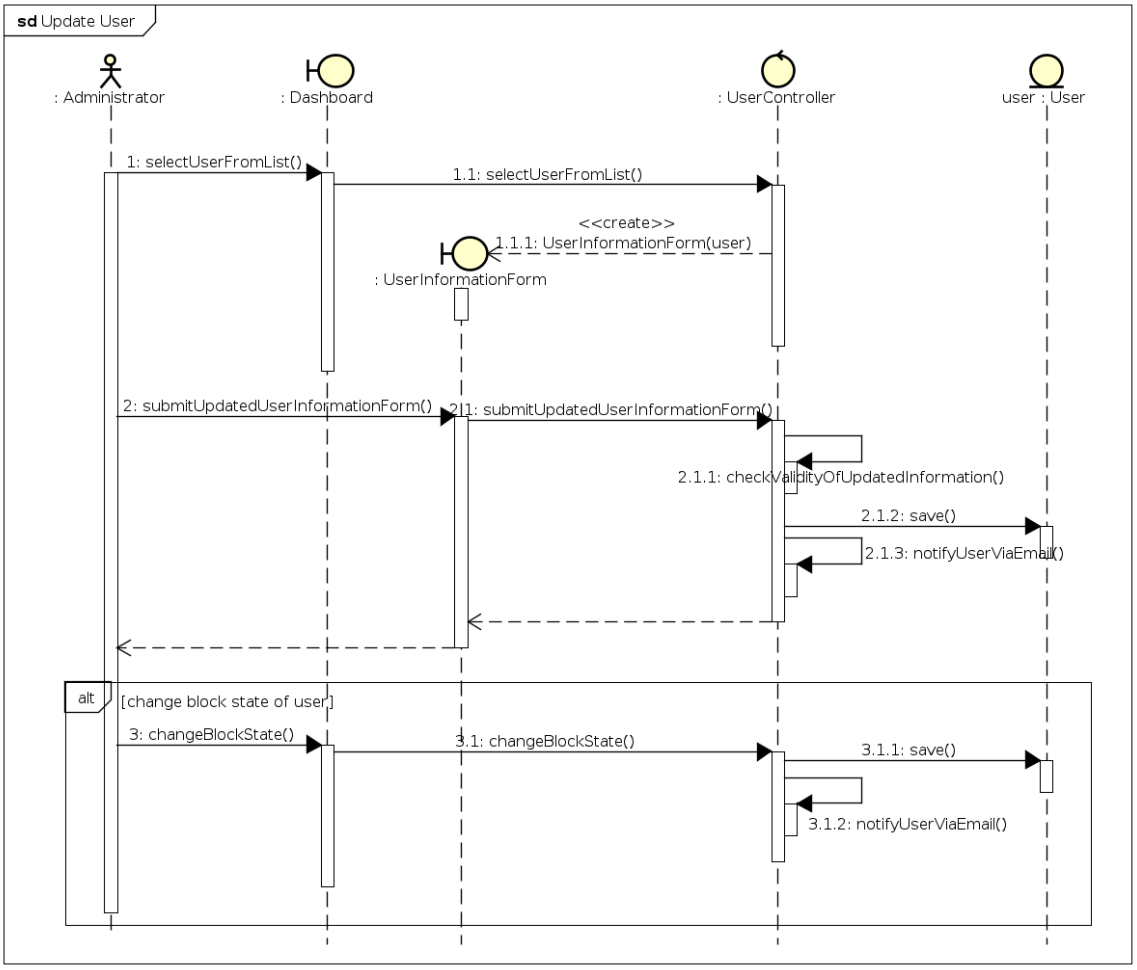
10. Update Cart



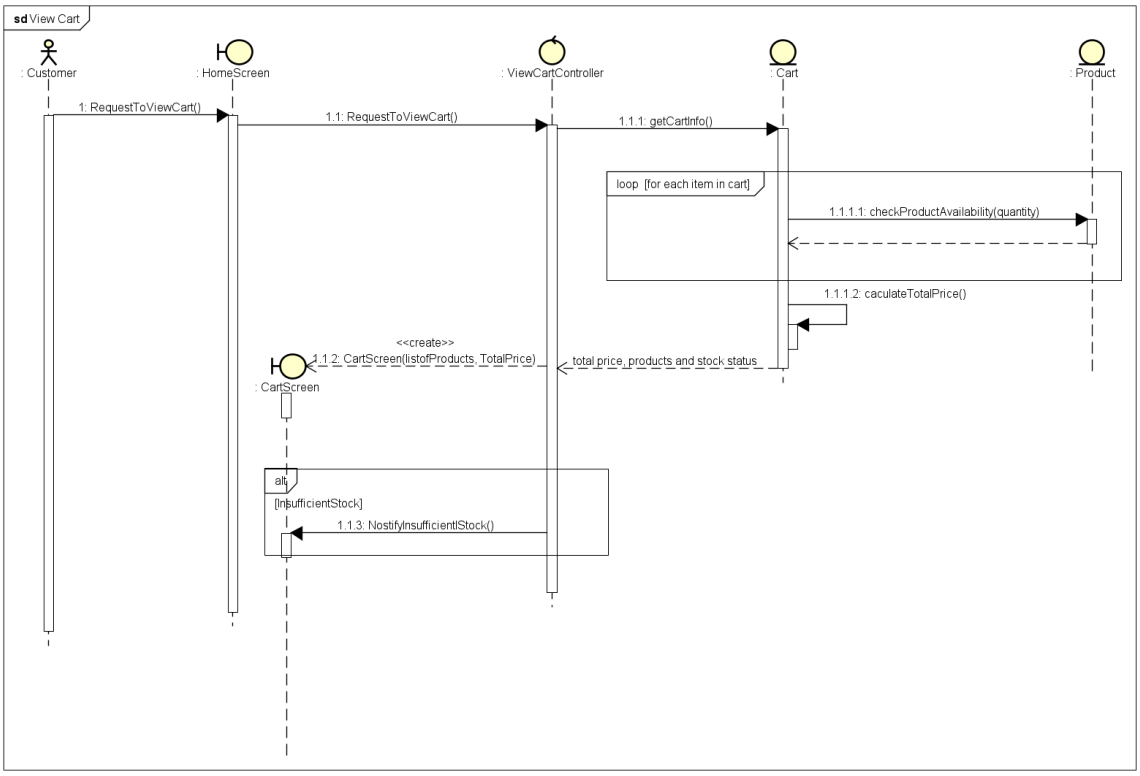
11. Update Products



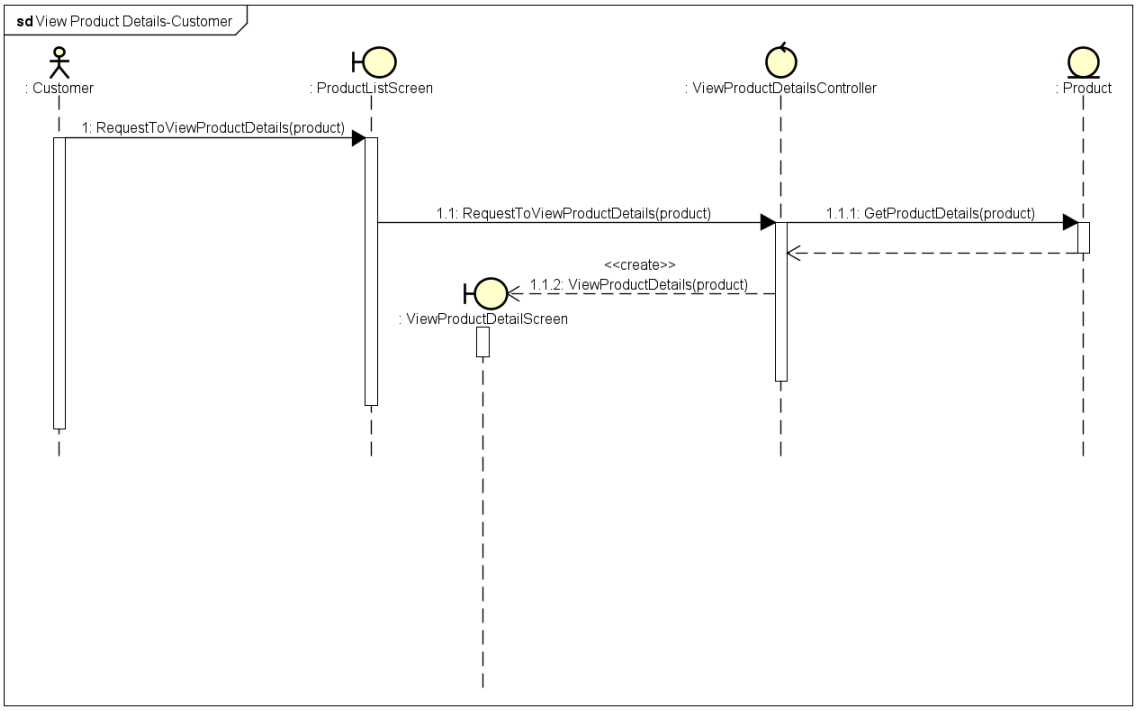
12. Update User


sd Update User
: Administrator
: Dashboard
: UserController
user : User
1: selectUserFromList()
1.1: selectUserFromList()
<<create>>
1.1.1: UserInformationForm(user)
: UserInformationForm
2: submitUpdatedUserInformationForm()
2.1: submitUpdatedUserInformationForm()
2.1.1: checkValidityOfUpdatedInformation()
2.1.2: save()
2.1.3: notifyUserViaEmail()
alt
[change block state of user]
3: changeBlockState()
3.1: changeBlockState()
3.1.1: save()
3.1.2: notifyUserViaEmail()

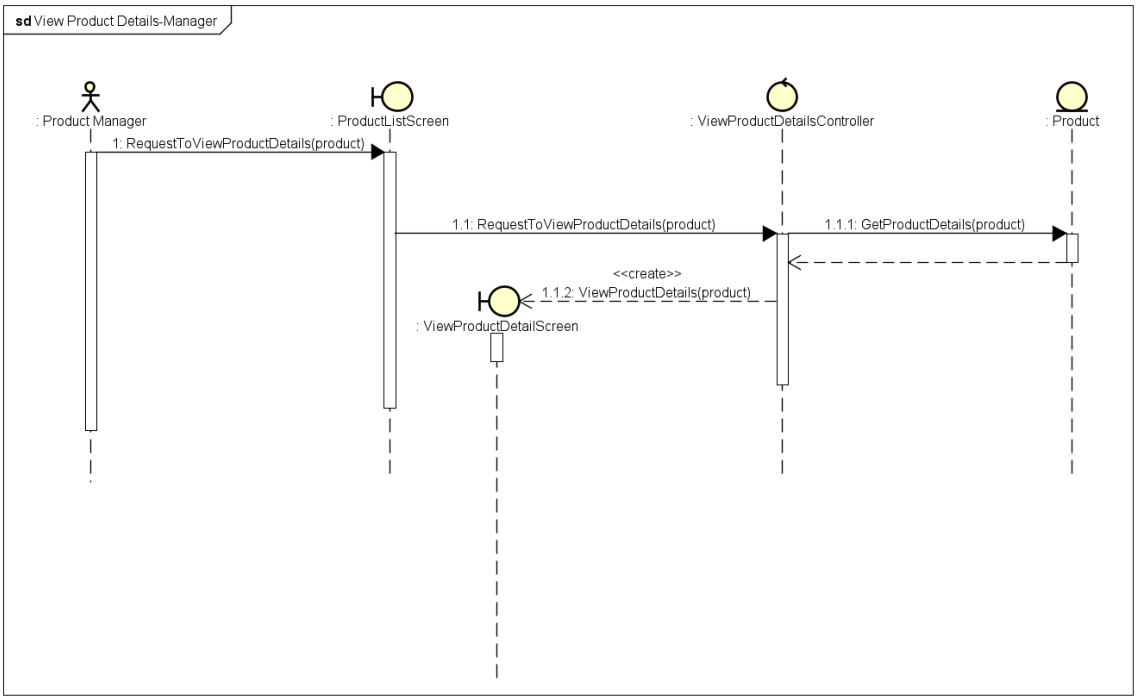
13. View cart



14. View Product Details - Customer

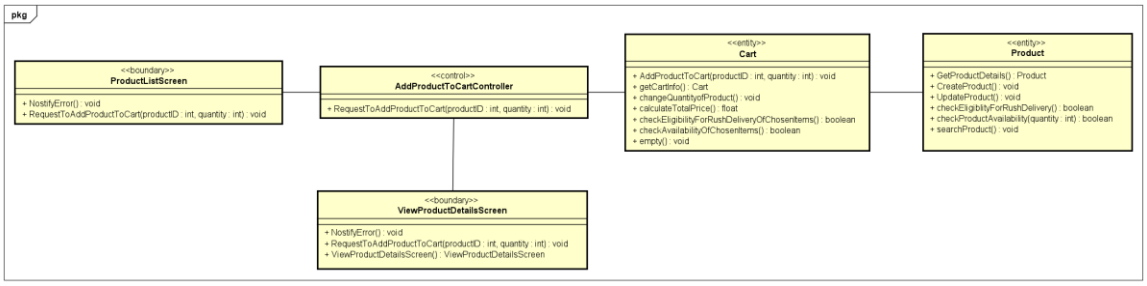


15. View Product Details – Manager

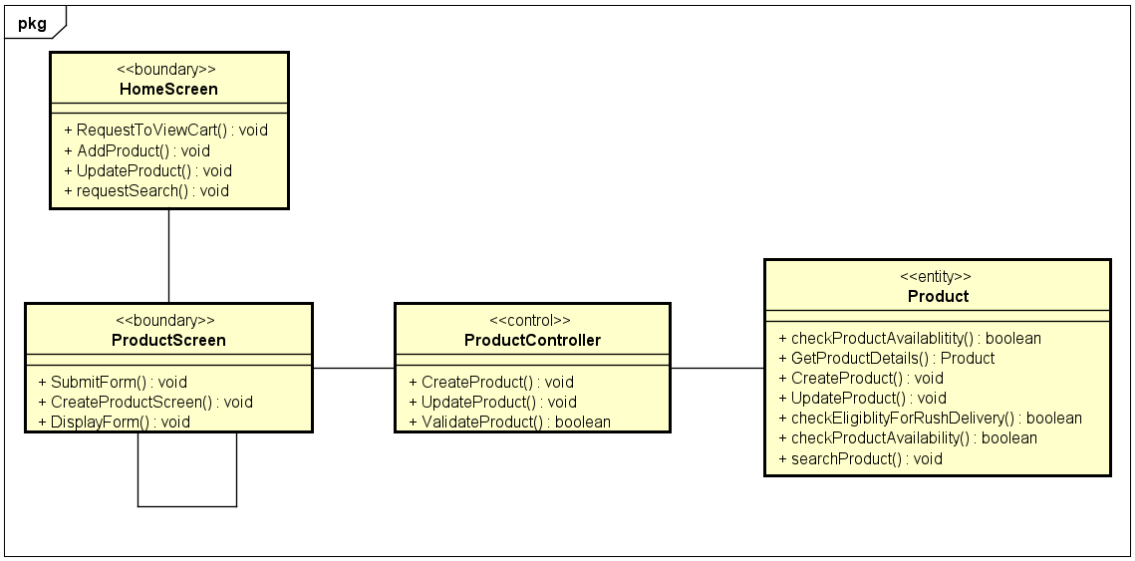


- ***Analysis Class Diagrams***

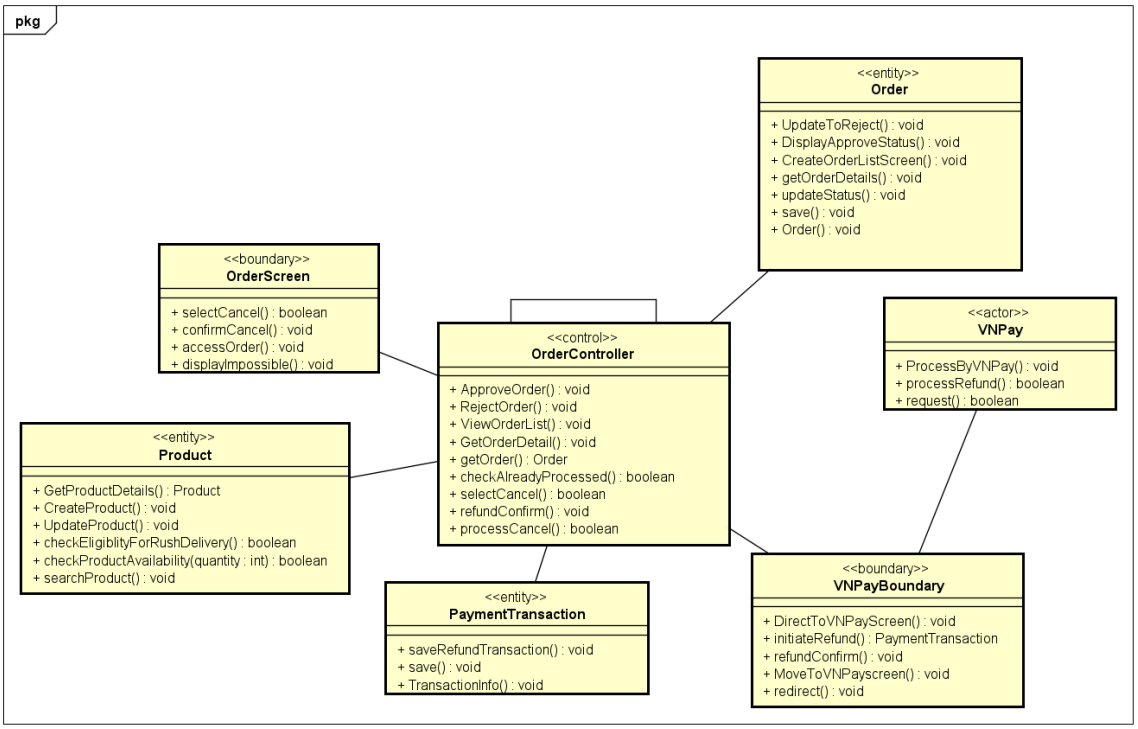
1. Add Products to Cart



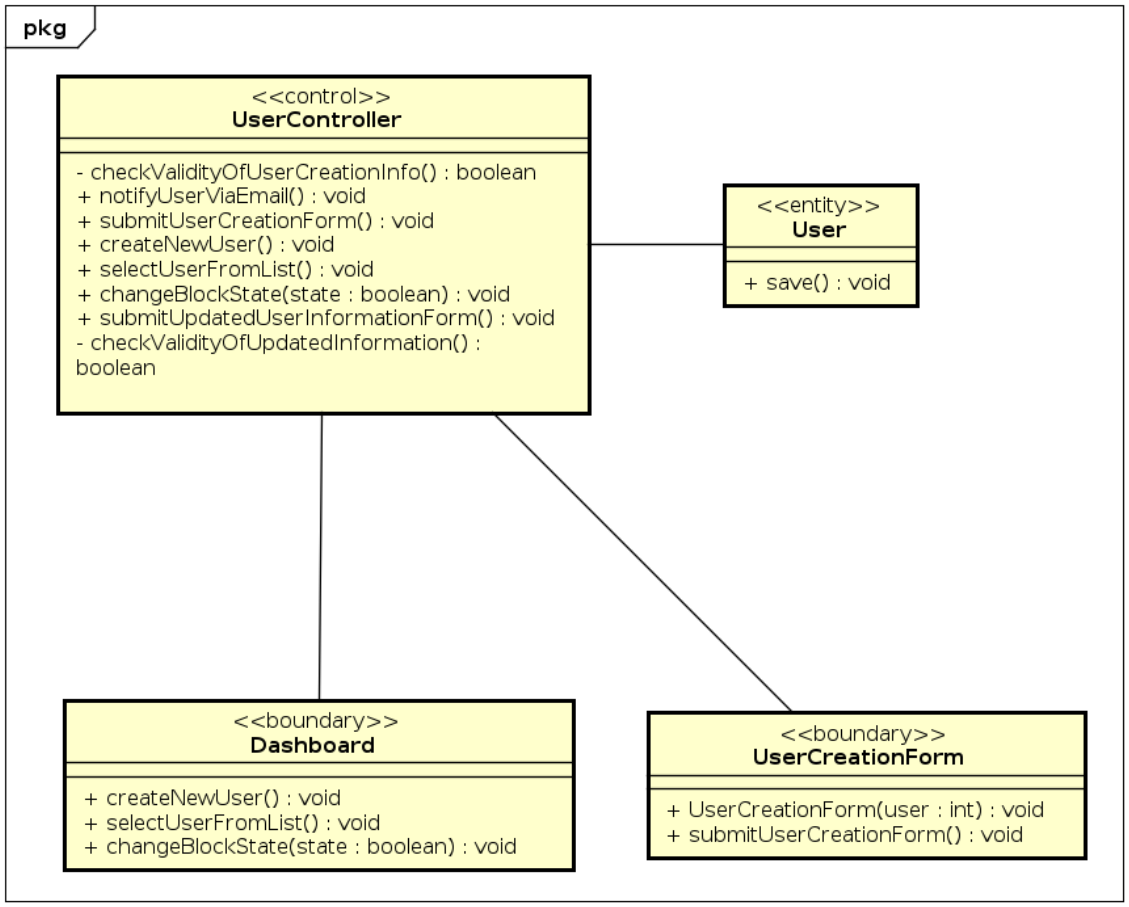
2. Add Products



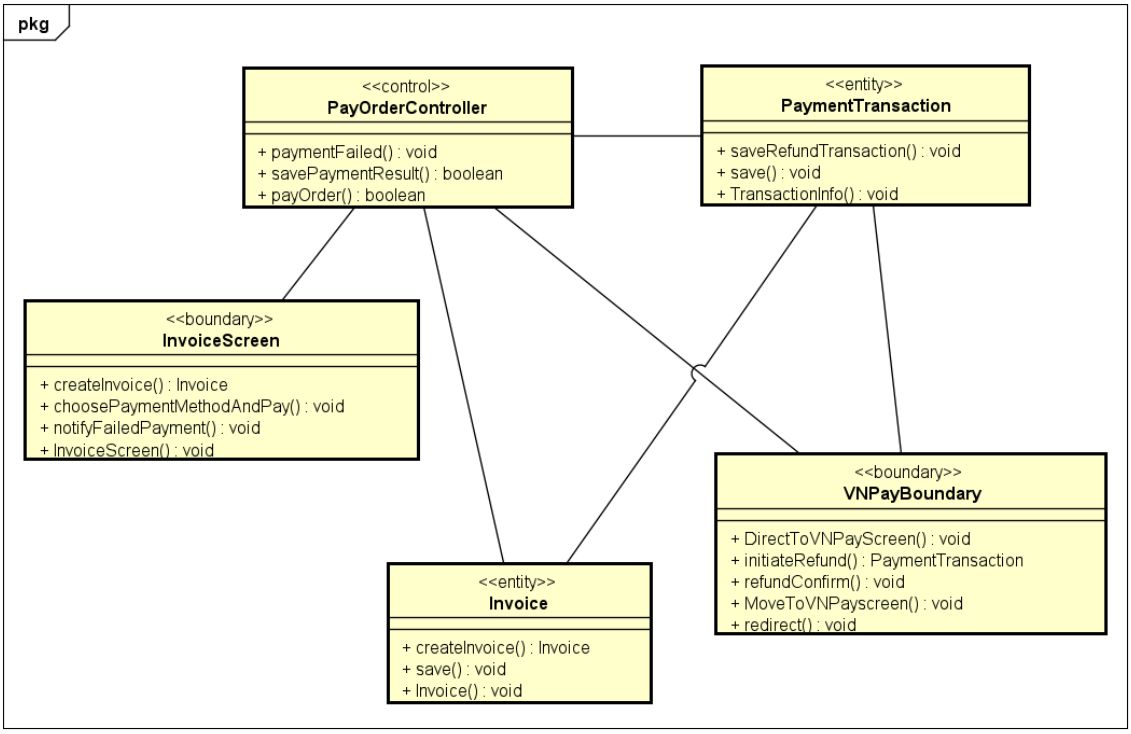
3. Cancel Order



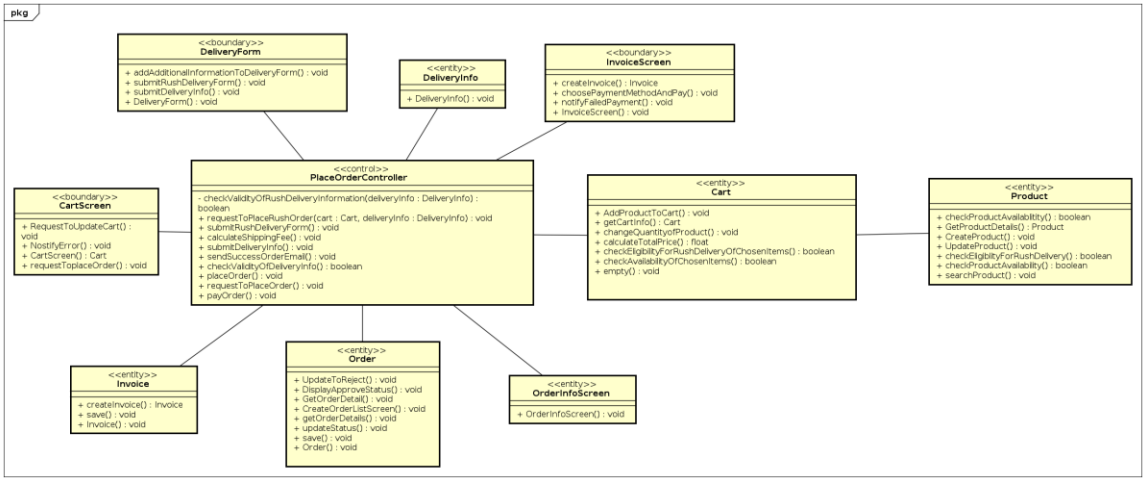
4. Create New User


pkg
<<control>>
UserController
- checkValidityOfUserCreationInfo() : boolean
+ notifyUserViaEmail() : void
+ submitUserCreationForm() : void
+ createNewUser() : void
+ selectUserFromList() : void
+ changeBlockState(state : boolean) : void
+ submitUpdatedUserInformationForm() : void
- checkValidityOfUpdatedInformation() :
boolean
<<entity>>
User
+ save() : void
<<boundary>>
Dashboard
+ createNewUser() : void
+ selectUserFromList() : void
+ changeBlockState(state : boolean) : void
<<boundary>>
UserCreationForm
+ UserCreationForm(user : int) : void
+ submitUserCreationForm() : void

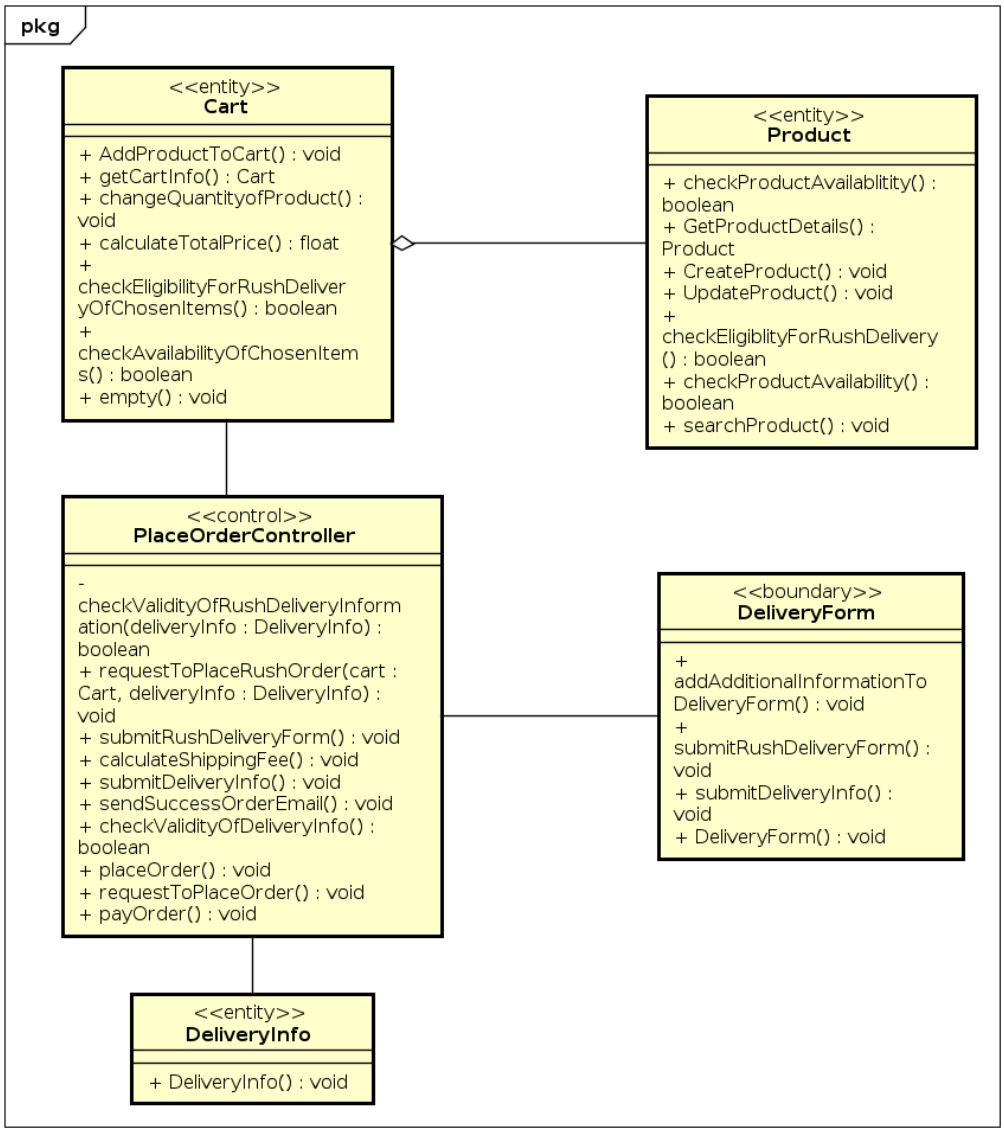
5. Pay Order



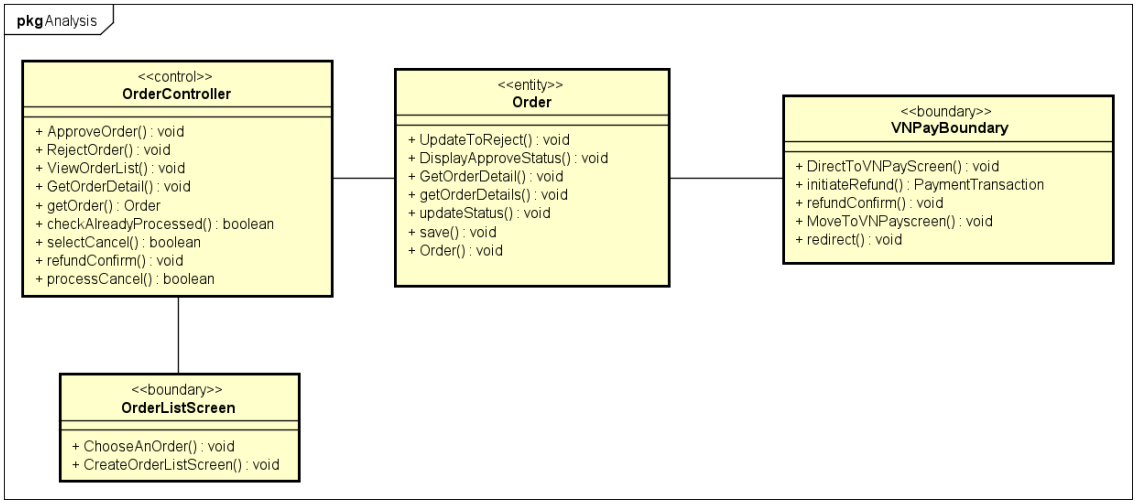
6. Place Order



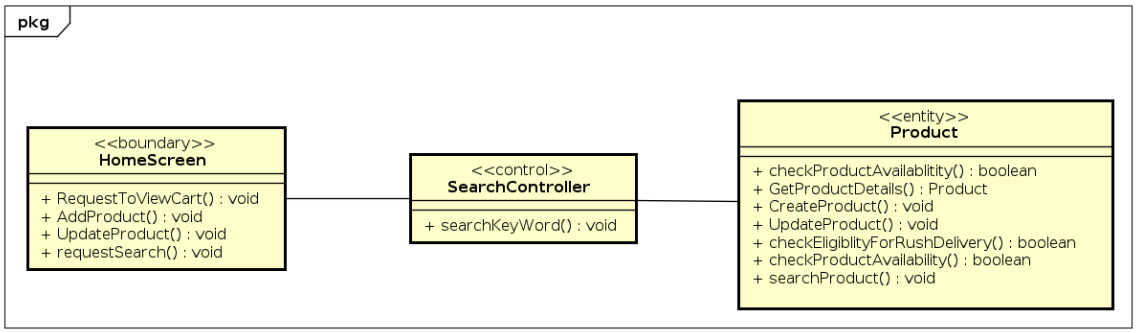
7. Place Rush Order



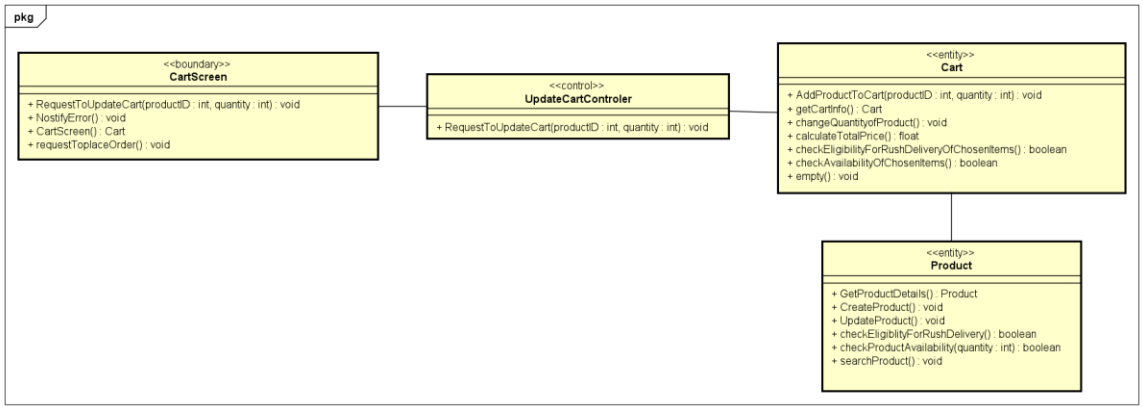
8. Process pending order



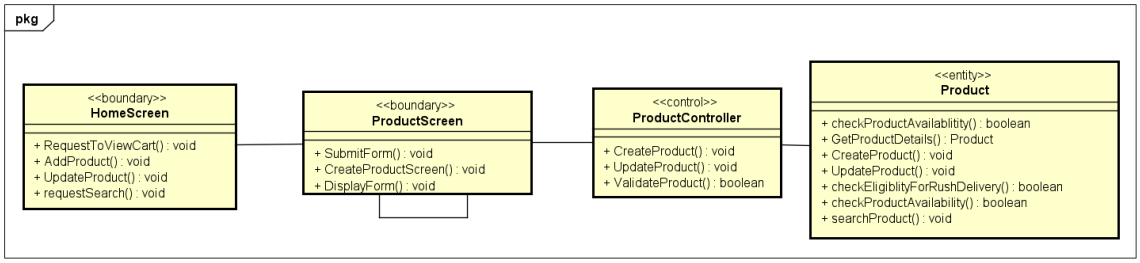
9. Search Products



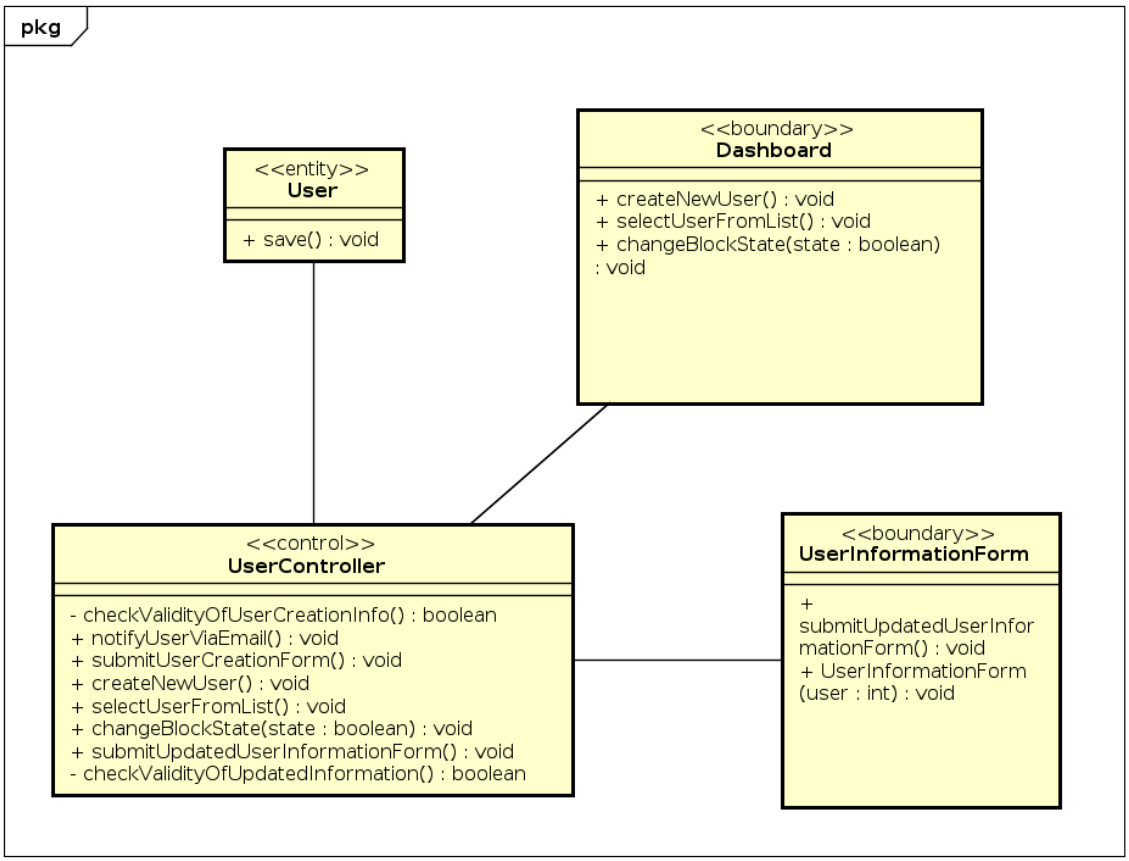
10. Update Cart



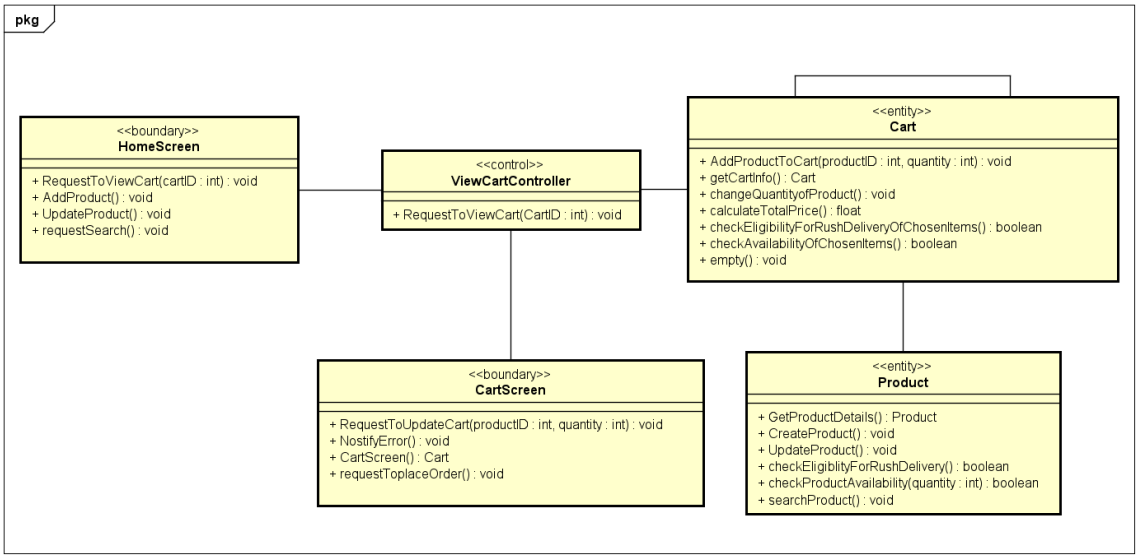
11. Update Products



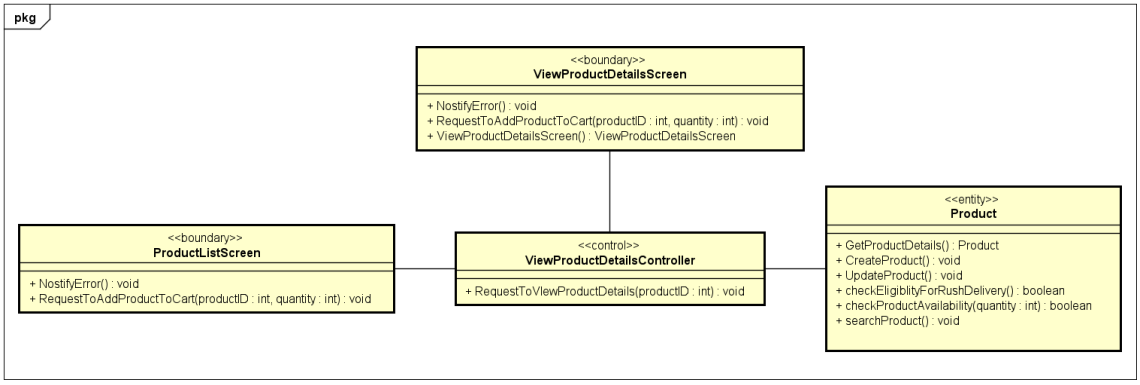
12.Update User



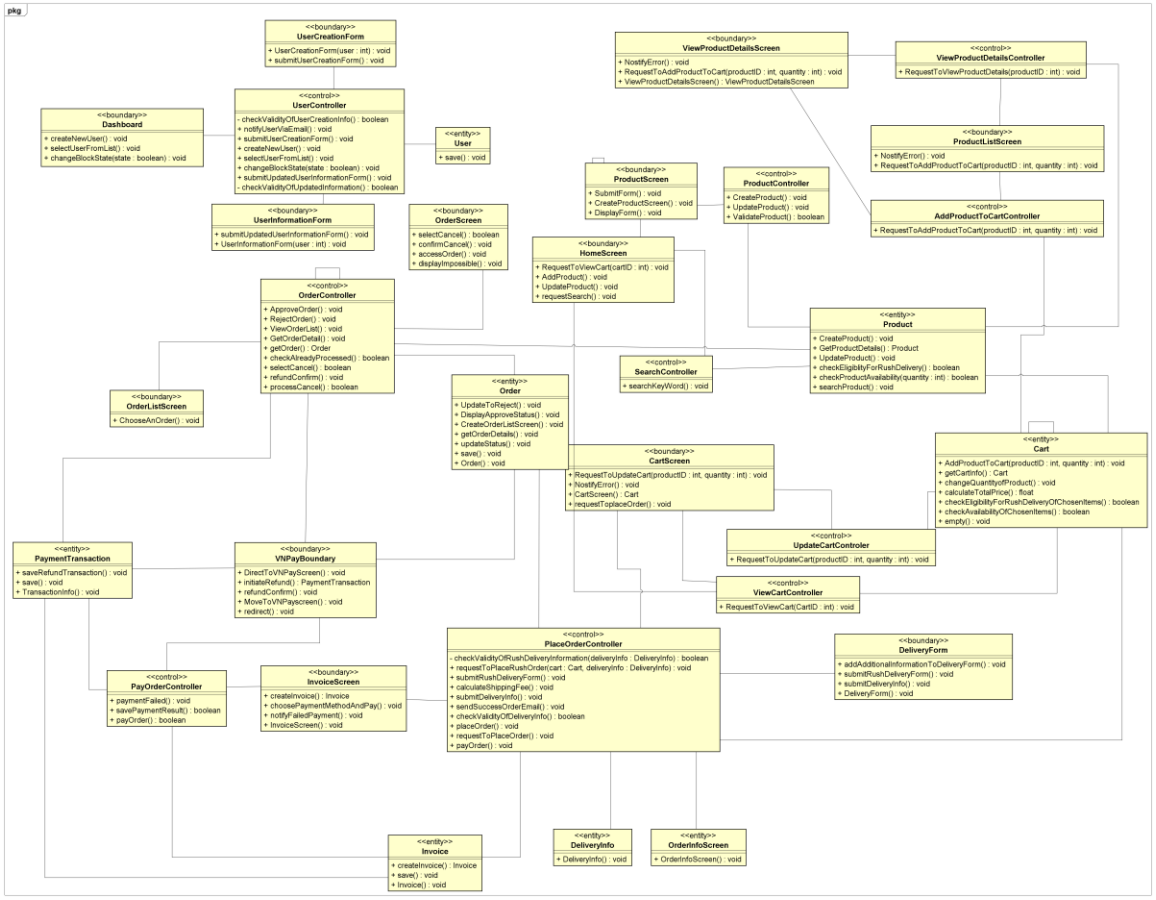
13.View Cart



14. View Product Details



- ***Unified Analysis Class Diagram***



- ***Security Software Architecture***

In a three-tier e-commerce setup built with ReactJS on the frontend and Spring Boot on the backend, combining HTTPS and JWT creates a strong security baseline. HTTPS encrypts all traffic between client and server, preventing interception or tampering. Meanwhile, JWT delivers a stateless, trustworthy way to authenticate users and control access.

a. Presentation Tier (ReactJS)

The React application enforces HTTPS for every API call—redirecting any HTTP requests and relying on a valid SSL/TLS certificate on the server. After a successful login, the backend issues a signed JWT that the client stores securely (for example, in local storage or an HTTP-only cookie). All subsequent requests include this token in the `Authorization: Bearer <token>` header to prove the user's identity.

b. Business Logic Tier (Spring Boot)

Spring Boot also requires HTTPS on all endpoints, redirecting HTTP and using a proper SSL/TLS setup. On login, it creates a JWT containing user ID, roles, and expiry, signed

with a secret or key pair. For protected routes, it verifies the token's signature and claims, enforces role-based access controls based on those claims, and checks expiration. A refresh mechanism issues new access tokens when old ones expire, preserving session continuity without forcing users to log in repeatedly.

c. Data Access Tier (Database)

Although encryption and tokens operate above the database layer, they ensure only authenticated and authorized requests reach it. To protect data at rest, passwords are hashed (e.g., bcrypt), and any payment details are stored as tokens from a payment gateway rather than raw card numbers. You can further encrypt sensitive fields (like emails or addresses) using AES-256. Access controls drawn from JWT claims ensure that queries return only the data the calling user is permitted to see.

Together, HTTPS and JWT in this architecture guarantee that all data moving between React and Spring Boot remains confidential and tamper-proof, while authentication stays efficient and stateless providing a secure, reliable foundation for your e-commerce platform.

● Detailed Design

● *User Interface Design*

<Suppose that you design a Graphical User Interface (GUI)>

● Screen Configuration Standardization

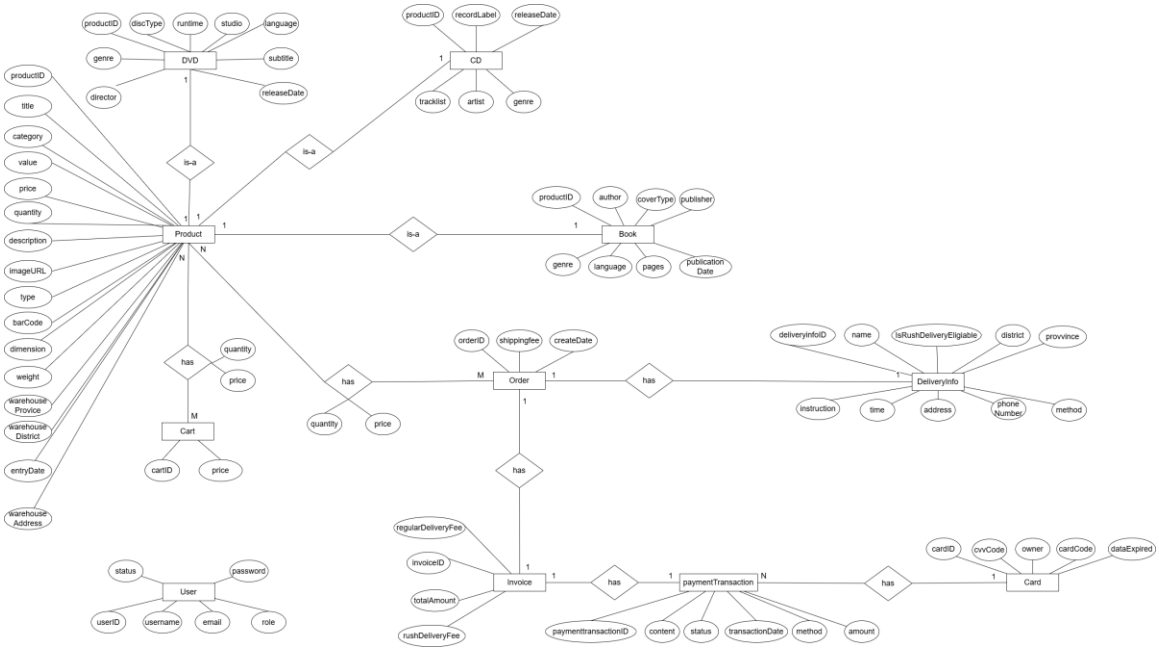
● Screen Transition Diagrams

● Screen Specifications

<Screen images should be included in the screen specifications>

● *Data Modeling*

● Conceptual Data Modeling



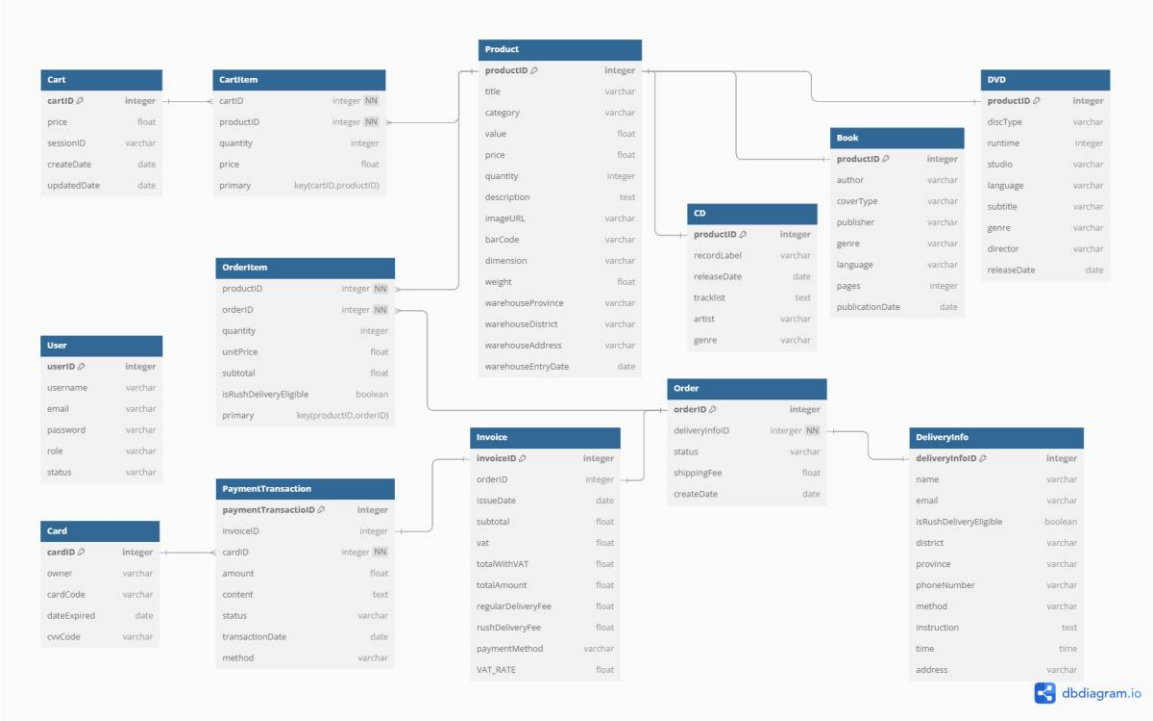
● Database Design

4.2.2.1 Database Management System

We decide to use SQLite as Database Management System.

SQLite3 is a lightweight, serverless, self-contained RDBMS that stores all data in a single file. It's ideal for mobile apps, embedded systems, and applications requiring fast, low-overhead data management. It offers ACID compliance and cross-platform support.

4.2.2.2 Database Diagram



4.2.2.3 Database Detail Design

Table 1. Product

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		productID	Integer	Yes	ID, auto increment
2			title	VARCHAR	Yes	Name of the product
3			category	VARCHAR	Yes	Product category (Book, CD, DVD, or LP record)
4			value	FLOAT	Yes	Base price of the product before VAT
5			price	FLOAT	Yes	Selling price of the product

6			quantity	Integer	Yes	Available quantity in stock
7			description	VARCHAR	No	Product description
8			imageURL	VARCHAR	No	The URL for the image of the product item
9			barCode	VARCHAR	No	Unique barcode for the product
10			warehouseEntryDate	DATE	No	Date when the product entered the warehouse
11			dimensions	VARCHAR	No	Physical dimensions (e.g., 10x20x5 cm)
12			weight	FLOAT	No	Weight in kilograms
13			warehouseProvince	VARCHAR	No	Province of warehouse storing the product
14			warehouseDistrict	VARCHAR	No	District of warehouse storing the product
15			warehouseAddress	VARCHAR	No	Detailed address of warehouse storing the product

Table 2. CD

#	PK	FK	Column Name	Data type	Mandatory	Description
1		x	productID	INTEGER	Yes	ID, same as ID of Product of which type is CD

2			artist	VARCHAR (100)	Yes	Name of the artist or band
3			recordLabel	VARCHAR (100)	Yes	Name of the record label
4			tracklist	VARCHAR (1000)	No	List of tracks in the CD
5			genre	VARCHAR (100)	Yes	Music genre of the CD
6			releaseDate	DATE	Yes	Official release date of the CD

Table 3. Book

#	PK	FK	Column Name	Data type	Mandatory	Description
1		x	productID	INTEGER	Yes	ID, same as ID of Product of which type is Book
2			author	VARCHAR(100)	Yes	Author of the book
3			coverType	VARCHAR(45)	Yes	Type of cover (paperback or hardcover)
4			publisher	VARCHAR(100)	Yes	Publisher of the book
5			publicationDate	DATE	Yes	Date of publication
6			pages	INTEGER	Yes	Number of pages
7			language	VARCHAR(45)	Yes	Language of the book
8			genre	VARCHAR(100)	Yes	Genre of the book

Table 4. DVD

#	PK	FK	Column Name	Data type	Mandatory	Description
1		x	productID	INTEGER	Yes	ID, same as ID of Product of which type is DVD
2			discType	VARCHAR(45)	Yes	Type of disc (e.g: Blu-ray, HD-DVD)
3			director	VARCHAR(100)	Yes	Name of the artist
4			runtime	VARCHAR(20)	Yes	Total runtime of the DVD
5			studio	VARCHAR(100)	Yes	Name of the studio
6			language	VARCHAR(45)	Yes	Language of the content in DVD
7			subtitle	VARCHAR(45)	Yes	Language subtitle
8			releaseDate	DATE	Yes	Release date of the DVD
9			genre	VARCHAR(100)	Yes	The genre of the DVD

Table 5. DeliveryInfo

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		deliveryinfoID	INTEGER	Yes	ID, auto increment
2			recipientName	VARCHAR(100)	Yes	Name of the recipient
3			email	VARCHAR(100)	Yes	Email address

4			phoneNumber	VARCHAR(20)	Yes	Phone number
5			province	VARCHAR(45)	Yes	Province/city
6			district	VARCHAR(45)	Yes	District
7			address	VARCHAR(200)	Yes	Delivery address
8			deliveryMethod	VARCHAR(20)	Yes	Delivery method (Regular/Rush)
9			deliveryTime	FLOAT	No	Requested time for rush delivery
10			isRushDeliveryEligible	BOOLEAN	Yes	Indicates if address is eligible for rush delivery
11			deliveryInstructions	VARCHAR(255)	No	Special delivery instructions

Table 6. Invoice

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		invoiceID	INTEGER	Yes	Unique Identifier for the invoice
2		x	orderID	INTEGER	Yes	Order associated with this invoice (FK to Order)
3			issueDate	DATE	Yes	Date when invoice was created
4			subtotal	FLOAT	Yes	Total price excluding VAT

5			VAT	FLOAT	Yes	Value Added Tax amount (10% of subtotal)
6			totalWithVAT	FLOAT	Yes	Subtotal + VAT
7			regularDeliveryFee	FLOAT	Yes	Fee for regular delivery items
8			rushDeliveryFee	FLOAT	Yes	Fee for rush delivery items
9			totalAmount	FLOAT	Yes	Final amount to be paid
10			paymentMethod	VARCHAR(45)	Yes	Method of payment
11			VAT_RATE	FLOAT	Yes	Value-added tax rate (constant 10%)

Table 7. Payment Transaction

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		transactionID	INTEGER	Yes	Unique Identifier for the transaction
2			amount	FLOAT	Yes	Payment amount
3			paymentMethod	VARCHAR(45)	Yes	Method of payment
4			transactionDate	DATE	Yes	Date of transaction
5			status	VARCHAR(20)	Yes	Transaction status (Pending, Success, Failed…)
6			content	VARCHAR(255)	No	Transaction details
7		x	cardID	INTEGER	Yes	ID of used card

8		x	invoiceID	INTEGER	Yes	FK to Invoice

Table 8. User

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		userID	INTEGER	Yes	Randomly auto-generated, exclusive
2			username	VARCHAR(50)	Yes	Exclusively exists to identify users using the software
3			email	VARCHAR(100)	No	Saved when using Gmail for registration
4			password	VARCHAR(255)	Yes	Must be hashed before stored into database
5			role	VARCHAR(45)	Yes	User role (Administrator / Product Manager)
6			status	BOOLEAN	Yes	True = Unblocked, False = Blocked

Table 9. Order

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		orderID	INTEGER	Yes	Unique identifier for the order
2			status	VARCHAR	Yes	Current status: "pending," "approved," "rejected," "canceled".
3			shippingFee	FLOAT	Yes	Calculated VAT amount (10% of subtotal)

4		x	deliveryInfoID	INTEGER	Yes	Reference to delivery information (FK to DeliveryInfo)

Table 10. OrderItem

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x	x	orderID	INTEGER	Yes	Reference to the order (FK to Order)
2	x	x	productID	INTEGER	Yes	ID of the product in the order
3			quantity	INTEGER	Yes	Quantity of the product ordered
4			unitPrice	FLOAT	Yes	Price of the product at time of order
5			subtotal	FLOAT	Yes	Total price for this item (unitPrice × quantity)
6			isRushDeliveryEligible	BOOLEAN	Yes	Whether this item is eligible for rush delivery
7			weight	FLOAT	No	Weight of one unit of this product

Table 111. Cart

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		cartID	INTEGER	Yes	Unique identifier for the cart
2			sessionID	VARCHAR	Yes	Identifier for the user's session

3			createdDate	DATE	Yes	Date when the cart was created
4			updatedDate	DATE	Yes	Date when the cart was last updated

Table 122. CartItem

#	PK	FK	Column Name	Data type	Mandatory	Description
1	X	X	cartID	INTEGER	Yes	ID of the cart containing this item
2	X	X	productID	INTEGER	Yes	ID of the product in the cart
3			quantity	INTEGER	Yes	Quantity of the product
4			price	FLOAT	Yes	Selling price of the product

Table 133. Card

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		cardID	INTEGER	Yes	Unique identifier for the card
2			cardCode	VARCHAR	Yes	Credit card number
3			owner	VARCHAR	Yes	Name of the cardholder
4			cvvCode	VARCHAR	Yes	Card verification value/code
5			dataExpired	DATE	Yes	Expiration date of the card

SQLite3 scripts to create database for AIMS project ---

CREATE TABLE "Product" ("productID" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "title" TEXT NOT NULL, "category" TEXT NOT NULL, "value" REAL NOT NULL, "price" REAL NOT NULL, "quantity" INT NOT NULL DEFAULT 0, "description" TEXT, "imageURL" TEXT NOT NULL, "barCode" TEXT, "warehouseEntryDate" DATE DEFAULT CURRENT_DATE, "dimensions" TEXT, "weight" REAL, "warehouseProvince" TEXT, "warehouseDistrict" TEXT, "warehouseAddress" TEXT, "createdAt" DATETIME DEFAULT CURRENT_TIMESTAMP, "updatedAt" DATETIME DEFAULT CURRENT_TIMESTAMP);

CREATE TABLE "CD"("productID" INTEGER PRIMARY KEY NOT NULL, "artist" TEXT NOT NULL, "recordLabel" TEXT NOT NULL, "tracklist" TEXT, "genre" TEXT NOT NULL, "releasedDate" DATE, CONSTRAINT "fk_CD_Product1" FOREIGN KEY("productID") REFERENCES "Product"("productID"));

CREATE TABLE "Book"("productID" INTEGER PRIMARY KEY NOT NULL, "author" TEXT NOT NULL, "coverType" TEXT NOT NULL, "publisher" TEXT NOT NULL, "publicationDate" DATE NOT NULL, "numOfPages" INTEGER NOT NULL, "language" TEXT NOT NULL, "genre" TEXT NOT NULL, CONSTRAINT "fk_Book_Product1" FOREIGN KEY("productID") REFERENCES "Product"("productID"));

CREATE TABLE "DVD" ("productID" INTEGER PRIMARY KEY NOT NULL, "discType" TEXT NOT NULL, "director" TEXT NOT NULL, "runtime" TEXT NOT NULL, "studio" TEXT NOT NULL, "language" TEXT NOT NULL, "subtitle" TEXT NOT NULL, "releaseDate" DATE NOT NULL, "genre" TEXT NOT NULL, CONSTRAINT "fk_DVD_Product1" FOREIGN KEY ("productID") REFERENCES "Product"("productID"));

CREATE TABLE "DeliveryInfo" ("deliveryInfoID" INTEGER PRIMARY KEY AUTOINCREMENT, "recipientName" TEXT NOT NULL, "email" TEXT NOT NULL, "phoneNumber" TEXT NOT NULL, "province" TEXT NOT NULL, "district" TEXT NOT NULL, "address" TEXT NOT NULL, "deliveryMethod" TEXT NOT NULL, "deliveryTime" REAL, "isRushDeliveryEligible" BOOLEAN NOT NULL, "deliveryInstructions" TEXT, "createdAt" DATETIME DEFAULT CURRENT_TIMESTAMP, "updatedAt" DATETIME DEFAULT CURRENT_TIMESTAMP);

CREATE TABLE "Card"("cardID" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "cardCode" TEXT NOT NULL, "owner" TEXT NOT NULL, "cvvCode" TEXT NOT NULL, "dateExpired" TEXT NOT NULL);

CREATE TABLE "User" ("userID" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "username" TEXT NOT NULL UNIQUE, "email" TEXT NOT NULL, "password" TEXT NOT NULL, "role" TEXT NOT NULL, "status" BOOLEAN NOT NULL, "createdAt" DATETIME DEFAULT CURRENT_TIMESTAMP, "updatedAt" DATETIME DEFAULT CURRENT_TIMESTAMP);

CREATE TABLE "Cart" ("cartID" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "sessionID" TEXT, "createdAt" DATETIME DEFAULT CURRENT_TIMESTAMP);

CREATE TABLE "CartItem" ("cartID" INTEGER NOT NULL, "productID" INTEGER NOT NULL, "quantity" INTEGER NOT NULL, "price" REAL NOT NULL, PRIMARY KEY ("cartID", "productID"), FOREIGN KEY ("cartID") REFERENCES "Cart"("cartID"), FOREIGN KEY ("productID") REFERENCES "Product"("productID"));

CREATE TABLE "Order" ("orderID" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "status" CHARACTER(15) NOT NULL, "shippingFee" REAL NOT NULL, "deliveryInfoID" INTEGER NOT NULL, FOREIGN KEY ("deliveryInfoID") REFERENCES "DeliveryInfo"("deliveryInfoID"));

CREATE TABLE "OrderItem" ("orderID" INTEGER NOT NULL, "productID" INTEGER NOT NULL, "quantity" INTEGER NOT NULL, "unitPrice" REAL NOT NULL, "subtotal" REAL NOT NULL, "isRushDeliveryEligible" BOOLEAN NOT NULL, "weight" REAL, PRIMARY KEY ("orderID", "productID"), FOREIGN KEY ("orderID") REFERENCES "Order"("orderID"), FOREIGN KEY ("productID") REFERENCES "Product"("productID"));

CREATE TABLE "Invoice" ("invoiceID" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "orderID" INTEGER NOT NULL, "issueDate" DATE NOT NULL, "subtotal" REAL NOT NULL, "VAT" REAL NOT NULL, "totalWithVAT" REAL NOT NULL, "regularDeliveryFee" REAL NOT NULL, "rushDeliveryFee" REAL NOT NULL, "totalAmount" REAL NOT NULL, "paymentMethod" TEXT NOT NULL, "VAT_RATE" REAL NOT NULL, FOREIGN KEY ("orderID") REFERENCES "Order"("orderID"));

CREATE TABLE "PaymentTransaction" ("transactionID" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "amount" REAL NOT NULL, "paymentMethod"

TEXT NOT NULL, "transactionDate" DATE NOT NULL, "status" TEXT NOT NULL, "content" TEXT, "cardID" INTEGER NOT NULL, "invoiceID" INTEGER NOT NULL, FOREIGN KEY ("cardID") REFERENCES "Card"("cardID"), FOREIGN KEY ("invoiceID") REFERENCES "Invoice"("invoiceID"));

CREATE INDEX "idx_product_category" ON "Product"("category"); CREATE INDEX "idx_product_title" ON "Product"("title"); CREATE INDEX "idx_order_status" ON "Order"("status");

CREATE TRIGGER "product_updatedAt_tg" AFTER UPDATE ON "Product" BEGIN UPDATE "Product" SET "updatedAt" = DATETIME('NOW', 'localtime') WHERE "productID" = OLD."productID"; END;

CREATE TRIGGER "user_updatedAt_tg" AFTER UPDATE ON "User" FOR EACH ROW BEGIN UPDATE "User" SET "updatedAt" = DATETIME('NOW', 'localtime') WHERE "userID" = OLD."userID"; END;

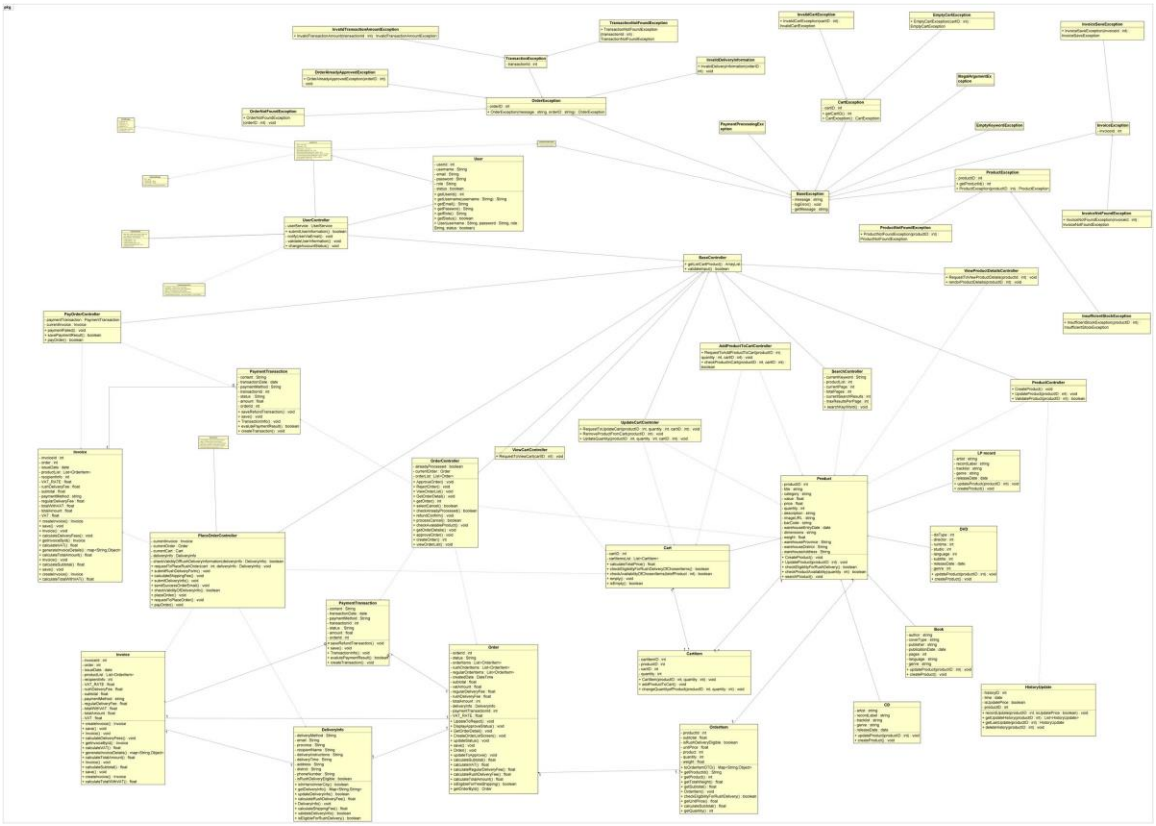
- ***Non-Database Management System Files***

<Provide the detailed description of all non-DBMS files if any and include a narrative description of the usage of each file that identifies if the file is used for input, output, or both, and if the file is a temporary file. Also provide an indication of which modules read and write the file and include file structures (refer to the data dictionary). As appropriate, the file structure information should include the following:

- *Record structures, record keys or indexes, and data elements referenced within the records*
- *Record length (fixed or maximum variable length) and blocking factors*
- *Access method (e.g., index sequential, virtual sequential, random access, etc.)*
- *Estimate of the file size or volume of data within the file, including overhead resulting from file access methods*
- *Definition of the update frequency of the file (If the file is part of an online transaction-based system, provide the estimated number of transactions per unit of time, and the statistical mean, mode, and distribution of those transactions.)*
- *Backup and recovery specifications>*

The group has worked on

- ***Class Design***
- **General Class Diagram**



4.4.3.1 Design Class: Product

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	Unique Identifier for the product
2	title	String	null	Name of the product
3	category	String	null	Product category (Book, CD, DVD, or LP record)
4	value	float	0.0	Base price of the product before VAT
5	price	float	0.0	Selling price of the product
6	quantity	int	0	Available quantity in stock
7	description	String	null	Product description
8	imageURL	String	null	The URL for the image of the product item
9	barCode	String	null	Unique barcode for the product
10	warehouseEntryDate	Date	null	Date when the product entered the warehouse
11	dimensions	String	null	Physical dimensions of the product (e.g., 10x20x5 cm)
12	weight	float	null	Weight of the product in kilograms
13	warehouseProvince	String	null	province of warehouse storing the product
14	warehouseDistrict	String	null	district of warehouse storing the product
15	warehouseAddress	String	null	detailed address of warehouse storing the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	checkProductAvailability	boolean	Checks if the product has sufficient quantity available
2	updateProduct	void	Updates product information in the system
3	checkEligibilityOfRushDelivery	boolean	Checks if product is eligible for rush delivery
4	createProduct	void	Create a new product in the system
5	searchProduct	void	Search for a specific product attributes

- **P**arameter:
  - checkProductAvailability(quantity: int): Boolean
    - quantity: An integer representing the number of products to check if the available inventory is sufficient.
    - Return: true if the available inventory is sufficient, otherwise false.
  - updateProduct(productId: int): void
    - productId: productId must exist
  - searchProduct(searchCriteria: Map<String, Object>): void
    - searchCriteria: A map containing search criteria (field names and values)

- Exception:

  - updateProduct(productId: int): void
    - ProductNotFoundException: If no product with the specified productId is found.
    - ExceedUpdateTimesException: If update more than 30 products a day
    - ExceedUpdatePriceTimesPerDayException: If update price more than twice a day

- Method:

  How to use parameters/attributes:

- checkProductAvailability(quantity: int): Boolean
    - Purpose: This method checks whether the available inventory is sufficient for the requested quantity of a product.
    - Usage: If the available stock is at least equal to the requested quantity, the method returns true; otherwise, it returns false.
  - updateProduct(productId: int): void
    - Purpose: This method updates the details of an existing product based on the provIded information.
    - Usage: If the product exists, the method updates attributes such as name, price, quantity, etc
  - createProduct(): void
    - Purpose: This method creates a new product with the provIded information.
    - Usage: If all required attributes are provIded and valId, the product is successfully created and added to the inventory.
  - checkEligibilityOfRushDelivery(address: String): Boolean
    - Purpose: to check rush delivery support
    - Usage: compare 'address' parameter (from delivery information) and address of product (based on information provIded by Product Manager)
  - searchProduct(searchCriteria: Map<String, Object>): void
    - Purpose: Searches for products matching specific criteria.
    - Usage: Used to find products based on various attributes.
    - Processing:
      - Constructs a database query based on the search criteria
      - Executes the query
      - Returns matching products

- State:

  - If stock > requested quantity → Available.
  - If stock < requested quantity → Not Available.
  - If product_Id is invalId → ProductNotFoundException is raised.

4.4.3.2 Design Class: Book

Table 1. Attribute design

#	Name	Data type	Defaul t value	Description

1	author	String	null	Author of the book
2	coverType	String	null	Type of cover (paperback or hardcover)
3	publisher	String	null	Publisher of the book
4	publicationDate	Date	null	Date of publication
5	pages	int	0	Number of pages
6	language	String	null	Language of the book
7	genre	String	null	Genre of the book

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	vold	Updates product information in the system
2	createProduct	vold	Create a new product in the system

- **Parameter: None Exception:**
    - **updateProduct(productId: int): vold**
        - **ExceedUpdatedTimesException: If update more than 30 products a day**
        - **ExceedUpdateTimesPerDayException: If update more than twice a day**

- **Method:**

How to use parameters/attributes:

- **updateProduct(productId: int): vold**
    - **Purpose: Updates attributes such as price, quantity, and description,.. of book.**
    - **Usage: If the book exists, the method updates attributes such as name, price, quantity, etc**

- **createProduct(): vold**
    - **Purpose: Creates a new Book with specified attributes such as name, price, stock quantity, and description.**
    - **Usage: If all required attributes are provlded and valld, the book is successfully created and added to the inventory.**

4.4.3.3Design Class: CD

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	artist	String	null	Name of the artist or band
2	recordLabel	String	null	Name of the record label
3	tracklist	String	null	List of tracks in the CD
4	genre	String	null	Music genre of the CD
5	releaseDate	Date	0	Official release date of the CD

Table 2. Operation design

#	Name	Retur n type	Description (purpose)
1	updateProduct	vold	Updates product information in the system
2	createProduct	vold	Create a new product in the system

- **Parameter: None Exception:**
    - **updateProduct(productId: int): vold**
        - **ExceedUpdatedTimesException: If update more than 30 products a day**
        - **ExceedUpdateTimesPerDayException: If update more than twice a day**
- **Method:**

How to use parameters/attributes:

- **updateProduct(productId: int): void**
  - **Purpose: Updates attributes such as price, quantity, and description,.. of CD.**
  - **Usage: If the CD exists, the method updates attributes such as name, price, quantity, etc**
- **createProduct(): void**
  - **Purpose: Creates a new CD with specified attributes such as name, price, stock quantity, and description.**
  - **Usage: If all required attributes are provIded and valId, the CD is successfully created and added to the inventory.**

4.4.3.4 Design Class: DVD

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	discType	String	null	Type of disc (e.g: Blu-ray, HD-DVD)
2	director	String	null	Name of the artist
3	runtime	String	null	Total runtime of the DVD
4	studio	String	null	Name of the studio
5	language	String	null	Language of the content in DVD
5	subtitle	String	null	Language subtitle
6	releaseDate	Date	null	Release date of the DVD
7	genre	String	null	The genre of the DVD

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	void	Updates product information in the system

2	createProduct	vold	Create a new product in the system

- **Parameter: None Exception:**
    - **updateProduct(productId: int): vold**
        - **ExceedUpdatedTimesException: If update more than 30 products a day**
        - **ExceedUpdateTimesPerDayException: If update more than twice a day**

- **Method:**

  How to use parameters/attributes:

    - **createProduct():**
        - **Purpose: Creates a new DVD with specified attributes such as name, price, stock quantity, and description.**
        - **Usage: If all required attributes are provlded and valld, the DVD is successfully created and added to the inventory.**
    - **updateProduct(productId: int): vold**
        - **Purpose: Updates attributes such as price, quantity, and description,.. of DVD.**
        - **Usage: If the DVD exists, the method updates attributes such as name, price, quantity, etc**

4.4.3.5 Design Class: LP record

Table 1. Attribute design

#	Name	Data type	Defaul t value	Description
1	artist	String	null	Name of the artist or band
2	recordLabel	String	null	Name of the record label
3	tracklist	String	null	List of tracks in the LP record
4	genre	String	null	Music genre of the LP record

5	releaseDate	Date	0	Official release date of the LP record

Table 2. Operation design

#	Name	Retur n type	Description (purpose)
1	updateProduct	vold	Updates product information in the system
2	createProduct	vold	Create a new product in the system

- **Parameter: None Exception:**
  - **updateProduct(productId: int): vold**
    - **ExceedUpdatedTimesException: If update more than 30 products a day**
    - **ExceedUpdateTimesPerDayException: If update more than twice a day**
- **Method:**

How to use parameters/attributes:

- **createProduct():**
  - **Purpose: Creates a new DVD with specified attributes such as name, price, stock quantity, and description.**
  - **Usage: If all required attributes are provlded and valld, the DVD is successfully created and added to the inventory.**
- **updateProduct(productId: int): vold**
  - **Purpose: Updates attributes such as price, quantity, and description,.. of DVD.**
  - **Usage: If the DVD exists, the method updates attributes such as name, price, quantity, etc**

4.4.3.6 Design Class: History update

Table 1. Atrribute design

#	Name	Data type	Default value	Description

1	historyId	int	null	Unique Identifier for the product
2	time	Date	null	The time product manager update the product

3	isUpdatePrice	boolean	null	The price of the product is update or not
4	productId	int	null	The Id of the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	recordUpdate	void	Records the update history of a product when changes are made
2	getUpdateHistory	List<HistoryUpdate>	Retrieves the update history of a product based on productId.
3	getLastUpdate	History Update	Retrieves the most recent update of a product.
4	deleteHistory	void	Deletes the entire update history of a product based on productId.

- **Parameter:**

- **recordUpdate(productId: int, isUpdatePrice: boolean): vold**
    - **productId: An integer representing the unique Identifier of the updated product.**
    - **isUpdatePrice: A boolean indicating whether the product price was updated.**
    - **Return: This method does not return a value. It records an update entry in the history log.**
- **getUpdateHistory(productId: int): List<HistoryUpdate>**
    - **productId: An integer representing the unique Identifier of the product.**
    - **Return: A list of HistoryUpdate objects containing all update records for the specified product. If no history exists, it returns an empty list.**
- **getLastUpdate(productId: int): HistoryUpdate**
    - **productId: An integer representing the unique Identifier of the product.**
    - **Return: A HistoryUpdate object containing the latest update details of the specified product. If no update history exists, it returns null or raises an exception.**
- **deleteHistory(productId: int): vold**
    - **productId: An integer representing the unique Identifier of the product.**
    - **Return: This method does not return a value. It deletes all update history records related to the specified product.**

- **Exception:**

    - **recordUpdate(productId: int, isUpdatePrice: boolean): vold**
        - **ProductNotFoundException: If no product with the specified product_Id is found.**
    - **getUpdateHistory(productId: int): List<HistoryUpdate>**
        - **ProductNotFoundException: If no product with the specified product_Id is found.**
    - **getLastUpdate(productId: int): HistoryUpdate**
        - **ProductNotFoundException: If no product with the specified product_Id is found.**
    - **deleteHistory(productId: int): vold**
        - **ProductNotFoundException: If no product with the specified product_Id is found.**

Method:

- **How to use parameters/attributes:**
- **recordUpdate(productId: int, isUpdatePrice: boolean): vold**
    - **Purpose: Records the update history of a product when changes are made.**
    - **Usage: When a product is updated (either information or price), this method creates a new record with time as the update timestamp and isUpdatePrice indicating whether the price was updated.**
  - **getUpdateHistory(productId: int): List<HistoryUpdate>**
    - **Purpose: Retrieves the update history of a product based on productId.**
    - **Usage: If the product has an update history, the method returns a list of previous updates; otherwise, it may return an empty list.**
  - **getLastUpdate(productId: int): HistoryUpdate**
    - **Purpose: Retrieves the most recent update of a product.**
    - **Usage: The method finds the latest update record of productId and returns it. If no history exists, it may return null or raise an error.**
  - **deleteHistory(productId: int): vold**
    - **Purpose: Deletes the entire update history of a product based on productId.**
    - **Usage: When the update history of a product needs to be removed (e.g., when the product is deleted from the system), this method deletes all related records.**

4.4.3.7 Design Class:ViewProductDetailsController

Table 1. Operation design

#	Name	Return type	Description (purpose)
1	RequestToViewProductDetails	void	Receives a request from the user to view product details based on productId.
2	renderProductDetails	void	Retrieves product information from

- **Parameter**
    - **requestToViewProductDetails(productId: int): void**
        - **productId: The unique Identifier of the product to view.**
    - **renderProductDetails(productId: String): void**
        - **productId: The unique Identifier of the product to view.**
- **Exception**
    - **requestToViewProductDetails(productId: int): void**
        - **ProductNotFoundException if the product does not exist in the database.**
    - **renderProductDetails(productId: String): void**
        - **None**

Method: How to Use Parameters/Attributes

- **RequestToViewProductDetails((productId: int): void**
    - **Purpose: Sends a request to retrieve product details.**
    - **Usage: If the product exists, its details are displayed on the screen. If the product does not exist, a ProductNotFoundException** is raised.
- **renderProductDetails**
    - **Purpose: Retrieves and displays product details.**
    - **Usage: Called internally by requestToViewProductDetails.**
- **State**

- **If product exists → Product details are displayed on the screen.**
- **If product does not exist → ProductNotFoundException is raised, and an error message is displayed.**

4.4.3.8 Design Class: AddProductToCartController

Table 1. Operation design

#	Name	Return type	Description (purpose)
1	requestToAddProductToCart	vold	Processes request to add product to cart
2	checkProductInCart	boolean	Checks if a specific product is already in the cart.

- **Parameter:**
  - **requestToAddProductToCart(productId: String, quantity: int, cartID: int): vold**
    - **productId: The unique Identifier of the product to add to the cart.**
    - **quantity: The number of units to add.**
    - **cartID: The unique Identifier of the cart to add products.**
    - **Return: None**
  - **checkProductInCart(productId: String, cartID: Cart): boolean**
    - **productId: The unique Identifier of the product to check.**
    - **cartID: The unique Identifier of the cart to add products.**
    - **Return: true if the product is present, otherwise false**

- **Exception:**
  - **requestToAddProductToCart productId: String, quantity: int, cartID: int): vold**
  - **ProductNotFoundException: If the specified product does not exist.**
    - **InsufficientStockException: If the requested quantity exceeds available stock.**

- **Method**:

  How to Use Parameters/Attributes

  - **requestToAddProductToCart(productId: String, quantity: int, cartID: int): void**
    - **Purpose: This method processes a request to add a product to the shopping cart.**
    - **Usage**:
      - If the product exists in inventory and the requested quantity is available, the product is added to the cart, and the inventory stock is reduced accordingly.
      - If the product does not exist, a **ProductNotFoundException** is raised.
      - If the requested quantity exceeds available stock, an **InsufficientStockException** is raised.
  - checkProductInCart(productId: String, cartID: Cart): boolean
    - Purpose: This method checks if a specific product is already in the shopping cart.
    - Usage: Call insIde **requestToAddProductToCart(productId: String, quantity: int, cartID: int): void**
      - Retrieve the product from currentCart using productId.
      - If the product is found, return true.
      - If the product is not found, return false.

- **State**

  - If **product exists** and **stock ≥ requested quantity**, product is added to cart, stock is updated.
  - If **product exists** but **stock < requested quantity** , **InsufficientStockException** is raised.
  - If **productId is invalId**, **ProductNotFoundException** is raised.

4.4.3.9 Design Class: UpdateCartController

Table 1. Example of operation design

#	Name	Return type	Description (purpose)
1	requestToUpdateCart	boolean	Initiates cart update process
2	updateQuantity	void	Updates the quantity of a specific product in the cart.
3	removeProductFromCart	void	Removes a specific product from the cart.

- **Parameter:**

  - **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
    - **productId: The unique Identifier of the product to update in the cart.**
    - **cartID: The shopping cart that contains the products to update quantity.**
    - **quantity: The new quantity of the product.**
    - **Return: None**
  - **updateQuantity(productId: int, quantity: int, cartID: int): void**
    - **productId: The unique Identifier of the product whose quantity needs to be updated.**
    - **quantity: The updated quantity to be set for the product.**
    - **cartID: The shopping cart that contains the products to update quantity.**
    - **Return: None**
  - **removeProductFromCart(productId: int, cartID: int): void**
    - **productId: The unique Identifier of the product to be removed from the cart.**
    - **cartID: The shopping cart that contains the products to update quantity.**

- **Exception:**

  - **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
    - **ProductNotFoundException: If the specified product does not exist.**
  - **updateQuantity(productId: int, quantity: int, cartID: int): void**
    - **InsufficientStockException: If the requested quantity exceeds available stock.**

Method:

How to Use Parameters/Attributes

- **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
  - **Purpose: This method initiates the cart update process when a user modifies the quantity or removes an item from the cart.**
  - **Usage**:
    - Retrieve the product using productId. If the product is not found, throw ProductNotFoundException.
    - Update the cart.
- **updateQuantity(productId: int, quantity: int, cartID: int): void**
  - **Purpose**: Updates the quantity of a specific product in the shopping cart.
  - **Usage**: Call insIde **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
    - Retrieve the product from currentCart using productId.
    - VaIIdate the quantity (e.g., ensure it's a positive integer).
    - Update the product quantity in currentCart.If the requested quantity exceeds available stock, throw
    - **InsufficientStockException**
- **removeProductFromCart(productId: int, cartID: int): void**
  - **Purpose**: Removes a product from the shopping cart based on its unique Identifier.
  - **Usage:** Call insIde **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
    - Retrieve the product from currentCart using productId.
    - Remove the product from currentCart.
    - Update the cart state accordingly.

- **State:**

- **Checking Stock** – Verifying the stock quantity of the product.
- **Available** – The requested quantity is in stock (stock > requested quantity).
- **Not Available** – Insufficient stock (stock < requested quantity).
- **Error: Product Not Found** – The product ID is invalid (ProductNotFoundException).

4.4.3.10 Design Class: ViewCartController

Table 1. operation design

#	Name	Return type	Description (purpose)
1	requestToViewCart	void	Returns the current cart for viewing

- **Parameter**

  - **requestToViewCart(cartID: int): void**
    - **cartID: The shopping cart ID to be viewed.**
    - **Return: None**

- **Exception**

  - **requestToViewCart(cartID: int): void**
    - **CartNotFoundException: If the shopping cart is not found or has not been initialized**

Method:

How to Use Parameters/Attributes

- **requestToViewCart(cartID: int): void**
  - **Purpose This method processes a request to view the shopping cart.**
  - **Usage: If cartID is valid and contains items, its contents are displayed. If the current cart is null or empty, a CartNotFoundException is raised.**

- **State**:

- **Checking Cart ID: The system verifies whether the cart ID is valid.**
- **Cart Found: The cart is available, and its contents are displayed.**
- **Cart Not Found: If the cart does not exist or is uninitialized, a CartNotFoundException is raised.**

4.4.3.11 Design Class: Cart

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	cartItemsList	List<CartItem>	empty	List of CartItem in the cart
2	cartId	float	0.0	Unique Identifier for the cart

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	calculateTotalPrice	float	Calculates and returns the total price
2	empty	void	Empties the cart
3	checkEligibilityOfRushDeliveryOfChosenItems	boolean	Checks if all items are eligible for rush delivery
4	checkAvailabilityOfChosenItems	boolean	Checks if all items have sufficient inventory
5	isEmpty	boolean	Checks if the cart is empty

- ***Parameter:***

- ***checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean***
    - ***province: The name of the province where the delivery is requested.***
    - ***district: The name of the district within the province.***
    - ***Return: true if all products are eligible for rush delivery, otherwise false.***

Exception: None

- ***Method:***

How to use parameters/attributes:

- **calculateTotalPrice(): float**
    - **Purpose: Computes the total price for the cart.**
    - **Usage: Iterates through all cart items, sums their prices, applies VAT, and returns the total price.**
- **empty(): vold**
    - **Purpose: Empties the cart by removing all items.**
    - **Usage: All items are cleared, and subtotal, VAT, and total price are reset to zero.**
- **checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean**
    - **Purpose: Determines if all products in the cart are eligible for rush delivery.**
    - **Usage: Checks each product's delivery eligibility and returns true if all qualify, otherwise false.**
- **checkAvailabilityOfChosenItems(province: String, district: String): boolean**
    - **Purpose: Checks if all items in the cart have sufficient stock.**
    - **Usage: Iterates through the chosen cart items and verifies stock availability. Returns true if all items are available, otherwise false.**
- **isEmpty(): boolean**
    - **Purpose: Checks if the cart is empty.**
    - **Usage: Returns true if there are no items in the cart, otherwise false.**

- **State:**

- **If cartItemsList is empty → Empty**
- **If cartItemsList contains at least one item → Active**
- **If checkout() is called and payment is not completed → CheckedOut**
- **If payment is successful → Paid**
- **If cart is emptied (either manually or after order completion) → Empty**

4.4.3.12 Class Design: CartItem

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	Id of the product in the cart
2	quantity	int	0	Quantity of the product
3	cartId	int	null	Unique Identifier for the cart
4	cartItemId	int	null	Unique Identifier for the cart item

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	changeQuantityOfProducts.	void	Updates the quantity and recalculates subtotal
2	addProductToCart	void	Adds a product to the cart with specified quantity

- ***Parameter:***

- ***addProductToCart(product: Product, quantity: int): volds***
  - ***product: A Product object representing the product to add.***
  - ***quantity: An integer representing the number of products to add.***
  - ***Return: None.***
- ***changeQuantityofProduct(product: Product, quantity: int): vold***
  - ***product: A Product object representing the product to update.***
  - ***quantity: An integer representing the new quantity.***
  - ***Return: None.***

- **Exception:**

  - **addProductToCart(productId: int, quantity: int): vold**
    - **ProductNotFoundException: Raised if the specified product does not exist in the system.**
    - **InsufficientStockException: Raised if the requested quantity exceeds the available stock.**
  - **changeQuantityOfProduct(productId: int, quantity: int): vold**
    - **ProductNotFoundException: Raised if the product is not found in the cart.**
    - **InsufficientStockException: Raised if the requested quantity exceeds the available stock.**

- ***Method:***

  How to use parameters/attributes:

  - **addProductToCart(ProductId: int):**
    - **Purpose: Adds a product to the cart with a specified quantity if stock is sufficient.**
    - **Usage: If the product exists and the requested quantity is available, it is added to the cart. Otherwise, InsufficientStockException is raised.**
  - **changeQuantityofProduct(ProductId: int):**
    - **Purpose: Updates the quantity of an existing product in the cart.**
    - **Usage: If the product is already in the cart and the stock allows, the quantity is updated. Otherwise, InsufficientStockException is raised.**

- **State**

- **Available :If stock > requested quantity, the product is available for purchase.**
- **Not Available :If stock < requested quantity, the requested quantity cannot be fulfilled.**
- **Product Not Found:If product_Id is invalId (i.e., the product does not exist in the system), a ProductNotFoundException is raised**

4.4.3.13Design Class: PaymentTransaction

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	int	null	Unique Identifier for the transaction
2	amount	float	0.0	Payment amount
3	paymentMethod	String	"Credit Card"	Method of payment
4	transactionDate	Date	current date	Date of transaction
5	status	String	"Pending"	Transaction status (Pending, Success, Failed, Refunded)
6	content	String	null	Transaction details or description
7	orderId	int	null	Id of the associated order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	evalutePaymentResult	boolean	Processes the payment result from VNPay
2	save	vold	Saves the transaction to the database
3	transactionInfo	String	Returns formatted transaction information

4	saveRefundTransaction	vold	Saves refund transaction information
5	createTransaction	vold	Creates (stores) a new payment transaction in the system

- **Parameter:**
    - **transactionInfo(transactionId: String) : PaymentTransaction**
        - **transactionId: An integer representing the unique Id of the transaction.**
        - **Return: A PaymentTransaction object containing the transaction details.**

- **Exception:**
    - **TransactionNotFoundException if referencing a non-existent transaction.**
    - **InvalIdTransactionAmountException – If the amount is invalId (e.g., negative or zero when it shouldn't be).**

- **Method:**
    - **save()**
        - **Purpose: Commits the current transaction state to persistent storage.**
        - **Usage: Called after creating or updating any transaction details (e.g., setting the status to "completed").**
    - **transactionInfo(transactionId: String): PaymentTransaction**
        - **Purpose: Retrieves the details of a specific payment transaction by its unique Id.**
        - **Usage: If transactionId exists, returns a PaymentTransaction object. If it does not exist, raises TransactionNotFoundException.**

- **State:**

- **If transactionId is invalId → TransactionNotFoundException is raised.**
- **If transactionStatus is "completed" → Payment is successful and no further changes unless a refund is requested.**
- **If processRefund is successful → transactionStatus might be changed to "refunded", and isRefunded is set to true.**

refundTransactionId may be generated or updated when a refund is initiated.

4.4.3.14 Design Class: PlaceOrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentCart	Cart	null	Reference to the customer's current shopping cart
2	currentOrder	Order	null	Order being created in the current process
3	deliveryInfo	Delivery Info	null	Delivery information for the current order
4	currentInvoice	Invoice	null	Invoice generated for the current order

Table 2. Operation design

#	Name	Retur n type	Description (purpose)
1	requestToPlaceOrd er	boolean	Initiates the order placement process with items from cart
2	checkAvailabilityOf ChosenItems	boolean	Verifies if all items in cart are available in sufficient quantity
3	Order	Order	Creates a new order with items from the cart
4	DeliveryForm	Delivery Form	Creates and returns a delivery form for customer input
5	submitDeliveryInfo	boolean	Processes the delivery information submitted by customer

6	checkValIdityOfDeliveryInfo	boolean	ValIdates the delivery information format and completeness
7	DeliveryInfo	DeliveryInfo	Creates a delivery info object from valIdated input
8	calculateShippingFee	float	Calculates shipping fee based on product weight and delivery location
9	Invoice	Invoice	Creates an invoice with order and delivery information
10	placeOrder	boolean	Finalizes the order placement after payment is processed
11	save	vold	Saves the confirmed order to the database
12	empty	vold	Clears the cart after successful order placement
13	sendSuccessOrderEmail	boolean	Sends order confirmation email to customer
14	OrderInfoScreen	OrderInfoScreen	Creates a screen displaying order and transaction details
15	placeRushOrder	boolean	Verify if there exists any products for rush delivery from cart

- **Parameter:**

  - **requestToPlaceOrder(Cart): boolean**
    - **cartObj: The shopping cart containing products to be ordered**
    - **Return: true if all products are available and order can proceed, false otherwise**
  - **checkAvailabilityOfChosenItems(Cart): boolean**
    - **cart: The shopping cart to check availability for**
    - **Return: true if all items are available in sufficient quantity, false otherwise**
  - **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
    - **deliveryInfo: Map containing delivery information fields (name, address, etc.)**
    - **Return: true if delivery information is valId, false otherwise**

- **checkValldityOfDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
  - **deliveryInfo: Map containing delivery information to valldate**
  - **Return: true if all required fields are valld, false otherwise**
- **calculateShippingFee(deliveryInfo: DeliveryInfo, order: Order): float**
  - **deliveryInfo: Customer's delivery information including location**
  - **order: The order containing products with weight information**
  - **Return: The calculated shipping fee in the local currency**
- **placeOrder(transactionInfo: Map<String, String>): boolean**
  - **transactionInfo: Payment transaction details**
  - **Return: true if order was successfully placed, false otherwise**
- **placeRushOrder(province: String, district: String): boolean**
  - **province: entered by a Administrator**
  - **district: entered by a Administrator**
  - **Return: true if there exists any cart items for rush delivery**

- **Exception:**

  - **requestToPlaceOrder(Cart): boolean**
    - **EmptyCartException: Thrown if cart is empty**
    - **InvalldCartException: Thrown if cart contains invalld items**
  - **checkAvailabilityOfChosenItems(Cart): boolean**
    - **DatabaseConnectionException: Thrown if unable to check product availability**
    - **InsufficientStockException: Details which products have insufficient stock**
  - **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
    - **InvalldDeliveryInfoException: Details which fields are invalld or missing**
  - **requestToPlaceOrder(Cart): boolean**
    - **UnsupportedLocationException: Thrown if delivery location is not supported**
    - **InvalldWeightException: Thrown if product weight data is missing or invalld**

- **Method:**

  - **requestToPlaceOrder(cartObj: Cart): boolean**
    - **Purpose: Initiates the order placement process by checking product availability**
    - **Processing**:
      - Valldates that the cart is not empty
      - For each item in the cart:
        - Calls checkProductAvailability on the Product object

            - If any product is unavailable, returns false and provIdes details on unavailable items
        - If all products are available, creates a new Order object
        - Creates and returns a DeliveryForm for customer input
    - **Return:** true if order process can continue, false if any products are unavailable
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
    - **Purpose:** Processes and valIdates delivery information provIded by customer
    - **Processing**:
        - ValIdates all required delivery information fields
        - Creates a DeliveryInfo object with the valIdated information
        - Calculates shipping fee based on product weights and delivery location
        - Creates an Invoice with order details, products, VAT, and shipping fee
        - Returns an InvoiceScreen for customer review
    - **Return:** true if delivery information is valId and processing continues, false otherwise
- **placeOrder(transactionInfo: Map<String, String>): boolean**
    - **Purpose:** Finalizes order placement after successful payment
    - **Processing**:
        - Saves the order to the database with "Pending Processing" status
        - Empties the cart
        - Sends order confirmation email to customer
        - Creates and displays OrderInfoScreen with order and transaction details
    - **Return:** true if order was successfully placed, false if any step fails

- **State:**

    - **Before requestToPlaceOrder is called, currentCart contains the customer's selected products**
    - **After successful requestToPlaceOrder, currentOrder contains a new Order object**
    - **After submitDeliveryInfo, deliveryInfo contains valIdated delivery information and currentInvoice contains the generated invoice**
    - **After successful placeOrder, the order is saved to the database, the cart is emptied, and confirmation email is sent**

4.4.3.15Design Class: SearchController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productList	List<Product>	empty list	List of products in the system
2	searchResults	List<Product>	Empty list	Products matching search
3	maxResultsPerPage	int	20	Maximum number of products displayed per page

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	searchKeyWord	List<Product>	Search for products based on keyword
2	searchProduct	List<Product>	Perform search within product list

- **Parameter:**
    - **searchKeyWord(keyword: String): List<Product>**
        - **keyword: String keyword input from HomeScreen**
        - **Return: List of products matching the keyword**
    - **searchProduct(keyword: String): List<Product>**
        - **keyword: String keyword to compare with product attributes**
        - **Return: List of products matching the keyword**
- **Exception:**

- **searchKeyWord(keyword: String): List<Product>**
  - **EmptyKeywordException: Thrown if search keyword is empty or null**

- **Method:**

searchKeyWord(keyword: String): List<Product>

- **Purpose: This method receives keywords from HomeScreen and processes the search**
- **Processing**:
  - **Valldate keyword**
  - **Call searchProduct method with provlded keyword**
  - **Return search results limited by maxResultsPerPage**
- **Return: List of products matching the keyword, limited to 20 products per page**
- **searchProduct(keyword: String): List<Product>**
  - **Purpose: Perform product search based on keyword**
  - **Processing**:
    - **Iterate through the product list**
    - **Compare keyword with product attributes (name, description, type, etc.)**
    - **Collect matching products**
  - **Return: List of all products matching the keyword**

- **State:**

  - **When no search has been performed, searchResults is an empty list**
  - **After search, searchResults contains products matching the keyword**
  - **If no matching products are found, searchResults is an empty list**

4.4.3.16 Design Class: Invoice

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	invoiceId	int	null	Unique Identifier for the invoice
2	order	Order	null	Order associated with this invoice

3	issueDate	Date	current date	Date when invoice was created
4	productList	List<OrderItem>	empty list	List of products in the invoice
5	subtotal	float	0.0	Total price excluding VAT
6	VAT	float	0.0	Value Added Tax amount (10% of subtotal)
7	totalWithVAT	float	0.0	Subtotal + VAT
8	regularDeliveryFee	float	0.0	Fee for regular delivery items
9	rushDeliveryFee	float	0.0	Fee for rush delivery items
10	totalAmount	float	0.0	Final amount to be paid
11	paymentMethod	String	"Credit Card"	Method of payment
12	recipientInfo	DeliveryInfo	null	Delivery information of the recipient
13	VAT_RATE	float	0.1	Value-added tax rate (constant 10%)

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	createInvoice	Invoice	Creates a new invoice with order and delivery information
2	calculateSubtotal	float	Calculates the subtotal price of all products
3	calculateVAT	float	Calculates the VAT amount based on subtotal
4	calculateTotalWithVAT	float	Calculates the total amount including VAT

5	calculateDeliveryFees	void	Calculates separate regular and rush delivery fees
6	calculateTotalAmount	float	Calculates the final total amount to be paid
7	generateInvoiceDetails	Map<String, Object>	Generates a complete map of invoice details
8	save	void	Saves the invoice to the system
9	getInvoiceById	Invoice	Retrieves an invoice by its Id
10	Invoice	void	Constructor for creating an invoice

- **Parameter:**
    - **createInvoice(cart: Cart) : Invoice**
        - **cart: The user's cart data.**
        - **Return: A fully built Invoice with items, total amounts, and an assigned Id.**

- **Exception:**
    - **InvoiceSaveException – Thrown if saving the invoice fails (database error, disk error, etc.).**
    - **InvalIdCartException – If cart is empty or invalId when calling createInvoice().**

- **Method:**

Invoice() (constructor)

        - **Purpose: Sets up default values (e.g., status = "unpaid").**
        - **Usage: Called internally or externally to create an invoice instance.**
    - **createInvoice(cart: Cart): Invoice**
        - **Purpose: Takes a cart object, calculates subtotal, VAT, and total, sets invoice items, then returns an Invoice.**
        - **Usage: Typically called by InvoiceScreen after the user finalizes the cart.**
    - **save()**
        - **Purpose: Persists the invoice data (items, amounts, status) into a database or storage.**
        - **Usage: Called once the invoice is ready to be stored or updated (e.g., after changing status).**

- **State**

    - **status transitions: "unpaid" → "paid" → possibly "cancelled."**
    - **totalAmountInclVAT = totalAmountExclVAT * 1.1 (assuming 10% VAT).**
    - **finalAmount = totalAmountInclVAT + deliveryFee.**

4.4.3.17 Design Class: DeliveryInfo

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	recipientName	String	null	Name of the recipient
2	email	String	null	Email address
3	phoneNumber	String	null	Phone number
4	province	String	null	Province/city
5	district	String	null	District
6	address	String	null	Delivery address
7	deliveryMethod	String	"Regular"	Delivery method (Regular/Rush)

8	deliveryTime	String	0.0	Requested time for rush delivery
9	isRushDelivery Eligible	boolean	false	Indicates if address is eligible for rush delivery
10	deliveryInstructions	String	null	Special delivery instructions

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DeliveryInfo	void	Constructor for creating delivery information
2	validateDeliveryInfo	boolean	Validates all required delivery information fields
3	isEligibleForRushDelivery	boolean	Calculates shipping fee based on product weight and location
4	calculateShippingFee	float	Calculates shipping fee based on product weight and location
5	calculateRushDeliveryFee	float	Calculates rush delivery fee for eligible products
6	getDeliveryInfo	Map<String, String>	Returns all delivery information as a map
7	updateDeliveryInfo	boolean	Updates the delivery information with new values
8	isInHanoiInnerCity	boolean	Checks if delivery is to inner Hanoi districts

- **Parameter:**

- **DeliveryInfo(recipientName: String, email: String, phoneNumber: String, province: String, district: String, address: String, deliveryMethod: String)**
  - **recipientName: Full name of the recipient**
  - **email: Contact email address**
  - **phoneNumber: Contact phone number**
  - **province: Delivery province/city**
  - **district: Delivery district**
  - **address: Detailed delivery address**
  - **deliveryMethod: Delivery method (Regular or Rush)**
- **validateDeliveryInfo(): boolean**
  - **Return: true if all required fields are valid, false otherwise**
- **isEligibleForRushDelivery(): boolean**
  - **Return: true if the address is in an area that supports rush delivery, false otherwise**
- **calculateShippingFee(weight: float): float**
  - **weight: Total weight of the products in kilograms**
  - **Return: The calculated shipping fee in VND**
- **calculateRushDeliveryFee(numItems: int): float**
  - **numItems: Number of items for rush delivery**
  - **Return: The calculated rush delivery fee in VND**
- **getDeliveryInfo(): Map<String, String>**
  - **Return: Map containing all delivery information fields**
- **updateDeliveryInfo(updatedInfo: Map<String, String>): boolean**
  - **updatedInfo: Map containing updated delivery information fields**
  - **Return: true if update was successful, false otherwise**
- **isInHanoiInnerCity(): boolean**
  - **Return: true if the delivery district is in inner Hanoi city, false otherwise**

- **Exception:**

  - **validateDeliveryInfo(): boolean**
    - **InvalidEmailException: Thrown if email format is invalid**

- **InvalIdPhoneNumberException: Thrown if phone number format is invalId**
- **MissingRequiredFieldException: Thrown if any required field is missing**

- **calculateShippingFee(weight: float): float**
  - **InvalIdWeightException: Thrown if weight is negative or zero**
  - **UnsupportedLocationException: Thrown if delivery location is not supported**

- **Method:**

  valIdateDeliveryInfo(): boolean

- **Purpose: Valldates all required delivery information fields for completeness and format**
- **Processing**:
  - **Checks that recipient name is not empty**
  - **Valldates email format using regex pattern**
  - **Valldates phone number format using regex pattern**
  - **Checks that province, district, and address are not empty**
  - **If delivery method is "Rush", checks that the address is eligible for rush delivery**
- **Return: true if all valldations pass, false otherwise with appropriate error messages**

- **isEligibleForRushDelivery(): boolean**
  - **Purpose: Determines if the delivery address is eligible for rush delivery**
  - **Processing**:
    - **Checks if province is "Hanoi"**
    - **Checks if district is one of the inner city districts of Hanoi**
  - **Return: true if in eligible area, false otherwise**
- **calculateShippingFee(weight: float): float**
  - **Purpose: Calculates the shipping fee based on product weight and delivery location**
  - **Processing**:
    - **Checks if delivery is to inner city Hanoi or Ho Chi Minh City**
      - **If yes, base fee is 22,000 VND for first 3kg**
      - **If no, base fee is 30,000 VND for first 0.5kg**
    - **Adds 2,500 VND for every additional 0.5kg**
    - **Applies free shipping if order value exceeds 100,000 VND (up to 25,000 VND)**
  - **Return: The calculated shipping fee**
- **calculateRushDeliveryFee(numItems: int): float**
  - **Purpose: Calculates additional fee for rush delivery items**
  - **Processing**:
    - **Multiplies number of rush delivery items by 10,000 VND**
    - **Adds this to the base shipping fee**
  - **Return: The calculated rush delivery fee**

- **State:**

- **After initialization, all attributes are set to their default values or to the values provIded in the constructor**
- **After valIdation, the object represents a valIdated delivery information record**
- **If rush delivery is requested, deliveryTime and deliveryInstructions are set, and isRushDeliveryEligible flag is checked**

4.4.3.18 Design Class: OrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentOrder	Order	null	Currently selected order
2	orderList	List<Order>	empty	A list of orders loaded for viewing or processing
3	alreadyProcessed	boolean	false	Indicates if an order has already been processed or canceled

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	approveOrder	vold	Approves a pending order
2	rejectOrder	vold	Rejects a pending order with reason
3	viewOrderList	vold	Retrieves or displays a list of pending orders.
4	getOrderDetails	vold	Gets details for a specific order
5	checkAvailableInventory	boolean	Checks if there's enough inventory for an order
6	processCancel	boolean	Processes the cancellation of an order
7	createOrder	Order	Creates a new order in the system

8	refundConfirm	void	Initiates or confirms a refund operation via VNPay.
9	selectCancel	boolean	Handles user requests to cancel the order.
10	checkAlreadyProcessed	boolean	Checks if the current order is already processed (approved/rejected).
11	getOrder	Order	Returns the current Order object in the controller

- **Parameter**.
    - **checkAlreadyProcessed(orderId: String): boolean**
        - **orderId: The Id of the order to check**
        - **Return: true if the order is already approved or rejected, otherwise false.**
    - **selectCancel(orderId: String): boolean**
        - **orderId: The order to attempt to cancel.**
        - **Return: true if cancellation is selected, false otherwise.**
    - **refundConfirm(orderId: String): void**
        - **orderId: The order to be refunded.**
        - **processCancel(orderId: String): boolean**
    - **ApproveOrder(orderId: String): void**
        - **orderId: The Id of the order to approve**
    - **RejectOrder(orderId: String): void**
        - **orderId: The Id of the order to reject**
    - **processCancel(orderId: String): boolean**
        - **orderId: The Id of the order to finalize cancellation for.**
        - **Return: true if successfully canceled, otherwise false.**
    - **GetOrderDetail(orderId: String)**
        - **orderId: The Id of the order whose details are to be fetched.**
        - **Return: detail information**

- **Exception**
    - **OrderNotFoundException: If the specified orderId does not exist.**
    - **OrderAlreadyApprovedException: If trying to approve/reject/cancel an order that is already processed.**

- **Method**

- **approveOrder(orderId)**
  - **Purpose: Set order status to "approved" and decrement inventory.**
  - **Usage: Called by product manager or automated system upon reviewing an order.**
- **rejectOrder(orderId)**
  - **Purpose: Set order status to "rejected". Possibly triggers a refund if payment was collected.**
- **checkAlreadyProcessed(orderId)**
  - **Purpose: Quickly verify whether an order is no longer in "pending" state.**
- **selectCancel(orderId) and processCancel(orderId)**
  - **Purpose: used in two-step cancellation processes.**

- **State**
  - **If orderStatus is approved or rejected → The order is processed, thus cannot be changed.**
  - **If product is insufficient → approveOrder might either reject the order or throw an exception.**

4.4.3.19 Design Class: User

Description: 'Model' component containing user information as its attributes, using setter/getter operations to create/update or retrieve its attribute. This model is mapped to an equivalent database table.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userId	Integer	null	−randomly auto-generated −exclusive
2	username	String	null	exclusively exists to Identify users using the software.
3	email	String	null	saved when using gmail for registration
4	password	String	null	must be hashed before stored into database

5	role	String	Product Manager	Administrator / Product Manager
6	status	Boolean	True	True as Unblocked, False as Blocked

Table 2. Operation design

#	Name	Return type	Description
1	getUserId	Integer	retrieve 'Id' attribute
2	getUsername	String	retrieve 'username' attribute
3	getEmail	String	retrieve 'email' attribute
4	getPassword	String	retrieve 'password' attribute
5	getRole	String	retrieve 'role' attribute
6	getStatus	Boolean	update 'status' attribute
7	User	constructor	constructor with all mandatory attributes

- **Parameter:**

  - **setter operations** have only a parameter with data type as the corresponding attribute's data type.
  - **getter operations** have no parameters.
  - **User(userId: Integer, username: String, email:**

String, password: String, role: String, status: Boolean)

Exception

- **all setter operations** can throw a **MandatoryFieldException** which avolds blanking fields.

- **Method:**

- **setter methods are implemented by assigning the corresponding attribute to a specific value.**
- **getter methods are implemented by returning the value of the corresponding attribute.**

- **State:**
  - **if any parameter in setter operations is blank, the method can throw 'MandatoryFieldException'.**

4.4.3.20 Design Class: UserService

Description: Service component managing user information in related-business logic.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	users	List<User>	null	list of all users from database
2	currentUser	User	null	current user

Table 2. Example of operation design

#	Name	Return	Description
1	getAllUsers	List<User>	fetch all User instances
2	getUserByUserId	User	fetch User instance by 'userId'
3	getUserByUsername	User	fetch User instance by 'username'
4	checkExistenceOfUsername	Boolean	check if the provIded username exists in AIMS system
5	checkSecurityOfPassword	Boolean	check constraints of password to secure password.

6	checkValidityOfEmail	Boolean	check if the provIded email address exists.
6	saveUser	vold	save a specific user into database

- **Parameter:**
    - **getAllUser(): vold**
        - **Return: true if the available inventory is sufficient, otherwise false.**
    - **getUserByUserId(usedId: Integer): User**
        - **userId: An integer representing the unique Identifier of the user.**
        - **Return: a User instance with a user Id equaling the given parameter.**
    - **checkExistenceOfUsername(username: String): Boolean**
        - **username: a String representing username of the user, entered when create/update user procedures**
        - **Return: return true if username exists on database and false if username does not exist.**
    - **checkSecurityOfPassword(password: String): Boolean**
        - **password: a String representing the password of the user entered when create/update user procedures, need to use the hash algorithm before comparison.**
        - **Return: return true if password matches security constraints, false if not.**
    - **checkExistenceOfEmail(email: String): Boolean**
        - **password: a String representing the email of the user entered when create/update user procedures**
        - **Return: return true if email exists, false if not.**
    - **saveUser(user: User): vold**
        - **user: an User instance which contains user information provIded by 'Administrator.**
        - **Return: vold.**
- **Method**

- **getAllUser(): vold**
  - **Purpose: to fetch all users to update its attribute ‘users’ after each time database updates.**
  - **Usage: connect to database then use fetch command of used DBMS.**
- **getUserByUserId(usedId: Integer): User**
  - **Purpose: to fetch a specific User which is used to display and valldate information.**
  - **Usage: connect to DBMS before executing the retrieve command.**
- **checkIfUsernameExists(username: String): Boolean**
  - **Purpose: used in login, create or update with user information.**
  - **Usage: compare with information contained in User instance.**
- **checkSecurityOfPassword(password: String): Boolean**
  - **Purpose: secure password**
  - **Usage: use several standard password constraints.**
- **saveUser(user: User): vold**
  - **Purpose: used in creating/ updating user information.**
  - **Usage: connect to DBMS before executing update command.**

- **Exception:**

  - **getUserByUserId, getUserByUsername: throwing UserNotFoundException if there is no User with the given ‘userId’.**

State: no specific information.

4.4.3.21 Design Class: UserController

Description: Controller component managing UI display following the related events.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userService	User Servi ce	null	- an interface provlding services to manage related-business logic

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	submitUserInformation	void	- triggered when a user clicks Button to finish entering a user information form.
2	validateUserInformation	Boolean	−subroutine of 'submitUserInformation' −validate user information −display notification on invalid fields
3	notifyUserViaEmail	boolean	−subroutine of 'submitUserInformation' −send email notification to users via EmailManager's services.
4	changeAccountStatus	void	- triggered when a user toggle the status Button on a specific user

- **Parameter:**

  - **submitUserInformation(user: User):**
    - **user**: A User instance containing new user information.
    - **Return**: void.
  - **validateUserInformation(user: User): Boolean**
    - **user**: A User instance containing new user information.
    - **Return**: true if user information is validated, otherwise, notify user to enter again.
  - **notifyUserViaEmail(user: User): void**
    - **user**: A User instance containing new user information.
    - **Return**: void.
  - **changeAccountStatus(user: User): void**
    - **user**: A User instance containing new user information.
    - **Return**: void.

  Exception: no specific information

- **Method:**

- **submitUserInformation(user: User): void**
  - **Purpose: wrap all subroutines from verification and notification after successful creation/ update.**
  - **Usage: called when a user finishes entering user information form**
- **validateUserInformation(user: User): Boolean**
  - **Purpose: verify all provided information**
  - **Usage: using checking services from its 'UserService' attribute.**
- **notifyUserViaEmail(user: User): void**
  - **Purpose: provide user information to log into the AIMS system.**
  - **Usage: using EmailManager to send a notification to a user.**
- **changeAccountStatus(): void**
  - **Purpose: modify status of a user's account from blocked to unblocked and from unblocked to blocked.**
  - **Usage: be called when you toggle Button 'status'**

- **State:**

  - **submitUserInformation(user: User): void**
    - **if 'validateUserInformation' succeeds, go through and 'notifyUserViaEmail'.**

4.4.3.22 Design Class: DashboardView

Description: View component managing UI components and listening to events to perform action provided by Controller component.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	tableView	javafx.scene.control.TableView <User>	null	display a list of all users
2	root	javafx.scene.layout.BorderPane	null	root UI component
3	userController	UserController	null	corresponding Controller

Table 2. Example of operation design

#	Name	Return	Description
1	createTopNav	void	create navigation bar on the top of dashboard interface
2	createSIdeNav	void	create navigation bar on the left sIde of dashboard interface
3	createUserTableView	void	create a table containing a list of all users.
4	createUserInformationForm	void	create a modal display a user information form

Parameter:

- **createTopNav(): void**
- **createSIdeNav(): void**
- **createUserTableView(): void**
- **createUserInformationForm(user: User): void Exception:** no specific information

- **Method:**
  - **createTopNav(): void**
    - **Purpose: initialize top navigation bar including several options.**
    - **Usage: called insIde class constructor.**
  - **createSIdeNav(): void**
    - **Purpose: initialize sIde navigation bar.**
    - **Usage: called insIde class constructor.**
  - **createUserTableView(): void**
    - **Purpose: create a recyclable stage to navigate between multiple contents.**
    - **Usage: called insIde class constructor.**
  - **createUserInformationForm(user: User): void**
    - **Purpose: create UI component to display user information form require user to enter all mandatory information.**
    - **Usage: be called when a user clicks Button to create a new user or update a specific user.**

State: no specific information

4.4.3.23 Design Class: UserInformationForm

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	saveButton	javafx.scene.control.Button	null	button to save entered user information
2	statusSwitch	javafx.scene.control.RadioButton	null	switch used to modify account status
2	labels	List<javafx.scene.control.Label>	null	“Username”, “Password”, …
3	textFields	List<javafx.scene.control.TextField>	null	input components

Table 2 . Operation design

#	Name	Return	Description
1	UserInformationForm	constructor	Constructor which initializes resource to render and handle UserInformationForm modal

- **Parameter:**
    - **UserInformationForm(user: User, userController: UserController)**

    Exception: no specific information
- **Method:**

- **UserInformationForm(user: User, userController: UserController)**
  - **Purpose: initialize a form to edit on.**
  - **Usage: called when you click edit/ create button on 'DashboardView' component.**

State: no specific information

4.4.3.25 Design Class: DeliveryForm

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	checkoutButton	javafx.scene.control.Button	null	button to save entered user information
2	labels	List<javafx.scene.control.Label>	null	"Username", "Password", …
3	textFields	List<javafx.scene.control.TextField>	null	input components
4	placeOrderController	PlaceOrderController	null	Controller managing rush delivery condition

Table 2 . Operation design

#	Name	Return	Description
1	DeliveryForm	constructor	Constructor which initializes resource to render and handle DeliveryForm modal

2	addAddtionalDeliveryInformation	void	display more field when Customer select rush delivery as delivery method

- **Parameter:**

  - **DeliveryForm(user: User, placeOrderController: PlaceOrderController)**
  - **addAddtionalDeliveryInformation(): void**

  Exception: no specific information

- **Method:**

  - **UserInformationForm(user: User, userController: UserController)**
    - **Purpose: initialize a form to edit on.**
    - **Usage: called when you click edit/ create button on 'DashboardView' component.**
  - **addAddtionalDeliveryInformation(): vold**
    - **Purpose: display more field when Customer select rush delivery as delivery method**
    - **Usage: called when Customer selects rush delivery and there exists some items for rush delivery.**

  State: no specific information

4.4.3.25 Design Class: DatabaseManager

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	URL	String	null	url to connect to database system
2	USERNAME	String	null	username to connect to database system

3	PASSWORD	String	null	password to connect database system

Table 2 . Operation design

#	Name	Return	Description
1	getCon nection	java.sql.Connection	get connection before command execution

Parameter:
getConnection(): java.sql.Connection Exception:
getConnection(): java.sql.Connection

- **throw java.sql.SQLException when can not connect to the database system, ensure configuration information and database server activated.**

- **Method:**

  - **getConnection(): java.sql.Connection**
    - **Purpose: used to establish connection between the AIMS application with the database system.**
    - **Usage: called when you need to execute a command in the database system.**

- **State:**

  - **getConnection(): java.sql.Connection**
    - **using 'try-catch' mechanism to avold system corruption.**
    -

4.4.3.26 Design Class: EmailManager

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	STMPHost	String	null	SMTP host
2	STMPPort	String	null	port to send email
3	senderEmail	String	null	email address of sender
4	password	String	null	password equivalent email address of sender

- **Table 2 . Operation design**

#	Name	Return	Description

1	isEmailValId	Boolean	check email format

2	sendEmail	Boolean	send email by this method

Parameter:

- **isEmailValId(email: String): Boolean**
  - **email: email address of sender**
  - **Return: true if email format is matched, otherwise false**
- **send(recipient: String, subject: String, messageBody: String): vold**
  - **email: email address of sender**
  - **Return: vold**

- **Exception:**

  - **send(recipient: String, subject: String, messageBody: String): vold**
    - **throw jakarta.mail.MessagingException when a user can not send email.**

- **Method:**

  - **send(recipient: String, subject: String, messageBody: String): vold**
    - **Purpose: verify email address and send success confirmation when creating/ updating user information.**
    - **Usage: called when notifying the updated user or created user from Controller components.**

- **State:**

  - **send(recipient: String, subject: String, messageBody: String): vold**
    - **using 'isEmailValId' to check before STMP connection establishment**
    - **using 'try-catch' mechanism to avold system corruption.**

4.4.3.27 Design Class: Order

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	Int	"Pending"	Unique Identifier for the order
2	status	String	empty list	Current status: "pending," "approved," "rejected," "canceled," etc.
3	orderItems	List<OrderItem>	empty list	Collection of items (products + quantity) in this order
4	rushOrderItems	List<OrderItem>	empty list	Collection of items eligible for rush delivery
5	regularOrderItems	List<OrderItem>	empty list	Collection of items for regular delivery
6	createdDate	DateTime	current time	Timestamp when the order is first created
7	subtotal	float	0.0	Sum of all product prices excluding VAT
8	vatAmount	float	0.0	Calculated VAT amount (10% of subtotal)
9	regularDeliveryFee	float	0.0	Shipping fee for regular delivery items
10	rushDeliveryFee	float	0.0	Shipping fee for rush delivery items
11	totalAmount	float	0.0	Total amount including VAT and all delivery fees
12	deliveryInfo	DeliveryInfo	null	Delivery information associated with this order
13	paymentTransactionId	int	null	Reference to payment transaction
14	VAT_RATE	float	0.1	Value-added tax rate (constant 10%)

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Order()	void	Constructor for creating a new Order object
2	updateToReject()	void	Sets the order's status to "rejected"
3	updateToApprove()	void	Sets the order's status to "approved"
4	displayApproveStatus()	void	Shows or logs that the order has been approved
5	getOrderDetails	Order	Retrieves/shows the order's item list, price, etc.
6	updateStatus()	void	Updates this order's status based on input
7	save()	void	Persists this order in storage
8	calculateSubtotal	float	Calculates the sum of all product prices excluding VAT
9	calculateVAT	float	Calculates the VAT amount based on subtotal
10	calculateRegularDeliveryFee	float	Calculates shipping fee for regular delivery items
11	calculateRushDeliveryFee	float	Calculates shipping fee for rush delivery items
12	calculateTotalAmount	float	Calculates the total amount including VAT and all delivery fees
13	isEligibleForFreeShipping	boolean	Checks if order is eligible for free regular shipping
14	createOrderListScreen	float	Creates a screen listing multiple orders (for manager UI)
15	getOrderById	Order	Retrieves an order by its Id

- **Parameter**:
    - **updateStatus(newStatus: String): void**
        - **newStatus: The updated status ("pending", "approved", "rejected", "canceled", etc.).**
    - **getOrderById(orderId: int): Order**
        - **orderId: The Identifier of the order to retrieve.**
        - **Return: The Order object if found; otherwise might throw an exception.**
- **calculateSubtotal(ord**

erItems: List<OrderItem>): float

- **orderItems: The list of ordered items to sum up.**
    - **Return: The subtotal of all items excluding VAT.**
- **calculateVAT(subtotal: float): float**
    - **subtotal: The total cost of items before VAT.**
    - **Return: subtotal * VAT_RATE.**
- **isEligibleForFreeShipping(): boolean**
    - **No parameters. Uses internal order data (subtotal, location, etc.).**
    - **Return: true if the order meets free shipping criteria, otherwise false.**

- **Exception:**
    - **OrderNotFoundException**
        - **Thrown if no order exists with the specified orderId when calling methods like getOrderById().**
    - **OrderAlreadyProcessedException**
        - **Thrown if attempting to approve, reject, or cancel an order that has already been processed (i.e., not in "pending").**

- **Method:**
  - **Order() (constructor)**
    - **Purpose: Initializes a new order object with default values (status = "pending", createdDate = current time).**
    - **Usage: Called when the system or user creates an order (e.g., after finalizing a cart).**
  - **updateToReject()**
    - **Purpose: Changes the order's status to "rejected".**
    - **Usage: Called by a manager or system logic if an order cannot be fulfilled (out of stock, other reasons).**
  - **updateToApprove()**
    - **Purpose: Changes the order's status to "approved".**
    - **Usage: Invoked once the manager or system confirms items are in stock and the order is valid.**
  - **save()**
    - **Purpose: Persists this order's current data (status, items, totalAmount, etc.) to a database or file.**
    - **Usage: Called whenever the order state is updated (e.g., after approving or rejecting).**
  - **calculateSubtotal(), calculateVAT(), calculateRegularDeliveryFee(), calculateRushDeliveryFee(), calculateTotalAmount()**
    - **Purpose: Each method breaks down the cost calculation to keep the code organized.**
    - **Usage: Typically chained together after the user finalizes or updates the order's items or delivery method.**
  - **isEligibleForFreeShipping()**
    - **Purpose: Checks rules (like "subtotal > 100,000 VND" or other promotional logic) to see if shipping can be waived.**
    - **Usage: Called during fee calculation to adjust regularDeliveryFee if free shipping applies.**
- **State**
  - **If status = "approved" or "rejected" → The order is consIdered processed and cannot be changed to another status without special rules.**
  - **If orderItems changes → Recalculate subtotal, vatAmount, rushDeliveryFee, regularDeliveryFee, totalAmount.**
  - **If free shipping is applicable → regularDeliveryFee might be set to 0 or partial discount.**
  - **If paymentTransactionId is set → A corresponding payment has been recorded for this order.**

4.4.3.28 Design Class: OrderItem

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	Int	null	Id of the product in the order
2	product	Product	null	Reference to the product object
3	quantity	int	0	Quantity of the product ordered
4	unitPrice	float	0.0	Price of the product at time of order
5	subtotal	float	0.0	Total price for this item (unitPrice × quantity)
6	isRushDeliveryEligible	boolean	false	Whether this item is eligible for rush delivery
7	weight	float	0.0	Weight of one unit of this product

Table 2 . Operation design

#	Name	Return type	Description (purpose)
1	OrderItem	void	Constructor for creating a new OrderItem
2	calculateSubtotal	float	Calculates subtotal for this item
3	getTotalWeight	float	Calculates total weight of this item
4	checkEligibilityForRushDelivery	boolean	Checks if eligible for rush delivery
5	toOrderItemDTO	Map<String, Object>	Converts to data transfer object for display
6	getProductId	String	Returns the product Id
7	getQuantity	int	Returns the quantity
8	getUnitPrice	float	Returns the unit price
9	getSubtotal	float	Returns the subtotal

10	getProduct	Product	Returns the product object

Parameter:
OrderItem(product: Product, quantity: int)
product: The complete Product object

        - **quantity: Integer representing the quantity of this product in the order**
    - **OrderItem(productId: String, quantity: int, unitPrice: float, weight: float, isRushDeliveryEligible: boolean)**
        - **productId: String representing the unique Identifier of the product**
        - **quantity: Integer representing the quantity of this product in the order**
        - **unitPrice: Float representing the price at time of order**
        - **weight: Float representing the weight of one unit**
        - **isRushDeliveryEligible: Boolean indicating if the product is eligible for rush delivery**
    - **calculateSubtotal(): float**
        - **Return: Float value representing the subtotal (unitPrice × quantity)**
    - **getTotalWeight(): float**
        - **Return: Float value representing the total weight (weight × quantity)**
    - **checkEligibilityForRushDelivery(): boolean**
        - **Return: Boolean indicating if this item is eligible for rush delivery**
    - **toOrderItemDTO(): Map<String, Object>**
        - **Return: Map containing item details for display**

- **Exception:**

    - **OrderItem(product: Product, quantity: int)**
        - **NullProductException: Thrown if product is null**
        - **InvalIdQuantityException: Thrown if quantity is negative or zero**
    - **OrderItem(productId: String, quantity: int, unitPrice: float, weight: float, isRushDeliveryEligible: boolean)**
        - **InvalIdProductIdException: Thrown if productId is null or empty**
        - **InvalIdQuantityException: Thrown if quantity is negative or zero**
        - **InvalIdPriceException: Thrown if unitPrice is negative**
        - **InvalIdWeightException: Thrown if weight is negative**
    - **getTotalWeight(): float**
        - **MissingWeightDataException: Thrown if weight information is not available**

- **Method:**

- **OrderItem(product: Product, quantity: int)**
    - **Purpose: Creates a new OrderItem from a Product object**
    - **Processing**:
        - ValIdates product and quantity
        - Extracts product details (Id, price, weight)
        - Sets isRushDeliveryEligible based on product.isSupportedPlaceRushOrder
        - Calculates initial subtotal
    - **SIde effects**: A new OrderItem is created with initialized attributes
- **OrderItem(productId: String, quantity: int, unitPrice: float, weight: float, isRushDeliveryEligible: boolean)**
    - **Purpose**: Creates a new OrderItem with specific attributes
    - **Processing**:
        - ValIdates all input parameters
        - Initializes attributes with provIded values
        - Calculates initial subtotal
    - **SIde effects**: A new OrderItem is created with initialized attributes
- **calculateSubtotal(): float**
    - **Purpose**: Calculates the total price for this order item
    - **Processing**: Multiplies the unitPrice by the quantity
    - **Return**: The calculated subtotal
- **getTotalWeight(): float**
    - **Purpose**: Calculates the total weight of this item for shipping calculations
    - **Processing**: Multiplies the weight by the quantity
    - **Return**: The total weight in kilograms
- **checkEligibilityForRushDelivery(): boolean**
    - **Purpose**: Determines if this item is eligible for rush delivery
    - **Processing**: Returns the value of isRushDeliveryEligible attribute
    - **Return**: True if eligible for rush delivery, false otherwise
- **toOrderItemDTO(): Map<String, Object>**
    - **Purpose**: Creates a data transfer object for display or API responses
    - **Processing**:
        - Collects relevant item data into a map
        - Includes product details, quantity, price information
    - **Return**: Map containing all item details

- **State**

- **OrderItem represents a snapshot of a product at the time of order**
- **It stores both the product Id and optionally a reference to the complete Product object**
- **The unitPrice reflects the price at the time of order, which may differ from the current product price**
- **The subtotal is calculated and stored based on quantity and unitPrice**
- **The isRushDeliveryEligible flag determines whether this item can be delivered with rush service**
- **The weight attribute stores the product weight for shipping calculations**

4.4.3.29 Design Class: VNPay

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	apiKey	String	null	VNPay API key
2	gatewayUrl	String	null	The VNPay endpoint (sandbox or production) for payment requests.

Table 2 . Operation design

#	Name	Return type	Description (purpose)
1	processByVNPay()	void	Processes payments via VNPay
2	processRefund()	boolean	Processes refunds via VNPay

- **Parameter:**
  - **ProcessByVNPay(transactionInfo: PaymentTransaction)**
    - **transactionInfo: Information required for VNPay payment.**
  - **processRefund(refundInfo: PaymentTransaction)**
    - **refundInfo: Contains details about the refund (order Id, amount, etc.).**
- **Exception**

- **VNPayConnectionException: If there is a failure to connect or authenticate with VNPay.**
- **VNPayResponseException: If VNPay returns an error response**

- **State**
  - **If the connection to VNPay fails → VNPayConnectionException.**
  - **If VNPay denies the transaction → VNPayResponseException or returns false.**

4.4.3.30 Design Class: ProductController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1				

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	CreateProduct()	vold	Create a new product with provlded information
2	UpdateProduct(productId: int)	vold	Update product with updated information
3	ValldateProduct(productId: String)	boolean	Check the provlded information is valld or not

- **Parameter:**

- **UpdateProduct(productId: int)**
  - **productId must be exist**
- **ValIdateProduct(productId: int)**
  - **productId must be exist**
  - **The input in the form must be valId**

- **Exception:**

  - **UpdateProduct(productId: int)**
    - **ProductNotFoundException: If no product with the specified productId is found.**
  - **ValIdateProduct(productId: int)**
    - **ProductNotFoundException: If no product with the specified productId is found.**
    - **WrongFormatInputException: If input is in wrong format**
    - **PriceOutOfRangeException: If input price is out of range 30%-150% value**

  Method:

  How to use parameters/attributes:

  - **CreateProduct(productId: int)**
    - **Purpose: This method creates a new product with the provIded information.**
    - **Usage: It receives product details from the user or system, then creates a new product and adds it to the product list.**
  - **UpdateProduct(productId: int)**
    - **Purpose: This method updates the details of an existing product based on the provIded information.**
    - **Usage: If the product exists, the method updates attributes such as name, price, quantity, etc**
  - **ValIdateProduct(productId: int)**
    - **Purpose: This method checks whether the provIded product information is valId.**
    - **Usage: It verifies required fields such as title, price, category, and quantity. If the information is valId, the method returns true; otherwise, it returns false (e.g., price out of the allowed range, negative quantity).**

  State:

4.4.3.31 Design Class: PayOrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentInvoice	Invoice	null	A reference to the invoice that is being processed for payment.
2	paymentTransaction	PaymentTransaction	null	A reference to the PaymentTransaction created once payment is initiated or completed.

Table 2. Operation Design

#	Name	Return type	Description (purpose)
1	payOrder	boolean	Initiates payment process for an invoice and returns success/failure
2	paymentFailed	vold	Handles the scenario when the payment fails (e.g., logs, notifies user)
3	savePaymentResult	boolean	Persists the final payment result (success or failure) into the system

- **Parameter:**
    - **payOrder(invoice: Invoice): boolean**
        - **invoice: The Invoice object to be pald**
        - **Return: true if payment initiation or completion is successful, otherwise false**
- **Exception:**
    - **PaymentProcessingException – Could be thrown if an error occurs during payment (e.g., network failure to VNPay, invalld invoice data, etc.).**

- **InvoiceNotFoundException – If the provIded invoice cannot be found or is invalId.**

Method:

How to use parameters/attributes:

- **payOrder(invoice: Invoice): boolean**
  - **Purpose: To process payment for the given Invoice. It may call external boundaries (e.g., VNPayBoundary) to complete the transaction.**
  - **Usage: Called after the invoice is created and the user chooses payment method. If successful, returns true; on error, might throw PaymentProcessingException or return false.**
- **paymentFailed()**
  - **Purpose: Triggered when the payment process fails. This method might notify the user interface and log the failure.**
  - **Usage: Called by the system or boundary when a payment gateway reports an error.**
- **savePaymentResult()**
  - **Purpose: To store the payment outcome (success, failure, transaction details) into the database or system logs.**
  - **Usage: Called once the payment is confirmed or fails, ensuring the final state is recorded.**

- **State**

  - **If payOrder succeeds → The system moves to a "payment completed" state for the Invoice.**
  - **If payOrder fails → paymentFailed() is invoked; the system may keep the invoice state as "unpaId," logs the failure, etc.**
  - **savePaymentResult() typically changes or finalizes the payment record in the database.**

4.4.3.32 Design Class: PlaceOrderService

Table 1. Attribute design

#	Name	Data type	Default value	Description

Table 2. Operation Design

#	Name	Return type	Description (purpose)
1	checkRushDelivery Support	boolean	check rush delivery condition

- **Parameter:**

  - **checkRushDeliveryCondition(province: String, district: String, cart: Cart): boolean**
    - **province, district: the provIded address information**
    - **cart: containing list of CartIem.**
    - **Return: true if payment initiation or completion is successful, otherwise false**

  Exception: no specific information

- **Method:**

  How to use parameters/attributes:

  - **checkRushDeliveryCondition(**
    - **Purpose: to verify rush delivery conditions and mark which one is eligible.**
    - **Usage: used by PlaceOrderController to check address information provIded by UI and mark which CartItem is eligible.**

- **State**

  - **If 'province' is not 'Hanoi', return false**
  - **If 'province' is Hanoi but 'district' is not specified, return false.**
  - **Else iterating through CartItem in the 'Cart' to mark which 'CartItem' is eligible.**

5.Design Considerations

<Describe issues which need to be addressed or resolved before attempting to devise a complete design solution. Remember that, you have to refactor your source code to strictly follow the final design>

● *Goals and Guidelines*

The following goals, guidelines, principles, and priorities dominate the design of the AIMS system and its software components.

● Performance Goals

1. **Response Time Optimization**
    a. **Goal**: Achieve maximum response time of 2 seconds under normal conditions and 5 seconds during peak hours
    b. **Rationale**: E-commerce applications require fast response times to prevent user frustration and cart abandonment
    c. **Implementation**:
        i. Implement caching mechanisms for frequently accessed product data
        ii. Use connection pooling for database operations
        iii. Optimize database queries with proper indexing
        iv. Implement lazy loading for product images
2. **Concurrent User Support**
    a. **Goal**: Support up to 1,000 simultaneous users without performance degradation
    b. **Rationale**: Scale to meet business requirements for multiple simultaneous customers
    c. **Implementation**:
        i. Thread-safe singleton patterns for shared resources
        ii. Session management with minimal memory footprint
        iii. Efficient connection handling and resource management
3. **System Reliability**
    a. **Goal**: 300 hours continuous operation without failure, 1-hour maximum recovery time
    b. **Rationale**: Minimize business disruption and revenue loss during outages
    c. **Implementation**:
        i. Robust exception handling and logging
        ii. Automatic failover mechanisms

iii. Health monitoring and alerting systems

- ## Software Design Principles

1. **Modularity and Maintainability**
    a. **Goal**: Create loosely coupled, highly cohesive components
    b. **Rationale**: Enable independent development, testing, and maintenance of system modules
    c. **Implementation**:
        i. Clear separation between presentation, business logic, and data access layers
        ii. Use of dependency injection for component dependencies
        iii. Interface-based programming for core services
2. **Scalability**
    a. **Goal**: Design for future growth in product catalog, user base, and feature set
    b. **Rationale**: Accommodate business expansion without major architectural changes
    c. **Implementation**:
        i. Abstract factory pattern for different media product types
        ii. Plugin architecture for payment gateway integration
        iii. Configurable business rules and pricing policies
3. **Security First**
    a. **Goal**: Protect sensitive customer and payment information
    b. **Rationale**: Build trust with customers and comply with data protection regulations
    c. **Implementation**:
        i. Token-based authentication for user sessions
        ii. Encrypted password storage using BCrypt
        iii. Secure communication with payment gateways
        iv. Input validation and SQL injection prevention

- ## User Experience Guidelines

1. **Intuitive Navigation**
    a. **Goal**: Minimize learning curve for new users
    b. **Rationale**: Reduce customer support burden and increase conversion rates
    c. **Implementation**:
        i. Consistent UI patterns across all screens
        ii. Clear visual hierarchy and navigation breadcrumbs

iii. Progressive disclosure of complex features

2. **Error Prevention and Recovery**
    a. **Goal**: Prevent user errors and provide clear recovery mechanisms
    b. **Rationale**: Reduce frustration and support requests
    c. **Implementation**:
        i. Real-time input validation with clear error messages
        ii. Confirmation dialogs for destructive actions
        iii. Automatic session recovery mechanisms

● Development Guidelines

1. **Coding Standards**
    a. **Naming Conventions**: CamelCase for classes, camelCase for methods and variables
    b. **Package Structure**: Organized by feature/module (e.g., com.aims.product, com.aims.order)
    c. **Code Documentation**: Javadoc comments for all public classes and methods
    d. **Version Control**: Feature branch workflow with pull request reviews
2. **Quality Assurance**
    a. **Unit Test Coverage**: Minimum 80% coverage for business logic classes
    b. **Integration Testing**: Comprehensive testing of API integrations
    c. **Performance Testing**: Load testing to verify concurrent user requirements
    d. **Security Testing**: Regular vulnerability scans and penetration testing
3. **Database Design Guidelines**
    a. **Normalization**: 3NF normalization with denormalization only for proven performance benefits
    b. **Indexing Strategy**: Indexes on all foreign keys and frequently searched columns
    c. **Transaction Management**: ACID compliance for all financial operations
    d. **Audit Trails**: Complete logging of all data modifications

● Integration Guidelines

1. **External Service Integration**
    a. **Goal**: Seamless integration with third-party services (VNPay, Email)
    b. **Rationale**: Leverage existing services while maintaining system independence
    c. **Implementation**:
        i. Adapter pattern for external service interfaces

ii. Retry mechanisms with exponential backoff
iii. Circuit breaker pattern for fault tolerance

2. **API Design**
    a. **Goal**: Consistent and versioned API interfaces for future integration
    b. **Rationale**: Enable future mobile applications or third-party integrations
    c. **Implementation**:
        i. RESTful principles for internal service communication
        ii. JSON for data exchange with clear schema definitions
        iii. API versioning strategy for backward compatibility

- **Business Logic Guidelines**

1. **Business Rule Centralization**
    a. **Goal**: Centralize business rules in service layer
    b. **Rationale**: Ensure consistency across different user interfaces and future integrations
    c. **Implementation**:
        i. Rule engine pattern for configurable business rules
        ii. Service facade pattern for complex business operations
        iii. Strategy pattern for different pricing and delivery calculations
2. **Transaction Handling**
    a. **Goal**: Ensure data consistency across all business operations
    b. **Rationale**: Prevent data corruption and maintain inventory accuracy
    c. **Implementation**:
        i. Unit of Work pattern for complex operations
        ii. Compensation transactions for external service failures
        iii. Event sourcing for audit and recovery capabilities

These goals and guidelines form the foundation for making design decisions throughout the development process, ensuring the system meets both functional requirements and quality attributes.

- ***Architectural Strategies***

The following architectural strategies provide insight into the key design decisions and abstractions used in the AIMS system architecture.

• Layered Architecture Strategy

Decision: Implement a 3-tier layered architecture with clear separation of concerns **Reasoning**:

- Supports modularity and maintainability goals
- Enables independent development and testing of layers
- Facilitates future enhancements without affecting other layers

Implementation:

- **Presentation Layer**: User interface components (Views, Controllers)
- **Business Logic Layer**: Service classes, business rules, and domain models
- **Data Access Layer**: Repository pattern implementation, database access

Trade-offs:

- Higher initial development complexity vs. long-term maintainability benefits
- Potential performance overhead from layer transitions vs. clean separation of concerns

• Technology Stack Selection

Decision: Java-based technology stack with MySQL database **Reasoning**:

- Platform independence for desktop deployment
- Robust ecosystem with extensive libraries
- Strong threading capabilities for concurrent user support
- Existing team expertise

Components:

- **Programming Language**: Java 17
- **Database**: MySQL 8.0
- **ORM Framework**: Hibernate
- **UI Framework**: JavaFX
- **Build Tool**: Maven
- **Version Control**: Git

Trade-offs:

- Java's memory footprint vs. performance and ecosystem benefits
- MySQL vs. NoSQL databases (chosen for ACID compliance requirements)

● Module-Based Architecture

Decision: Organize system into functional modules with well-defined interfaces **Reasoning**:

- Supports scalability goals for future feature additions
- Enables parallel development by different team members
- Facilitates plugin architecture for payment gateways

Key Modules:

- **Product Management Module**: Catalog, inventory, pricing
- **Order Processing Module**: Cart, checkout, delivery
- **User Management Module**: Authentication, authorization
- **Payment Integration Module**: VNPay interface, transaction handling
- **Notification Module**: Email services, order updates

Communication Strategy: Event-driven architecture within modules, direct method calls between layers

● Data Management Strategy

Decision: Centralized database with distributed caching **Reasoning**:

- Ensures data consistency for financial transactions
- Supports concurrent user requirements through caching
- Simplifies backup and recovery procedures

Implementation Details:

- **Primary Storage**: MySQL database with master-slave replication
- **Caching**: In-memory cache (Ehcache) for product catalog
- **Session Management**: Database-backed session storage
- **Transaction Management**: JTA for distributed transactions

Trade-offs:

- Single database vs. microservices approach (chosen for data consistency)

- Cache complexity vs. performance requirements

● External Integration Strategy

Decision: Adapter pattern for external service integration **Reasoning**:

- Isolates system from changes in external APIs
- Enables easy switching of service providers
- Supports testing with mock implementations

External Services:

- **Payment Gateway**: VNPay Sandbox API
- **Email Service**: JavaMail API
- **Future Extensions**: Google Maps API for delivery tracking

Implementation:

- Interface-based service definitions
- Retry mechanisms with circuit breaker pattern
- Asynchronous processing for non-critical operations

● Concurrency and Synchronization Strategy

Decision: Thread-safe implementations with optimistic locking **Reasoning**:

- Supports 1,000 concurrent users requirement
- Minimizes performance impact from locking
- Prevents race conditions in inventory management

Key Approaches:

- **Inventory Management**: Optimistic locking with version control
- **Session Management**: ThreadLocal storage for user context
- **Database Connections**: Connection pooling (HikariCP)
- **Async Operations**: CompletableFuture for email notifications

Critical Sections:

- Product price updates (limited to 2 per day)
- Order placement with inventory reservation

- Payment transaction processing

● Error Detection and Recovery Strategy

Decision: Comprehensive exception handling with graceful degradation **Reasoning**:

- Meets 1-hour maximum recovery time requirement
- Maintains system availability during partial failures
- Provides clear error reporting for debugging

Implementation:

- **Global Exception Handler**: Catches uncaught exceptions
- **Compensation Transactions**: Rollback for failed operations
- **Circuit Breaker**: Prevents cascade failures in external services
- **Logging Framework**: SLF4J with Logback for audit trails

Recovery Mechanisms:

- Automatic retry for transient failures
- Manual intervention workflows for critical errors
- Data backup and restore procedures

● Security Architecture Strategy

Decision: Token-based authentication with role-based access control **Reasoning**:

- Supports stateless operation for scalability
- Enables fine-grained permission management
- Integrates with existing Java security frameworks

Security Components:

- **Authentication**: JWT tokens with expiration
- **Authorization**: Spring Security with custom role definitions
- **Data Encryption**: BCrypt for passwords, TLS for communication
- **Input Validation**: Bean Validation framework

Security Layers:

- Network security (HTTPS)

- Application security (authentication/authorization)
- Data security (encryption at rest and in transit)

● User Interface Strategy

Decision: JavaFX-based desktop application with MVC pattern **Reasoning**:

- Meets desktop application requirement
- Provides rich UI capabilities for complex interactions
- Enables responsive design for various screen sizes

UI Architecture:

- **Model**: Presentation models with observable properties
- **View**: FXML-based layouts with CSS styling
- **Controller**: Event handlers with business logic delegation

UX Principles:

- Consistent visual language across all screens
- Progressive disclosure for complex workflows
- Real-time validation with user feedback

● Extensibility Strategy

Decision: Plugin architecture for future feature additions **Reasoning**:

- Supports long-term maintenance and enhancement
- Enables third-party integrations without core changes
- Facilitates A/B testing of new features

Extension Points:

- Payment gateway interfaces for new providers
- Delivery service integrations
- Reporting module for business analytics
- Product type extensions (digital media in future)

Configuration Management:

- External configuration files for business rules

- Environment-specific settings management
- Feature flags for gradual rollout

These architectural strategies collectively address the system's requirements while providing a foundation for future growth and maintenance. Each decision balances immediate needs against long-term sustainability, resulting in a robust and scalable e-commerce platform.

● *Design and Program Evaluation*

<Evaluate your design and describe which levels of coupling and cohesion that your design is at. Give proofs for your assumptions. Explain if there is any special design or exceptions>

<You may show the previous design from which you made improvements to get better levels of coupling and cohesion. You should clarify how and why you did these improvements>

<Does your design follow the SOLID principles if there are new requirements/changing requirements in the future? Give proofs for your assumptions. Explain if there is any special design or exceptions>

<You may show the previous design from which you made improvements to get a better design, which follows SOLID principles in spite of additional requirements. You should clarify how and why you did these improvements>

● Cohesion & SRP

#	Class Name	PIC	Cohesion	Single Responsibility	Solution
1	AddProductToCartController	Vu Hai Dang-20225962	Communicational - Methods operate on the same data (carts and products) but serve multiple responsibilities. Contains validation, product checking, and cart modification logic .	No - Handles multiple concerns: 1) Product retrieval and validation 2) Cart retrieval and management 3) Inventory checking.	Extract services: 1) Create CartService with methods like getCart(), addProductToCart() 2) Create ProductService with getProduct(), checkProductAvailability() 3) Make controller focus only on coordinating these services.

2	UpdateCartController	Vu Hai Dang-20225962	Communicational - Similar to AddProductToCartController, duplicates logic but focuses on cart updates. Contains similar validation, retrieval, and modification logic.	No - Contains multiple concerns: 1) Product retrieval, 2) Cart retrieval and management, 3) Quantity validation	Share services: 1) Use the same CartService and ProductService as AddProductToCartController. 2) Remove duplicated getProduct() and getCart() methods. 3)Focus controller solely on update operations logic.
3	ViewCartController	Vu Hai Dang-20225962	Communicational - Methods operate on cart data, but mix display and validation concerns. Contains product availability checking mixed with cart retrieval and display logic	No - Handles mixed concerns: 1) Cart retrieval/display 2) Product availability checks 3) Price calculation.	Separate concerns: 1) Use shared CartService for cart retrieval, 2) Move product availability checking to InventoryService 3) Create dedicated ViewService if needed for display formatting 4) Focus controller solely on coordinating display operations.
4	ViewProductDetailsController	Vu Hai Dang-20225962	Functional - All methods work together with a clear, singular purpose: to retrieve and display product details. Methods are cohesive and focused on one task without mixing responsibilities.	Yes - Focused solely on product detail retrieval and display. The class has a single reason to change: if the way products are retrieved or displayed changes. Uses a clean pattern with requestToViewProductDetails and renderProductDetails.	Maintain design: Already well-designed with good cohesion and SRP compliance.
5	CartItem	Vu Hai Dang-20225962	Functional - All properties and methods focus on representing and managing a single	Yes - Represents and manages a single cart item entity. Contains only data and behaviors related to	No changes needed: Class already demonstrates high cohesion and SRP compliance. Properly

			cart item. Methods like changeQuantityOfProduct directly serve the cart item management purpose. Contains appropriate data and behavior for a cart item.	cart items (ID, quantity, price, product reference). Class would only change if cart item representation needs to change.	encapsulates cart item data and behavior. Could consider making immutable for additional safety if cart item properties shouldn't change after creation.
6	Product (and subclasses like Book, CD, DVD, LPRecord)	Vu Hai Dang-20225962	Functional - Classes properly encapsulate product-specific behavior with appropriate inheritance hierarchy. Each subclass (Book, CD, DVD, LPRecord) focuses on its specific product type while sharing common behavior through the base Product class.	Yes - Each class in the hierarchy has clear responsibilities: base Product handles common attributes/behaviors, while subclasses handle specific product type concerns. Clear separation of concerns through inheritance.	No changes needed: Class hierarchy follows good OO design with proper inheritance and encapsulation. Consider using factory pattern if product creation becomes more complex. Ensure serialization/deserialization is handled appropriately if objects are persisted.
7	Cart	Vu Hai Dang-20225962	Functional - All methods focus on managing cart contents and operations (add, update, remove, calculate). Internal implementation details are encapsulated, and public methods all serve the central purpose of cart management.	Yes - Focused on cart operations with a cohesive set of methods for managing the shopping cart. Contains appropriate validation and operations for a shopping cart entity. Responsibilities are clearly defined around cart management.	No significant changes needed: Well-designed with good cohesion and SRP compliance. Consider adding events/observers if other components need to react to cart changes. Could extract a CartRepository interface if persistence becomes more complex.
8	History	Le Dai Lam -	Functional – All methods	Yes – Focused solely on update history	Solution: No changes needed.

	Update	20225982	(recordUpdate(), getUpdateHistory(), getLastUpdate(), deleteHistory()) operate on the same concept of update history.	management. Handles a single concern: tracking product update activities. The class would only change if the way update history is stored or retrieved changes.	
9	ProductController	Le Dai Lam - 20225982	Communicational – Methods (CreateProduct(), UpdateProduct(), ValidateProduct()) operate on product data but involve two concerns: 1) Coordination 2) Validation logic	No – Handles multiple concerns: 1) Product data validation 2) Business logic coordination (create/update) 3) Error handling and routing. These concerns should be separated into different classes/services.	Solution: 1) Extract a ProductValidationService. 2) Keep the controller focused only on calling services like ProductService.create() and update() after validation.
10	OrderController	Le Dai Lam - 20225982	Procedural – Contains too many responsibilities: approval, cancellation, refund, product checking, and data retrieval.	No – Handles multiple concerns: 1) Order approval and rejection 2) Order cancellation and refund 3) Order display and status rendering 4) Order-product checking logic. The class should be split into focused controllers for each use case.	Solution: 1) Split into OrderApprovalController, OrderQueryController, OrderCancellationController, etc. 2) Each controller should handle only one use case.
11	Order	Le Dai Lam - 20225982	Temporal – Combines UI logic (display methods), business logic (calculations), persistence (save()), and data (fields).	No – Handles mixed concerns: 1) Order data representation (ID, list of items, status) 2) UI logic (display approval status,	Solution: 1) Retain only data fields in Order. 2) Move calculations to OrderService. 3) Move save/get to OrderRepository.

				render order screen) 3) Business logic (subtotal, VAT, shipping calculation) 4) Persistence (save/getById methods). Each of these concerns should be placed in separate components.	4) Move UI logic (e.g. DisplayApproveStatus) to a separate class or view/controller.
12	PaymentTransaction	Nguyen Minh Khoi-20226050	Logical – Class handles data (ID, amount, status), logic (evaluatePaymentResult), and persistence (save), making it a mix of responsibilities.	No – Combines: 1) Transaction data 2) Business logic (VNPay result simulation) 3) Persistence logic (save, saveRefundTransaction)	Split into: 1) PaymentTransaction (pure data) 2) PaymentService (evaluate and manage result) 3) TransactionRepository (save methods)
13	Invoice	Nguyen Minh Khoi-20226050	Logical – Contains pricing data, price/VAT/delivery calculations, persistence, and formatting logic, leading to multiple purposes.	No – Handles: 1) Business data 2) Price calculation logic 3) Persistence (save) 4) Invoice rendering	Split into: 1) Invoice as pure model 2) InvoiceCalculator 3) InvoiceRepository 4) Optional: InvoicePresenter
14	VNPay	Nguyen Minh Khoi-20226050	Functional – All methods and fields support a single responsibility: interacting with the VNPay payment gateway.	Yes – Clearly focused on external payment communication. Has single reason to change: if VNPay interaction changes.	No changes needed.
15	PayOrderController	Nguyen Minh Khoi-20226050	Logical – Orchestrates multiple responsibilities like payment flow, exception handling, persistence, and diagnostic printing.	No – Handles: 1) Payment coordination 2) Persistence (tx/invoice save) 3) Logging & diagnostics	Refactor into: 1) PayOrderController (orchestration only) 2) PaymentService (handles VNPay + transaction result) 3) InvoiceService 4) Move logging to

					separate logger if needed.
16	PlaceRushOrderController	Bui Xuan Son - 2022 6065	Logical - The class handles multiple actions that could be grouped by a control structure or logic (e.g., "if X, validate; if Y, notify"). However, it's an anti-pattern when one class chooses between unrelated operations.	Yes, handle UI event and using routines from 'service' layer to process the needed data	No changes needed.
17	UserControtroller	Bui Xuan Son - 2022 6065	Logical - The class contains logic for different but related operations grouped in the same controller (create vs update logic vs policy checks), decided by code branches or method calls.	Yes, handle UI event and using routines from 'service' layer to process the needed data	No changes needed.
18	DeliveryInfo	Bui Xuan Son - 2022 6065	Functional - All fields and methods work together to represent a single concept: delivery information.	Yes, wrap all attributes of a delivery information and provide methods to initialize, retrieve and update	No changes needed.
19	CartRepository	Bui Xuan Son - 2022 6065	Communicational - The method uses shared data (cartItems, products) to produce output — they are processed together.	No – Handles: 1) Database connection 2) Execute cart-related queries	Build 'utility' class which work on database connection
20	UserRepository	Bui Xuan Son - 2022 6065	Communicational - The methods operate on shared data (users list).	No – Handles: 1) Database connection 2) Execute user-related queries	Build 'utility' class which work on database connection

21	CartService	Bui Xuan Son - 20226065	Functional - The method performs one specific function: mapping cart items to their delivery eligibility.	Yes, using services from 'repository' class to retrieve the needed data, processing them, making them convenient for related 'controller' classes	No changes needed.
22	UserService	Bui Xuan Son - 20226065	Functional - All methods serve the purpose of handling user-related operations.	Yes, using services from 'repository' class to retrieve the needed data, processing them, making them convenient for related 'controller' classes	No changes needed.
23	User	Bui Xuan Son - 20226065	Functional - All fields and methods are directly related to representing a user object.	Yes, wrap all attributes of an user information and provide methods to initialize, retrieve and update	No changes needed.
24	PlaceOrderController	Pham Thanh Nam - 20225989	Functional - All methods focus specifically on place order workflow with clear delegation pattern	No - Handles multiple concerns: 1) Cart validation 2) Delivery info validation 3) Inventory checking 4) Order creation 5) Invoice generation	Solution: 1) Keep the controller focused on orchestration only 2) Extract validation logic to dedicated validators 3) Move invoice generation to a dedicated service 4) Use dependency injection for all services

Table 14. Cohesion & SRP of AIMS

#	*Class Name*	*PIC*	*Cohesion*	*SRP*	*Solution*
1	AddProductToCartController	Vu Hai Dang-20225962	Communicational - Methods operate on the same data (carts	No - Handles multiple concerns:	Extract services: 1) Create CartService with methods like getCart(),

			and products) but serve multiple responsibilities. Contains validation, product checking, and cart modification logic .	1) Product retrieval and validation 2) Cart retrieval and management 3) Inventory checking.	addProductToCart() 2) Create ProductService with getProduct(), checkProductAvailability() 3) Make controller focus only on coordinating these services.
2	UpdateCartController	Vu Hai Dang-20225962	Communicational - Similar to AddProductToCartController, duplicates logic but focuses on cart updates. Contains similar validation, retrieval, and modification logic.	No - Contains multiple concerns: 1) Product retrieval, 2) Cart retrieval and management, 3) Quantity validation	Share services: 1) Use the same CartService and ProductService as AddProductToCartController. 2) Remove duplicated getProduct() and getCart() methods. 3)Focus controller solely on update operations logic.
3	ViewCartController	Vu Hai Dang-20225962	Communicational - Methods operate on cart data, but mix display and validation concerns. Contains product availability checking mixed with cart retrieval and display logic	No - Handles mixed concerns: 1) Cart retrieval/display 2) Product availability checks 3) Price calculation.	Separate concerns: 1) Use shared CartService for cart retrieval, 2) Move product availability checking to InventoryService 3) Create dedicated ViewService if needed for display formatting 4) Focus controller solely on coordinating display operations.
4	ViewProductDetailsController	Vu Hai Dang-20225962	Functional - All methods work together with a clear, singular purpose: to retrieve and display product details. Methods are cohesive and focused on one task without mixing responsibilities.	Yes - Focused solely on product detail retrieval and display. The class has a single reason to change: if the way products are retrieved or displayed changes. Uses a clean pattern with requestToViewProductDetails and renderProductDetails.	Maintain design: Already well-designed with good cohesion and SRP compliance.

5	CartItem	Vu Hai Dang-20225962	Functional - All properties and methods focus on representing and managing a single cart item. Methods like changeQuantityOfProduct directly serve the cart item management purpose. Contains appropriate data and behavior for a cart item.	Yes - Represents and manages a single cart item entity. Contains only data and behaviors related to cart items (ID, quantity, price, product reference). Class would only change if cart item representation needs to change.	No changes needed: Class already demonstrates high cohesion and SRP compliance. Properly encapsulates cart item data and behavior. Could consider making immutable for additional safety if cart item properties shouldn't change after creation.
6	Product (and subclasses like Book, CD, DVD, LP Record)	Vu Hai Dang-20225962	Functional - Classes properly encapsulate product-specific behavior with appropriate inheritance hierarchy. Each subclass (Book, CD, DVD, LPRecord) focuses on its specific product type while sharing common behavior through the base Product class.	Yes - Each class in the hierarchy has clear responsibilities: base Product handles common attributes/behaviors, while subclasses handle specific product type concerns. Clear separation of concerns through inheritance.	No changes needed: Class hierarchy follows good OO design with proper inheritance and encapsulation. Consider using factory pattern if product creation becomes more complex. Ensure serialization/deserialization is handled appropriately if objects are persisted.
7	Cart	Vu Hai Dang-20225962	Functional - All methods focus on managing cart contents and operations (add, update, remove, calculate). Internal implementation details are encapsulated, and public methods all serve the central purpose of cart management.	Yes - Focused on cart operations with a cohesive set of methods for managing the shopping cart. Contains appropriate validation and operations for a shopping cart entity. Responsibilities are clearly defined around	No significant changes needed: Well-designed with good cohesion and SRP compliance. Consider adding events/observers if other components need to react to cart changes. Could extract a CartRepository interface if persistence becomes more complex.

				cart management.	
8	HistoryUpdate	Le Dai Lam - 20225982	Functional – All methods (recordUpdate(), getUpdateHistory(), getLastUpdate(), deleteHistory()) operate on the same concept of update history.	Yes – Focused solely on update history management. Handles a single concern: tracking product update activities. The class would only change if the way update history is stored or retrieved changes.	Solution: No changes needed.
9	ProductController	Le Dai Lam - 20225982	Communicational – Methods (CreateProduct(), UpdateProduct(), ValidateProduct()) operate on product data but involve two concerns: 1) Coordination 2) Validation logic	No – Handles multiple concerns: 1) Product data validation 2) Business logic coordination (create/update) 3) Error handling and routing. These concerns should be separated into different classes/services.	Solution: 1) Extract a ProductValidationService. 2) Keep the controller focused only on calling services like ProductService.create() and update() after validation.
10	OrderController	Le Dai Lam - 20225982	Procedural – Contains too many responsibilities: approval, cancellation, refund, product checking, and data retrieval.	No – Handles multiple concerns: 1) Order approval and rejection 2) Order cancellation and refund 3) Order display and status rendering 4) Order-product checking logic. The class should be split into	Solution: 1) Split into OrderApprovalController, OrderQueryController, OrderCancellationController, etc. 2) Each controller should handle only one use case.

				focused controllers for each use case.	
11	Order	Le Dai Lam - 20225982	Temporal – Combines UI logic (display methods), business logic (calculations), persistence (save()), and data (fields).	No – Handles mixed concerns: 1) Order data representation (ID, list of items, status) 2) UI logic (display approval status, render order screen) 3) Business logic (subtotal, VAT, shipping calculation) 4) Persistence (save/getById methods). Each of these concerns should be placed in separate components.	Solution: 1) Retain only data fields in Order. 2) Move calculations to OrderService. 3) Move save/get to OrderRepository. 4) Move UI logic (e.g. DisplayApproveStatus) to a separate class or view/controller.
12	PaymentTransaction	Nguyen Minh Khoi-20226050	Logical – Class handles data (ID, amount, status), logic (evaluatePaymentResult), and persistence (save), making it a mix of responsibilities.	No – Combines: 1) Transaction data 2) Business logic (VNPay result simulation) 3) Persistence logic (save, saveRefundTransaction)	Split into: 1) PaymentTransaction (pure data) 2) PaymentService (evaluate and manage result) 3) TransactionRepository (save methods)
13	Invoice	Nguyen Minh Khoi-20226050	Logical – Contains pricing data, price/VAT/delivery calculations, persistence, and formatting logic, leading to multiple purposes.	No – Handles: 1) Business data 2) Price calculation logic 3) Persistence (save) 4) Invoice rendering	Split into: 1) Invoice as pure model 2) InvoiceCalculator 3) InvoiceRepository 4) Optional: InvoicePresenter
14	VNPay	Nguyen Minh Khoi-20226050	Functional – All methods and fields support a single responsibility: interacting with the	Yes – Clearly focused on external payment communication. Has single reason	No changes needed.

			VNPay payment gateway.	to change: if VNPay interaction changes.	
15	PayOrderController	Nguyen Minh Khoi-20226050	Logical – Orchestrates multiple responsibilities like payment flow, exception handling, persistence, and diagnostic printing.	No – Handles: 1) Payment coordination 2) Persistence (tx/invoice save) 3) Logging & diagnostics	Refactor into: 1) PayOrderController (orchestration only) 2) PaymentService (handles VNPay + transaction result) 3) InvoiceService 4) Move logging to separate logger if needed.

- **Coupling & Other SOLID**

Explain in detail for each problem in the table. Each problem, you may provide a class diagram before and after the improvement.

Table 15. Coupling & other SOLID of AIMS

#	*Problem*	*PIC*	*Location*	*Solution*
1	Control coupling	TuấnNH	Class A and class B	Create a new class C as a superclass for A and B…
2	<Any bad coupling level or any SOLID violation>	<person in charge>	<list of class or you may provide a class diagram if necessary>	

- ***Design Patterns***

<Do you use any design patterns for your design? If yes, describe detailly why you use those design patterns? Describe in detail on the solutions and how to implement each design pattern>